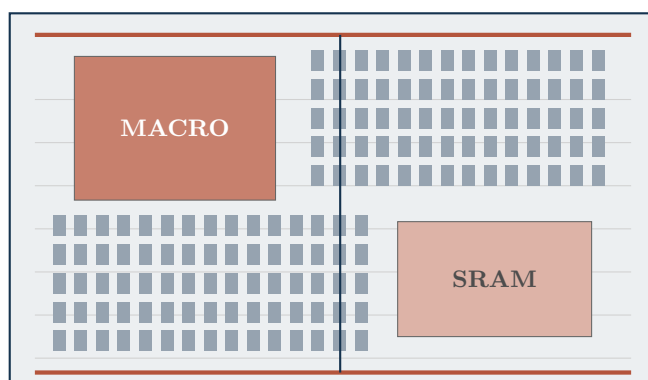


OPTIMIZATION OF PHYSICAL DESIGN

A Complete Preparation Handbook

Theory, Algorithms, Tool Flows, and Worked Problems

for Interviews, Coursework, Sign-off, and Contests
from First Principles to Sign-off



Interview Prep • Academic Study • Sign-off Engineering • VLSI Contests

Floorplan → Power Plan → Placement → CTS → Routing → Sign-off

by A. J. A. Jubayer Talukder

github.com/JubayerONROB · ajajubayertalukder@gmail.com

Edition 1 — Open Source Release

About This Edition

Author. A. J. A. Jubayer Talukder

GitHub. github.com/JubayerONROB

Contact. ajajubayertalukder@gmail.com

This handbook began as preparation material for a single VLSI physical design contest and grew into a general-purpose reference. This first public edition broadens its scope deliberately: the theory, worked examples, and problem bank are written so that the same chapters serve a job-interview candidate revising STA before a screen, a student building a semester’s foundation in physical design, a working engineer chasing down a sign-off ECO, and a contestant with twelve hours on the clock at a hackathon.

License. This work is released under the [Creative Commons Attribution 4.0 International \(CC BY 4.0\)](https://creativecommons.org/licenses/by/4.0/) license. You may share and adapt it for any purpose, including commercially, provided you give appropriate credit. See the LICENSE file in the project repository for the full text.

Corrections and contributions. Errata, clarifications, and additional worked problems are welcome via the project’s GitHub repository. See README.md for contribution guidelines.

How to Use This Book

This handbook grew out of preparation material for a chip design contest whose only announced theme was *optimization of physical design*, with no rulebook released in advance. That origin shaped the book’s structure: rather than covering one narrow path deeply, it covers the entire surface of physical design optimization — every metric you could be judged on, every knob that moves that metric, the algorithm inside the tool that turns the knob, and the arithmetic you need to defend an answer on paper when a whiteboard or a pen-and-paper round is all you have. That same completeness is exactly what makes it useful well beyond a single contest.

Who this book is for

- **Job interviews and technical screenings.** STA derivations, logical effort, the lever tables for placement, CTS, and routing, and the report-reading chapter are the exact material that fills a VLSI physical design interview — both the conceptual questions and the “walk me through fixing this timing path” whiteboard exercises.
- **University coursework and self-study.** Chapters build from first-principles CMOS and RC theory up to tool-level algorithms, with worked examples at every step, so the book functions as a standalone course text or a companion to one.
- **Sign-off and ECO problem solving in real design work.** Part III (the optimization sweep) and the report-interpretation chapter are written to be pulled off the shelf when a real design has a real violation — the diagnostic tables and fix-ordering ladders are usable as-is.
- **Competitive hackathons and VLSI contests** (VLSITHON, Chipathon, and similar). The full problem bank, the rapid-fire question set, and the hour-by-hour contest plan in Chapter 21 remain in place for exactly this use.

The structure

- **Part I — Foundations.** CMOS and standard cells, the library and physical data files, timing theory, and interconnect modelling. Nothing in later parts makes sense without these. If you are already strong here, skim the boxed summaries and the formula recaps.
- **Part II — The Flow, Stage by Stage.** Import, floorplan, power plan, placement, CTS, routing, sign-off. Each chapter states what the stage *decides*, what it *optimizes*, what *breaks* if you get it wrong, and which reports prove it.
- **Part III — The Optimization Sweep.** One chapter per objective: area, timing, power, congestion, signal integrity, IR drop, electromigration, reliability, and finally the multi-objective trade-off problem that is what a contest actually scores.
- **Part IV — Algorithms Inside the Tools.** Partitioning, floorplan representations, placement mathematics, Steiner trees and maze routing, DME clock trees, van Ginneken buffering, logical effort. Knowing these is what separates a contestant who *drives* a tool from one who *pokes* it.

- **Part V — Practice.** Tool command flows for both commercial and open-source stacks, report interpretation, a bank of fully worked contest-style problems from simple to complex, a mini end-to-end case study, a rapid-fire question bank, and cheat sheets.

Reading paths

Semester-length self-study (four weeks): Part I in week 1, Part II in week 2, Part III in week 3, Part V problems throughout, Part IV skimmed in week 4. Chapter 25 lays this out day by day.

Interview prep (one week): Chapters 3, 6, 8, 9, then all of Part III, then the rapid-fire question bank (Chapter 22) and every worked problem in Chapter 21. Rehearse explaining each answer out loud, not just solving it on paper — that is what a screening actually tests.

Sign-off / ECO troubleshooting (as-needed): jump directly to the relevant chapter in Part III for the symptom you are seeing, then the matching diagnostic table in Chapter 20. This book is designed to be used out of order for this purpose — every chapter’s key ideas are boxed so you do not need the surrounding context to use them.

Contest / hackathon, last night before: Chapter 24 formula sheet, then Chapter 21 problems 1–15, then the checklists in Chapter 23, then the hour-by-hour contest plan (Problem 32).

A note on numbers

Every worked example in this book uses *illustrative* technology numbers chosen to make the arithmetic clean and the physics visible. They are not the parameters of any real foundry process. When you get a real library, re-derive everything from its actual `.lib` and `.lef`. The *method* transfers; the constants do not.

Contents

About This Edition	ii
How to Use This Book	iii
I Foundations	1
1 The Design Cycle and Where Optimization Lives	2
1.1 From specification to silicon	2
1.2 The physical design flow	3
1.3 The three axes: PPA	3
1.4 Cost functions you will actually be scored against	3
1.5 Vocabulary you must be fluent in	4
2 CMOS, Standard Cells, and the Library	5
2.1 The MOSFET as a delay element	5
2.2 Anatomy of a standard cell	6
2.3 Drive strength, V_t flavour, and the cell menu	6
2.4 The physical and timing data files	7
2.4.1 Reading a Liberty timing arc	7
2.5 Special cells you must not forget	8
3 Timing Theory and Static Timing Analysis	9
3.1 Why static, not dynamic	9
3.2 The four path groups	9
3.3 Setup and hold, derived properly	9
3.3.1 Setup: data must arrive early enough	10
3.3.2 Hold: data must not arrive too early	10
3.3.3 Worked slack computation	10
3.4 Transition, capacitance, and fanout constraints (DRV)	11
3.5 Corners, PVT, and MMMC	11
3.6 OCV, derating, and advanced variation models	12
3.7 Timing exceptions	12
4 Interconnect: Resistance, Capacitance, and Wire Delay	14
4.1 Why wires became the problem	14
4.2 Metal stack anatomy	14
4.3 Lumped models: L , Π , T , and distributed	15
4.4 Elmore delay	15
4.5 Optimal buffer insertion for a long wire	15
4.6 Coupling capacitance and the Miller factor	16

4.7	Wire load models vs. real extraction	17
II	The Flow, Stage by Stage	18
5	Design Import and Sanity Checks	19
5.1	What you load and in what order	19
5.2	The eight checks that save your run	19
6	Floorplanning	21
6.1	Area arithmetic	21
6.2	Choosing utilization	22
6.3	Aspect ratio	22
6.4	Macro placement — the highest-leverage manual step	22
6.4.1	Rules	23
6.5	Blockages	23
6.6	IO and pin placement	23
6.7	Floorplan quality metrics	24
7	Power Planning	25
7.1	Structure	25
7.2	IR drop	26
7.3	Electromigration	27
7.4	Decap, tap, and endcap insertion order	27
8	Placement	29
8.1	Sub-stages	29
8.2	The objective: wirelength models	29
8.2.1	Half-perimeter wirelength (HPWL)	29
8.2.2	Other models	29
8.3	Timing-driven placement	30
8.4	Congestion-driven placement	30
8.5	Placement-stage optimization moves	31
8.5.1	Pin swapping, concretely	32
8.5.2	Cloning versus buffering	32
8.6	Scan chain reordering	32
8.7	Placement quality checklist	33
9	Clock Tree Synthesis	34
9.1	Definitions, precisely	34
9.2	Tree topologies	35
9.3	The CTS algorithm in outline	35
9.4	Non-default routing rules for clocks	36
9.5	Clock gating	36
9.6	Useful skew, quantitatively	37
9.7	Post-CTS hold fixing	37
10	Routing	39
10.1	Sub-stages	39
10.2	Routing algorithms in brief	40
10.3	The antenna effect	40
10.4	Crosstalk: noise and delta delay	41

10.4.1 Crosstalk noise (glitch)	41
10.4.2 Crosstalk delay (delta delay)	41
10.4.3 Crosstalk mitigation, ranked	42
10.5 Via optimization	42
10.6 Routing DRC categories	42
10.7 Post-route optimization	42
11 Sign-off and ECO	44
11.1 The sign-off matrix	44
11.2 Types of ECO	45
11.3 ECO discipline	45
III The Optimization Sweep	46
12 Area Optimization	47
12.1 What “area” actually means	47
12.2 Levers, ranked by impact	47
12.2.1 1. Architecture and RTL (largest, usually out of scope)	47
12.2.2 2. Synthesis constraints	47
12.2.3 3. Gate downsizing on slack paths	48
12.2.4 4. Buffer removal	48
12.2.5 5. Utilization increase	48
12.2.6 6. Floorplan shape and macro packing	48
12.2.7 7. Library choice	48
12.3 Area optimization procedure	49
13 Timing Optimization	50
13.1 The complete lever list	50
13.2 Logical effort: sizing without guessing	50
13.3 Ordering your timing fixes	52
13.4 TNS vs. WNS: a strategic distinction	52
13.5 Hold optimization	53
13.6 Common timing pathologies and their signatures	54
14 Power Optimization	55
14.1 The three components	55
14.2 Leakage mechanisms	55
14.2.1 Stacking effect	55
14.3 The technique catalogue	55
14.4 Power gating anatomy	56
14.5 Multi- V_t optimization as a knapsack problem	57
14.6 Power analysis accuracy	58
14.7 A complete power optimization worked example	59
15 Congestion and Routability Optimization	61
15.1 Definitions and measurement	61
15.2 Root causes, in the order you should check them	61
15.3 Fixes, ranked by cost	62
15.4 High-fanout net synthesis	62
15.5 Pin access at advanced nodes	63
15.6 Congestion vs. timing: the tension	63

16 Signal Integrity, IR Drop, and Reliability	64
16.1 The reliability checklist	64
16.2 Timing windows: the reason SI analysis is iterative	64
16.3 Decoupling capacitance sizing	65
16.4 Aging and margin	65
17 Multi-Objective Optimization and the PPA Trade-off	66
17.1 Why single-objective thinking loses	66
17.2 Pareto fronts	66
17.3 Design of experiments: sweeping systematically	67
17.4 A scoring-aware decision procedure	68
IV Algorithms Inside the Tools	69
18 Partitioning, Floorplan Representations, and Placement Mathematics	70
18.1 Why you should know these	70
18.2 Partitioning	70
18.2.1 The problem	70
18.2.2 Kernighan–Lin (KL)	70
18.2.3 Fiduccia–Mattheyses (FM)	70
18.2.4 Multilevel partitioning (hMETIS and successors)	71
18.3 Floorplan representations	71
18.3.1 Polish expressions	72
18.3.2 Sequence pair decoding	72
18.4 Placement algorithms	73
18.4.1 Simulated annealing (TimberWolf lineage)	73
18.4.2 Min-cut / recursive bisection (Capo)	73
18.4.3 Quadratic placement	73
18.4.4 Nonlinear / analytical placement (the modern standard)	74
18.4.5 Legalization	74
18.4.6 Detailed placement	75
18.5 Routing algorithms	75
18.5.1 Lee’s maze router	75
18.5.2 Steiner trees	75
18.5.3 Channel routing — the left-edge algorithm	76
18.5.4 Negotiated congestion (PathFinder)	76
18.6 Clock tree algorithms	77
18.6.1 Deferred Merge Embedding (DME)	77
18.6.2 Zero-skew merge point	77
18.7 Van Ginneken buffer insertion	78
18.8 Complexity summary	79
V Practice	80
19 Tools, Flows, and Scripts	81
19.1 The tool landscape	81
19.2 An annotated Innovus-style flow	81
19.3 The equivalent OpenLane configuration	84
19.4 A sweep harness	84

20 Reading Reports Like an Engineer	86
20.1 A timing report, dissected	86
20.1.1 How to read it in 30 seconds	86
20.1.2 The fix, with estimated value	87
20.2 A congestion report	87
20.3 A power report	87
20.4 A DRC report	88
20.5 Metrics table you should maintain for every run	89
21 The Problem Bank: Simple to Complex	90
21.1 Tier 1 — Fundamentals	90
21.2 Tier 2 — Linked Reasoning	92
21.3 Tier 3 — Multi-Constraint Optimization	99
22 Rapid-Fire Question Bank	109
22.1 Block A — Fundamentals	109
22.2 Block B — Timing	109
22.3 Block C — Floorplan, Power, Placement	110
22.4 Block D — CTS, Routing, Power	111
22.5 Block E — Algorithms and Strategy	112
23 Checklists	114
23.1 Before you start	114
23.2 Floorplan	114
23.3 Power plan	114
23.4 Placement	115
23.5 CTS	115
23.6 Routing	115
23.7 Sign-off	116
23.8 The thirty-minutes-left checklist	116
24 Formula and Concept Cheat Sheet	117
24.1 Timing	117
24.2 Interconnect	117
24.3 Logical effort	117
24.4 Power	118
24.5 Physical / floorplan	118
24.6 Algorithms	118
24.7 Quick reference numbers	119
24.8 Decision heuristics	119
25 A Four-Week Preparation Plan	120
25.1 Week 1 — Foundations and a working flow	120
25.2 Week 2 — The flow in depth	120
25.3 Week 3 — Optimization	121
25.4 Week 4 — Algorithms, speed, and rehearsal	121
25.5 If you are on a team	121
26 Glossary	123
Closing Note	125

Part I

Foundations

CHAPTER 1

The Design Cycle and Where Optimization Lives

1.1 From specification to silicon

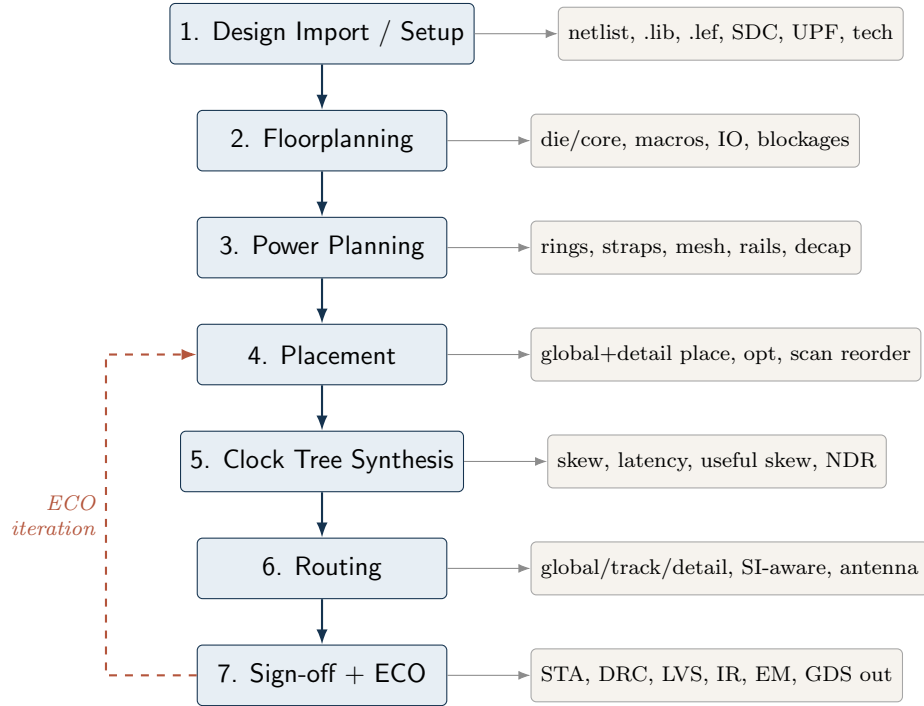
A digital chip travels a long road before it becomes a mask set. The road is conventionally divided as follows.

1. **Specification.** Function, throughput, clock frequency, power budget, area budget, process node, package, operating conditions.
2. **Architecture / microarchitecture.** Pipeline depth, memory hierarchy, bus widths, clock domains. Most of the eventual PPA is decided here, before a single gate exists.
3. **RTL design.** Verilog/VHDL/SystemVerilog description of behaviour.
4. **Functional verification.** Simulation, formal, emulation.
5. **Logic synthesis.** RTL $\rightarrow$ gate-level netlist mapped to a standard cell library, under timing/area/power constraints.
6. **Design for test (DFT).** Scan chain insertion, MBIST, boundary scan, compression.
7. **Physical design (PD).** Netlist $\rightarrow$ GDSII layout. This is our subject.
8. **Physical verification and sign-off.** DRC, LVS, ERC, antenna, STA, IR/EM, DFM.
9. **Tape-out, mask making, fabrication, packaging, test.**

The one-sentence definition

Physical design is the process of deciding *where every transistor sits* and *how every wire runs*, such that the resulting geometry is manufacturable, functionally identical to the netlist, and meets its timing, power, and area targets simultaneously.

1.2 The physical design flow



1.3 The three axes: PPA

Every optimization decision in physical design is a movement in a three-dimensional space.

Axis	Measured by	You usually buy it with
Performance	WNS, TNS, f_{max}	Area (bigger cells), power (low- V_t , more buffers)
Power	Total = dynamic + short-circuit + leakage	Performance (high- V_t , downsizing), area (gating logic)
Area	Die size, core utilization, cell count	Performance (congestion, longer wires), effort

The competition insight

A contest almost never asks “make it fast.” It asks “make it fast *subject to* an area cap” or “minimize power *at this frequency*.” The winner is not the team that pushes one axis hardest — it is the team that finds the *knee* of the trade-off curve and can explain why it is the knee.

1.4 Cost functions you will actually be scored against

Most PD competitions and most internal design reviews score some weighted combination of:

$$\mathcal{C} = w_1 \cdot \frac{A}{A_{ref}} + w_2 \cdot \frac{P}{P_{ref}} + w_3 \cdot \text{TNS}^- + w_4 \cdot \frac{L_{wire}}{L_{ref}} + w_5 \cdot N_{DRC} + w_6 \cdot N_{LVS}$$

with the hard constraint that $N_{DRC} = N_{LVS} = 0$ and $\text{WNS} \geq 0$, otherwise the entry is disqualified or heavily penalized.

Rule zero

A design with brilliant timing and 3 DRC violations scores *zero* in most rulebooks. Clean first, optimal second. Budget the last 20% of your contest time for closing violations, not for chasing another 50 ps.

1.5 Vocabulary you must be fluent in

Term	Meaning
Standard cell	Pre-characterized logic gate of fixed height, variable width.
Macro	Large hard block: SRAM, PLL, analog IP, ROM. Placed manually.
Site / row	The unit grid a standard cell snaps to; rows tile the core.
Utilization	$(\text{cell area} + \text{macro area}) / \text{core area}$.
Pitch	Centre-to-centre spacing of routing tracks on a metal layer.
Slack	Required time – arrival time. Negative slack is a violation.
WNS / TNS	Worst / total negative slack over all endpoints.
Skew	Difference in clock arrival time between two flops.
Insertion delay	Clock source to clock pin latency.
Congestion	Demand for routing tracks over supply, per region.
Antenna	Charge accumulation on a partially-built net during fabrication.
IR drop	Voltage lost across the power grid resistance.
EM	Electromigration: metal atom transport under high current density.
ECO	Engineering Change Order: a late, surgical netlist/layout edit.

CHAPTER 2

CMOS, Standard Cells, and the Library

2.1 The MOSFET as a delay element

For physical design purposes you need only a compact view of the transistor. In the saturation region the drain current of a MOSFET is

$$I_D = \frac{1}{2} \mu C_{ox} \frac{W}{L} (V_{GS} - V_{th})^\alpha$$

with α between 1 and 2 in modern short-channel devices (the α -power law). Three consequences drive everything:

1. **Current scales with W/L .** Wider transistor $\Rightarrow$ more drive current $\Rightarrow$ faster charging of the load $\Rightarrow$ shorter delay. This is why *upsizing* fixes timing.
2. **Current falls as V_{th} rises.** A high- V_t device is slower.
3. **Leakage rises exponentially as V_{th} falls:** $I_{sub} \propto e^{-qV_{th}/nkT}$. This is why low- V_t cells are a *leakage* liability, and why multi- V_t optimization exists.

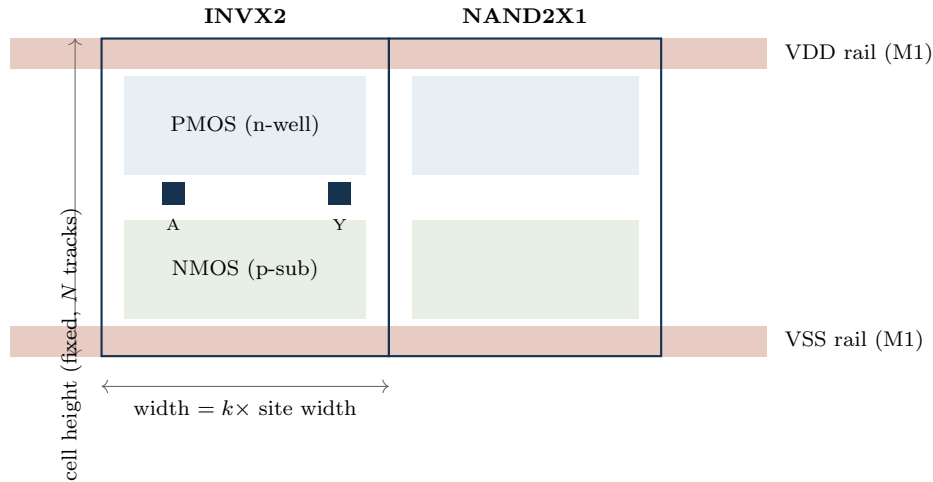
The first-order gate delay model that matters:

$$t_{pd} \approx \frac{C_{load} V_{DD}}{2 I_{D,sat}} \propto \frac{C_{load}}{W}$$

The single most useful sentence in physical design

Delay is load divided by drive. Every timing fix is either (a) reduce the load, (b) increase the drive, or (c) split the load between two drives. Buffering, sizing, cloning, and net splitting are all instances of this one idea.

2.2 Anatomy of a standard cell



Key structural facts:

- **Height is constant** across the library so that cells abut in rows and share power rails. Height is quoted in *routing tracks*: a 9-track library is taller/faster/leakier than a 7-track library.
- **Width is a multiple of the site width**, which is why placement legalization is a discrete, not continuous, problem.
- **Rows alternate orientation** (flipped vertically) so VDD rails abut VDD and VSS abuts VSS, halving the number of rails.
- **Pins are on M1 (and sometimes M2)**. Pin accessibility — whether the router can reach a pin at all — is a first-order routability problem at advanced nodes.

2.3 Drive strength, V_t flavour, and the cell menu

A library offers the same logical function many times over:

Cell	Rel. area	Rel. delay	Rel. leakage	Input cap
INV_X1_HVT	1.0	1.80	0.15	1.0
INV_X1_SVT	1.0	1.00	1.00	1.0
INV_X1_LVT	1.0	0.72	6.50	1.05
INV_X2_SVT	1.7	0.62	1.95	1.9
INV_X4_SVT	3.1	0.41	3.80	3.7
INV_X8_SVT	6.0	0.32	7.40	7.3

Read this table as an optimization map

Moving *down* the size column buys delay with area and leakage, and *costs your driver* because input capacitance rises — the load moves one stage upstream. Moving *across* the V_t column buys delay with leakage only, at zero area cost. That is why V_t **swapping is the cheapest timing/leakage knob in the entire flow** and is always the first ECO tried.

2.4 The physical and timing data files

File	Full name	Contains
.lib	Liberty	Timing arcs, transition/delay tables (NLDM) or current source models (CCS/ECSM), input capacitance, leakage per state, setup/hold constraints, power tables. One per PVT corner.
.lef	Library Exchange Format	<i>Tech LEF</i> : layers, pitches, widths, spacing, via rules, units. <i>Cell LEF</i> : cell size, pin shapes, obstructions (blockages inside the cell).
.def	Design Exchange Format	The design's physical state: die area, rows, component placements, nets, special nets, routing.
.v	Verilog netlist	Gate-level connectivity.
.sdc	Synopsys Design Constraints	Clocks, IO delays, exceptions, transition/capacitance limits.
.spef	Standard Parasitic Exchange Format	Extracted R and C per net, per corner.
.upf/.cpf	Unified/Common Power Format	Power domains, switches, isolation, level shifters, retention.
.gds/.oas	GDSII / OASIS	Final mask geometry.

2.4.1 Reading a Liberty timing arc

A non-linear delay model (NLDM) arc is a 2-D lookup table indexed by *input transition* and *output load*:

Listing 2.1: Excerpt of a Liberty cell definition

```

1  cell (INVX1) {
2      area : 1.596 ;
3      pin (A) { direction : input;  capacitance : 0.0021; }
4      pin (Y) {
5          direction : output;
6          function : "!A";
7          timing () {
8              related_pin : "A";
9              timing_sense : negative_unate;
10             cell_fall (delay_template_5x5) {
11                 index_1 ("0.010, 0.030, 0.070, 0.150, 0.310"); /* input slew ns */
12                 index_2 ("0.001, 0.005, 0.012, 0.030, 0.070"); /* output cap pF */
13                 values ( "0.014, 0.026, 0.049, 0.104, 0.230", \
14                         "0.019, 0.031, 0.054, 0.109, 0.235", \
15                         "0.028, 0.041, 0.064, 0.119, 0.245", \
16                         "0.045, 0.058, 0.081, 0.136, 0.262", \
17                         "0.081, 0.094, 0.117, 0.172, 0.298" );
18             }
19         }
20     }
21 }
```

Micro-exercise

From the table above: input slew = 0.030 ns, output load = 0.012 pF where pF denotes picofarads. What is the fall delay?

Answer: row 2, column 3 $\Rightarrow$ 0.054 ns = 54 ps. Now double the load to 0.030 pF: delay becomes 0.109 ns. Doubling load roughly *doubled* the delay — confirming $t_{pd} \propto C_{load}$.

2.5 Special cells you must not forget

Cell type	Purpose and placement rule
Filler / decap	Fill gaps in rows to continue n-well and implant layers; decaps add local charge reservoir against dynamic IR drop. Inserted <i>after</i> all placement/optimization.
Tap (well tie)	Tie n-well to VDD and substrate to VSS to prevent latch-up. Placed on a fixed pitch (e.g. every 30 μm) across all rows.
Endcap / boundary	Placed at row ends and against macros to satisfy well/implant edge rules.
Tie-hi / tie-lo	Provide logic 1/0 to constant inputs without connecting the gate directly to a rail (gate-oxide reliability).
Antenna diode	Discharges accumulated charge on long partially-routed nets.
Level shifter	Crosses a voltage domain boundary.
Isolation cell	Clamps outputs of a powered-down domain.
Retention flop	Keeps state through power-down via an always-on latch.
Clock gating cell (ICG)	Integrated latch + AND, glitch-free clock enable.
Spare cells	Unconnected gates scattered for post-route metal-only ECOs.

Common contest mistake

Forgetting tap cells or endcaps produces a design that *routes perfectly* and *fails DRC catastrophically* at sign-off. Insert them immediately after floorplan, before placement. Forgetting fillers leaves base-layer DRC errors that appear only in the final GDS check — too late to fix.

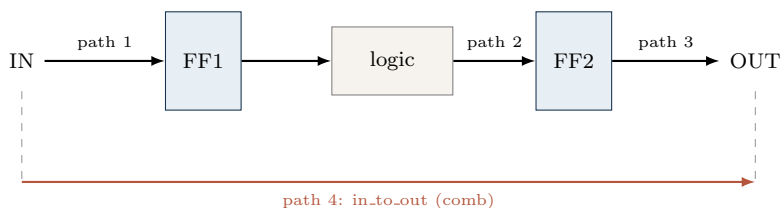
CHAPTER 3

Timing Theory and Static Timing Analysis

3.1 Why static, not dynamic

Simulating every input vector to verify timing is exponentially infeasible. Static Timing Analysis (STA) instead computes, for every timing path, the *worst possible* arrival time regardless of vector, and compares it to the requirement. It is vector-less, exhaustive over paths, and fast — at the cost of being pessimistic and unable to see false paths unless you tell it.

3.2 The four path groups



1. **Input to register (in2reg)** — constrained by `set_input_delay`.
2. **Register to register (reg2reg)** — the internal paths; usually the hardest and the ones you optimize.
3. **Register to output (reg2out)** — constrained by `set_output_delay`.
4. **Input to output (in2out)** — pure combinational feed-through.

3.3 Setup and hold, derived properly

Consider a launch flop FF1 and a capture flop FF2 on the same clock of period T . Define:

T_{cq}	clock-to-Q delay of launch flop
T_{comb}	combinational logic delay between them
T_{setup}	setup requirement of capture flop
T_{hold}	hold requirement of capture flop
T_{cl1}, T_{cl2}	clock insertion delay to FF1, FF2
$T_{skew} = T_{cl2} - T_{cl1}$	skew (positive = capture clock arrives later)
T_{unc}	clock uncertainty (jitter + margin)

3.3.1 Setup: data must arrive early enough

$$\text{Data arrival} = T_{cl1} + T_{cq} + T_{comb} \quad (3.1)$$

$$\text{Data required} = T + T_{cl2} - T_{setup} - T_{unc} \quad (3.2)$$

$$\boxed{\text{Slack}_{setup} = T + T_{skew} - T_{setup} - T_{unc} - T_{cq} - T_{comb}} \quad (3.3)$$

3.3.2 Hold: data must not arrive too early

$$\text{Data arrival} = T_{cl1} + T_{cq,min} + T_{comb,min} \quad (3.4)$$

$$\text{Data required} = T_{cl2} + T_{hold} + T_{unc,hold} \quad (3.5)$$

$$\boxed{\text{Slack}_{hold} = T_{cq,min} + T_{comb,min} - T_{skew} - T_{hold} - T_{unc,hold}} \quad (3.6)$$

The asymmetry that governs your entire strategy

Setup slack contains T ; hold slack does not. Therefore:

- **Setup violations can be fixed by slowing the clock.** Hold violations cannot — they are frequency-independent.
- **Positive skew helps setup, hurts hold.** Negative skew does the reverse. This is the entire basis of *useful skew*.
- Setup is checked on the *slow* corner (max delay); hold on the *fast* corner (min delay).
- Setup fixes make cells *bigger/faster*; hold fixes make paths *longer* (insert delay buffers). They fight each other.

3.3.3 Worked slack computation

Example 3.1 — Basic slack

$T = 1.000$ ns, $T_{cq} = 0.090$ ns, $T_{comb} = 0.760$ ns, $T_{setup} = 0.055$ ns, $T_{unc} = 0.040$ ns, launch insertion = 0.310 ns, capture insertion = 0.335 ns. Find setup slack. Then, given $T_{cq,min} = 0.055$ ns, $T_{comb,min} = 0.070$ ns, $T_{hold} = 0.030$ ns, $T_{unc,hold} = 0.020$ ns, find hold slack.

Solution

Skew = $0.335 - 0.310 = +0.025$ ns.

$$\begin{aligned} \text{Slack}_{setup} &= 1.000 + 0.025 - 0.055 - 0.040 - 0.090 - 0.760 \\ &= +\mathbf{0.080} \text{ ns} \quad (\text{met, } 80 \text{ ps margin}) \end{aligned}$$

$$\begin{aligned} \text{Slack}_{hold} &= 0.055 + 0.070 - 0.025 - 0.030 - 0.020 \\ &= +\mathbf{0.050} \text{ ns} \quad (\text{met}) \end{aligned}$$

Now the interesting part. Suppose you must raise the frequency to 1.25 GHz, i.e. $T = 0.800$ ns. Setup slack becomes $0.800 + 0.025 - 0.055 - 0.040 - 0.090 - 0.760 = -0.120$ ns. You need 120 ps. Hold slack is *unchanged* at +0.050 ns.

Options, cheapest first:

1. V_t swap the 3 slowest cells on the path: $\sim 25\text{--}30\%$ delay reduction on those cells at zero area cost. If those cells account for 400 ps of the 760 ps, this alone yields ~ 110 ps. Cost: leakage.

2. Upsize the two highest-load drivers: another 30–50 ps, cost area and the upstream load penalty.
3. Add useful skew: push capture clock +40 ps later. But check the *next* stage — you have just made *its* launch clock later, so its setup slack drops by 40 ps and its hold slack improves.

3.4 Transition, capacitance, and fanout constraints (DRV)

Design Rule Violations in the timing sense — distinct from geometric DRC:

Check	SDC command	Why it matters
Max transition	<code>set_max_transition</code>	Slow edges $\Rightarrow$ both transistors on $\Rightarrow$ short-circuit power; also invalidates library characterization range.
Max capacitance	<code>set_max_capacitance</code>	Guarantees the driver can actually drive the net within characterized limits.
Max fanout	<code>set_max_fanout</code>	A synthesis-time proxy for load.
Min pulse width	<code>set_min_pulse_width</code>	Clock pulse must be wide enough for the flop to register.

Caution

Fix DRVs *before* chasing slack. A max-transition violation makes every delay number downstream of it unreliable, so optimizing timing on top of open DRVs is optimizing against noise.

3.5 Corners, PVT, and MMMC

Delay depends on process (P), voltage (V), and temperature (T). A *corner* is a named combination.

Corner	Process	Voltage	Temp	Used for
SS / worst	slow	$V_{nom} - 10\%$	125°C	Setup (max delay)
FF / best	fast	$V_{nom} + 10\%$	−40°C	Hold (min delay), also EM/noise
TT / typical	typical	V_{nom}	25°C	Power, sanity
SF, FS	mixed	—	—	Cross-corner skew, level shifters

Temperature inversion

At advanced nodes and low V_{DD} , cells can be *slower at cold* than at hot, because the mobility gain at low temperature is outweighed by the V_{th} increase. This breaks the naive “hot = slow” rule and forces you to check setup at *both* temperature extremes. Do not assume; read the library.

MMMC (Multi-Mode Multi-Corner) analysis runs the design across {functional, test, sleep, ...} $\times$ {SS, FF, TT, ...} simultaneously. The number of *scenarios* is the product, and runtime scales with it. In a contest, identify the *dominant* scenario and optimize against it, then verify the rest.

3.6 OCV, derating, and advanced variation models

Two identical cells on the same die are not identical. Variation modelling has evolved:

1. **OCV (On-Chip Variation)**. Flat derate: e.g. launch path $\times 1.05$, capture path $\times 0.95$. Simple, very pessimistic.
2. **AOCV (Advanced OCV)**. Derate depends on *path depth* and *distance*. Longer paths average out random variation, so derate shrinks with depth — statistically sound and less pessimistic.
3. **POCV / SOCV (Parametric/Statistical OCV)**. Each arc carries a σ ; delays are combined as random variables: $\sigma_{path} = \sqrt{\sum_i \sigma_i^2}$. Least pessimistic, now standard.

CRPR / CPPR (Clock Reconvergence Pessimism Removal) credits back the derate applied to the *common* portion of the launch and capture clock paths, since a shared buffer cannot simultaneously be fast and slow.

Example 3.2 — Derate arithmetic

A reg2reg path: launch clock path = 300 ps (of which 220 ps is common with capture), capture clock path = 310 ps, data path = 600 ps, $T = 1000$ ps, $T_{setup} = 50$ ps. Late derate = +5%, early derate = -5%. Compute setup slack with and without CRPR.

Solution

Without CRPR. Launch (late) = $300 \times 1.05 = 315$ ps; data (late) = $600 \times 1.05 = 630$ ps; capture (early) = $310 \times 0.95 = 294.5$ ps.

$$\text{Arrival} = 315 + 630 = 945, \quad \text{Required} = 1000 + 294.5 - 50 = 1244.5$$

$$\Rightarrow \text{Slack} = +299.5 \text{ ps}$$

Wait — this looks generous because the common path was derated in *opposite* directions, which is the pessimism. Recompute the pessimism term: the common 220 ps was treated as $220 \times 1.05 = 231$ on launch and $220 \times 0.95 = 209$ on capture, a spurious 22 ps of difference.

With CRPR, credit 22 ps back: Slack = $299.5 + 22 = +321.5$ ps.

Lesson: on a design with deep clock trees, CRPR routinely recovers 50–150 ps of *fake* violation. Enable it before you spend a single gate fixing timing.

3.7 Timing exceptions

Exception	Meaning and danger
<code>set_false_path</code>	Path is never functionally exercised; STA ignores it entirely. <i>Danger:</i> a wrongly declared false path is silent silicon failure.
<code>set_multicycle_path</code>	Data is allowed N cycles. Must set both setup <i>and</i> hold multicycle correctly or you create a hold disaster.
<code>set_max_delay</code> / <code>set_min_delay</code>	Absolute path constraint, overriding clock-derived requirements.
<code>set_case_analysis</code>	Fixes a pin to a constant (mode select), pruning huge swathes of paths.

Exception	Meaning and danger
<code>set_clock_groups</code> <code>-asynchronous</code>	Declares two clocks unrelated; cleaner than pairwise false paths.
<code>set_disable_timing</code>	Kills a specific arc, e.g. to break a combinational loop.

Multicycle hold trap

`set_multicycle_path 2 -setup` without a matching `set_multicycle_path 1 -hold` moves the hold check to the wrong edge and generates hundreds of impossible hold violations. Always pair them.

CHAPTER 4

Interconnect: Resistance, Capacitance, and Wire Delay

4.1 Why wires became the problem

Gate delay scales down with each process node. Wire delay does not — wire resistance per unit length *rises* as cross-sections shrink, while capacitance per unit length stays roughly constant. Past roughly the 130 nm node, interconnect dominates the delay of any net that crosses more than a few hundred microns.

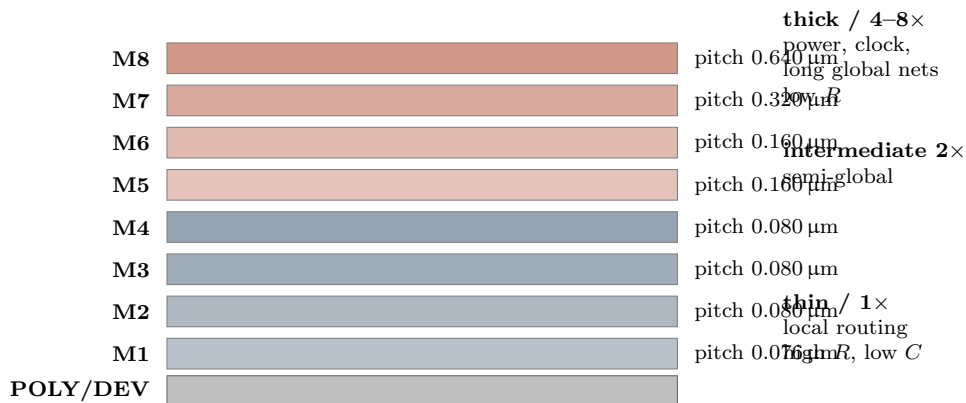
$$R_{wire} = \frac{\rho L}{Wt} = R_{sheet} \cdot \frac{L}{W}, \quad C_{wire} = C_{area} WL + C_{fringe} 2L + C_{coupling}$$

Note that both R and C are proportional to L , so unbuffered wire delay grows as L^2 . This quadratic is the single most important fact about interconnect.

Wire delay is quadratic; buffering makes it linear

$t_{wire} \propto RC \propto L^2$. Insert k evenly spaced buffers and each segment is L/k , so total wire delay $\approx k \cdot (L/k)^2 = L^2/k$, plus $k \cdot t_{buf}$. Minimizing the sum over k gives an optimal buffer count — derived in Section 4.5.

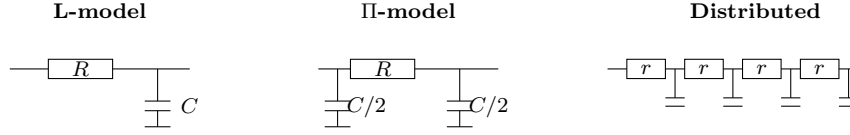
4.2 Metal stack anatomy



- **Preferred directions alternate:** M1 horizontal, M2 vertical, M3 horizontal, and so on. Routing against preferred direction is legal but expensive in tracks.
- **Upper layers are thicker and wider** $\Rightarrow$ much lower resistance per micron $\Rightarrow$ reserve them for clock and long global nets.

- **Vias have resistance too.** A stack from M1 to M8 can add tens of ohms; via arrays (multi-cut) reduce this and improve yield/EM.

4.3 Lumped models: L , Π , T , and distributed



The Π -model is the workhorse for STA. The distributed model, in the limit, gives the exact solution to the diffusion equation; its 50% delay for a uniform line is $0.38RC$, versus $0.69RC$ for the lumped RC .

4.4 Elmore delay

For an RC tree rooted at the driver, the Elmore delay to node i is

$$T_{D,i} = \sum_k R_{ik} C_k$$

where R_{ik} is the resistance of the portion of the path from the root to node i that is *shared* with the path to node k .

Example 4.1 — Elmore delay of a three-branch net

A driver feeds a wire of $R_1 = 100\ \Omega$ with $C_1 = 20\ \text{fF}$ at its far end, where it splits into two branches: branch A with $R_2 = 50\ \Omega$, $C_2 = 30\ \text{fF}$; branch B with $R_3 = 200\ \Omega$, $C_3 = 15\ \text{fF}$. Compute Elmore delay to the end of each branch.

Solution

Shared resistance from root: R_1 is shared by all three capacitors.

$$\begin{aligned} T_A &= R_1(C_1 + C_2 + C_3) + R_2C_2 \\ &= 100(20 + 30 + 15) \times 10^{-15} + 50 \times 30 \times 10^{-15} \\ &= 100(65\ \text{fF}) + 1500\ \Omega\text{fF} = 6500 + 1500 = 8000\ \Omega\text{fF} = \mathbf{8.0\ ps} \end{aligned}$$

$$T_B = R_1(C_1 + C_2 + C_3) + R_3C_3 = 6500 + 200 \times 15 = 9500\ \Omega\text{fF} = \mathbf{9.5\ ps}$$

Insight: branch B is slower not because it has more capacitance (it has *less*) but because it has more resistance. *Both* R and C matter, and the shared trunk resistance R_1 dominates both — which is exactly why widening or up-layering the *trunk* of a high-fanout net is more effective than touching the branches.

4.5 Optimal buffer insertion for a long wire

Let a wire of length L have per-unit resistance r and capacitance c . Insert k identical buffers of intrinsic delay t_b , output resistance R_b , input capacitance C_b . Each segment has length L/k .

Segment delay (using 0.69 for lumped RC plus driver terms):

$$t_{seg} = 0.69 \left[R_b \left(\frac{cL}{k} + C_b \right) + \frac{rL}{k} \left(\frac{cL}{2k} + C_b \right) \right] + t_b$$

Total $T(k) = k t_{seg}$. Keeping the k -dependent terms:

$$T(k) \approx k(t_b + 0.69R_bC_b) + 0.69(R_b cL + rLC_b) + \frac{0.345rcL^2}{k}$$

Differentiate and set to zero:

$$k_{opt} = L \sqrt{\frac{0.345rc}{t_b + 0.69R_bC_b}}$$

so $k_{opt} \propto L$: **buffers should be inserted at a constant pitch**, and the resulting total delay is *linear* in L .

Example 4.2 — Buffer count and pitch

$L = 4000\mu\text{m}$, $r = 0.20\ \Omega/\mu\text{m}$, $c = 0.20\ \text{fF}/\mu\text{m}$, buffer with $t_b = 25\ \text{ps}$, $R_b = 1.2\ \text{k}\Omega$, $C_b = 4\ \text{fF}$. Find k_{opt} , the buffer pitch, and compare buffered vs unbuffered delay.

Solution

$$rc = 0.20 \times 0.20 = 0.04\ \Omega\text{fF}/\mu\text{m}^2 = 0.04 \times 10^{-15}\ \text{s}/\mu\text{m}^2 = 0.04\ \text{ps}/\mu\text{m}^2.$$

$$\text{Denominator: } t_b + 0.69R_bC_b = 25 + 0.69(1200 \times 4 \times 10^{-15}\ \text{s}) = 25\ \text{ps} + 0.69(4.8\ \text{ps}) = 25 + 3.3 = 28.3\ \text{ps}.$$

$$k_{opt} = 4000 \sqrt{\frac{0.345 \times 0.04}{28.3}} = 4000 \sqrt{4.876 \times 10^{-4}} = 4000(0.02208) \approx 88$$

Buffer pitch = $4000/88 \approx 45\mu\text{m}$.

Unbuffered delay: $0.38rcL^2 = 0.38 \times 0.04 \times (4000)^2\ \text{ps} = 0.38 \times 0.04 \times 1.6 \times 10^7 = 243,200\ \text{ps} = 243\ \text{ns}$. Catastrophic.

Buffered: $T \approx k(t_b + 0.69R_bC_b) + 0.345rcL^2/k = 88(28.3) + 0.345(0.04)(1.6 \times 10^7)/88 = 2490 + 2509 \approx 5000\ \text{ps} = 5\ \text{ns}$.

A **48×** improvement. Note also that at the optimum the two terms are approximately *equal* — a useful sanity check that you have found the minimum.

Practical corollary

Real tools do not insert 88 identical buffers; they use van Ginneken's dynamic program (Chapter 18) with a library of sizes. But the $45\mu\text{m}$ pitch is the right order of magnitude, and if your post-route report shows a $600\mu\text{m}$ unbuffered net, you have found a bug, not a design.

4.6 Coupling capacitance and the Miller factor

Between neighbouring wires sits C_c . Its *effective* value depends on what the aggressor is doing:

Aggressor	Miller factor k_{MF}	Effect
Quiet (static)	1	Nominal
Switching same direction	0	Victim speeds up
Switching opposite direction	2	Victim slows down (worst case)
Worst-case analytical	up to 3	Used in pessimistic pre-route estimates

Effective load $C_{eff} = C_{gnd} + k_{MF}C_c$. This is why SI-aware extraction and timing produce *different* numbers from non-SI runs, and why the delta-delay report exists.

4.7 Wire load models vs. real extraction

Stage	Parasitic source	Accuracy
Synthesis (early)	Wire load model (fanout-based statistical table)	Poor; $\pm 50\%$ or worse.
Post-placement	Virtual route / trial route estimate	Moderate; captures placement locality.
Post-CTS	Same, clock nets real	Better.
Post-route	Full extraction to SPEF	Sign-off grade.

The classic surprise

A design that closes timing at synthesis and blows up post-route was never really closed. Always sanity-check with a placement-driven estimate before believing synthesis timing. In a competition this is the number-one cause of “it worked yesterday.”

Part II

The Flow, Stage by Stage

CHAPTER 5

Design Import and Sanity Checks

5.1 What you load and in what order

1. Technology LEF (layers, vias, rules, units).
2. Cell LEF for every library used (standard cells, macros, IO).
3. Liberty `.lib` for every corner you will analyse.
4. Gate-level Verilog netlist (post-synthesis, post-DFT).
5. SDC constraints.
6. UPF/CPF if multi-voltage.
7. Optional: floorplan DEF, IO constraint file, scan DEF.

5.2 The eight checks that save your run

#	Check	What you are looking for
1	<code>check_design</code>	Unconnected pins, floating nets, multi-driven nets, assign statements, empty modules.
2	<code>check_timing</code>	Unconstrained endpoints, missing clock definitions, no input/output delay, generated clocks not propagated.
3	Library consistency	Same cell present in LEF and LIB; matching pin names; matching site definitions.
4	Clock definition audit	Every clock created, all generated clocks derived, uncertainty set, transitions set, groups declared.
5	Exception audit	Every false path and multicycle justified; no accidental <code>-through</code> that kills real paths.
6	Netlist statistics	Cell count, macro count, sequential count, total cell area, average fanout.
7	Power intent	Domains, ports, isolation strategy consistent with netlist.
8	Missing physical data	Cells with no LEF, macros with no obstruction layers, no antenna properties.

Listing 5.1: A defensive import script (Innovus-style)

```

1  # --- MMMC setup -----
2  create_library_set -name ss_lib -timing {ss_0p81v_125c.lib}
3  create_library_set -name ff_lib -timing {ff_0p99v_m40c.lib}
4  create_rc_corner   -name rc_max -T 125 -qx_tech_file qrc_cmax.tch
5  create_rc_corner   -name rc_min -T 40  -qx_tech_file qrc_cmin.tch
6  create_delay_corner -name dc_slow -library_set ss_lib -rc_corner rc_max
7  create_delay_corner -name dc_fast -library_set ff_lib -rc_corner rc_min
8  create_constraint_mode -name func -sdc_files {func.sdc}
9  create_analysis_view -name setup_view -constraint_mode func -delay_corner
   dc_slow
10 create_analysis_view -name hold_view  -constraint_mode func -delay_corner
   dc_fast
11 set_analysis_view -setup {setup_view} -hold {hold_view}
12
13 # --- sanity -----
14 check_design -all
15 check_timing -verbose
16 report_timing -late -max_paths 20 > rpt/pre_place_setup.rpt
17 report_timing -early -max_paths 20 > rpt/pre_place_hold.rpt
18
19 # quick netlist census
20 puts "cells      : [llength [dbGet top.insts]]"
21 puts "seq        : [llength [dbGet top.insts.cell.isSequential 1]]"
22 puts "macros      : [llength [dbGet top.insts.cell.isBlock 1]]"
23 puts "std area: [dbGet top.fPlan.coreArea]"

```

Caution

An unconstrained endpoint is invisible in every timing report — it is not a violation, it is simply not checked. Teams lose competitions to a missing `create_clock` on a generated divide-by-2 clock. Run `check_timing` and read *all* of its output, not just the errors.

CHAPTER 6

Floorplanning

What this stage decides

Die and core dimensions, aspect ratio, target utilization, IO pad/pin locations, macro placement and orientation, blockages and halos, row structure, and the power grid skeleton. **Roughly 70–80% of your final PPA is locked in here.** No amount of downstream optimization rescues a bad floorplan.

6.1 Area arithmetic

$$\text{Utilization } U = \frac{A_{std} + A_{macro}}{A_{core}} \quad A_{core} = \frac{A_{std} + A_{macro}}{U}$$

$$A_{die} = (W_{core} + 2m)(H_{core} + 2m) \quad \text{Aspect ratio } AR = \frac{H_{core}}{W_{core}}$$

where m is the IO/seal-ring margin.

Example 6.1 — Sizing a core from scratch

A netlist has 184,000 standard cell instances with an average cell area of $1.35 \mu\text{m}^2$, plus four SRAM macros each $420 \mu\text{m} \times 260 \mu\text{m}$. Target utilization 70%, aspect ratio 1.0, IO margin $40 \mu\text{m}$ on each side. Compute core and die dimensions. Then find the utilization if the same design must fit a fixed $2000 \mu\text{m} \times 2000 \mu\text{m}$ die.

Solution

Standard cell area: $184,000 \times 1.35 = 248,400 \mu\text{m}^2$.

Macro area: $4 \times (420 \times 260) = 4 \times 109,200 = 436,800 \mu\text{m}^2$.

Total placeable area: $685,200 \mu\text{m}^2$.

Core area: $685,200 / 0.70 = 978,857 \mu\text{m}^2$.

Core side (AR = 1): $\sqrt{978,857} = 989.4 \mu\text{m}$. Round up to $990 \mu\text{m}$.

Die: $(990 + 80) \times (990 + 80) = 1070 \mu\text{m} \times 1070 \mu\text{m} = 1.145 \text{mm}^2$.

Fixed-die case: core would be $(2000 - 80)^2 = 1920^2 = 3,686,400 \mu\text{m}^2$, giving $U = 685,200 / 3,686,400 = \mathbf{18.6\%}$.

Interpretation. 18.6% is absurdly low — you are paying for $2.5\times$ the silicon you need. If the die is genuinely fixed (pad-limited design, package constraint), then the correct optimization is not to shrink but to *spend* the area: upsize aggressively, use low- V_t liberally, widen the power grid, and push frequency. Conversely if you were at 92% you would be fighting congestion for the whole contest.

6.2 Choosing utilization

Utilization	Consequence
< 50%	Wasteful area; long wires between sparse cells actually <i>hurt</i> timing and power.
55–65%	Comfortable for congested, high-fanout, or macro-heavy designs; leaves room for CTS buffers and ECO cells.
65–75%	The usual sweet spot for a general digital block.
75–85%	Aggressive; only for regular datapath-like designs with low routing demand. Expect congestion work.
> 85%	Legalization failures, routing DRCs, no room for hold buffers or ECO. Almost always a false economy.

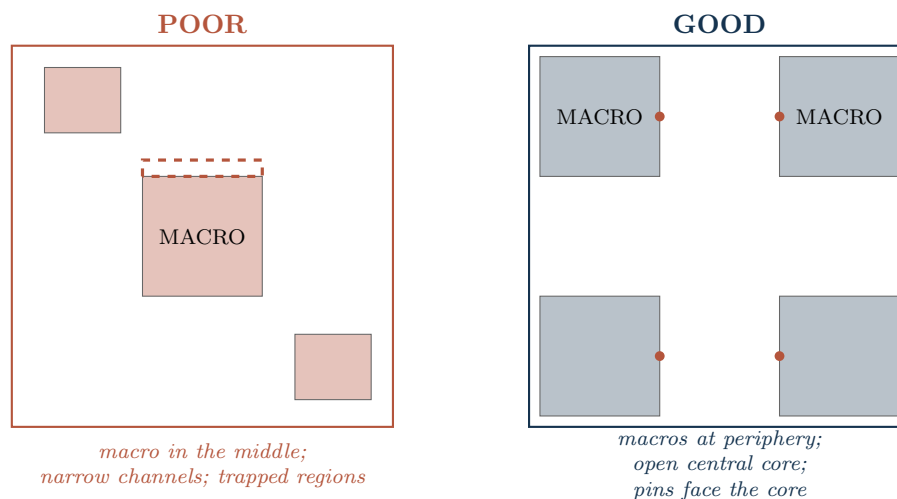
Budget for what comes later

CTS will add buffers (3–8% of cell area on a clock-heavy design). Hold fixing will add delay cells (1–3%). ECOs will add more. If you floorplan at exactly the utilization your placed netlist needs, you will have no room at CTS. **Leave headroom.**

6.3 Aspect ratio

- $AR = 1$ (square) minimizes perimeter for a given area and gives the shortest average intra-core distance. Default choice.
- Elongated cores ($AR \gg 1$ or $\ll 1$) lengthen the worst-case wire and make the clock tree harder to balance — but may be forced by package, by a hierarchical top-level slot, or by an IO-limited design.
- Non-rectangular (L-shaped, T-shaped) cores are legal and sometimes necessary; they complicate power grid and CTS and should be avoided when free to choose.

6.4 Macro placement — the highest-leverage manual step



6.4.1 Rules

1. **Push macros to the periphery**, leaving a contiguous open region for standard cells. A macro in the centre partitions the core and destroys both routing and clock tree balance.
2. **Orient macro pins toward the core**. Pins facing the die edge force every net to route around the macro.
3. **Group macros that talk to each other** (e.g. a memory bank and its controller) — fly-line display is your guide.
4. **Avoid narrow channels**. A channel narrower than roughly 20–30 routing tracks becomes a congestion trap. Either widen it or close it entirely with a placement blockage.
5. **Add halos (keep-out margins)** around each macro, typically 5–15 μm , so that standard cells do not crowd the macro pins and block pin access.
6. **Align macros** in rows/columns so that channels are straight and the power grid can run through uninterrupted.
7. **Respect the macro’s own power connection** requirements — some memories need power straps on specific layers directly over them.

6.5 Blockages

Type		Effect
Hard	placement blockage	No cell may be placed. Used over macros, in channels you wish to keep empty, under power straps.
Soft	placement blockage	Global placement avoids it; legalization may still place. Useful for gentle steering.
Partial blockage		Caps utilization in a region (e.g. “max 40% here”). The best tool for congestion hot-spots.
Routing blockage		Reserves tracks on given layers, e.g. over a macro, or under a top-level bus.
Halo	/ keep-out margin	Moves with the macro; blocks placement within $N\mu\text{m}$ of it.

Partial blockage is the congestion scalpel

When one region shows > 100% overflow, do not globally reduce utilization — that grows the die. Drop a *partial* blockage of 30–50% over the hot region only. You trade a little local density for a large routability gain at near-zero area cost.

6.6 IO and pin placement

For a hard macro / block-level design, pin placement is yours to choose:

- Place a pin on the side *nearest* the logic it feeds.
- Spread bus pins evenly; do not bunch 64 bits into 20 μm .
- Assign pins to layers consistent with preferred direction (vertical layers for top/bottom edges, horizontal for left/right).

- Keep clock and reset pins away from noisy high-activity buses.
- Match the pin order to the neighbouring block at top level to avoid crossovers.

6.7 Floorplan quality metrics

Metric	Where from	Target
Core utilization	<code>report_design</code>	65–75% typical
Total HPWL (fly-lines)	trial placement	minimize; compare alternatives
Global route overflow	trial route	< 0.5%, ideally 0
Pin density near macros	density map	no local hot-spot
Longest unbuffered net	trial route	< buffer pitch (§4.5)
Clock endpoint spread	fly-lines from clock root	compact, not diagonal

Floorplan iteration loop — do this 5 times, not once

1. Propose floorplan → 2. coarse placement (no optimization) → 3. trial global route → 4. read congestion map + HPWL + WNS → 5. change one thing → repeat.

Each loop should take minutes, not hours. Log the numbers in a table. In a contest this loop is where you win, because it is cheap and highly informative.

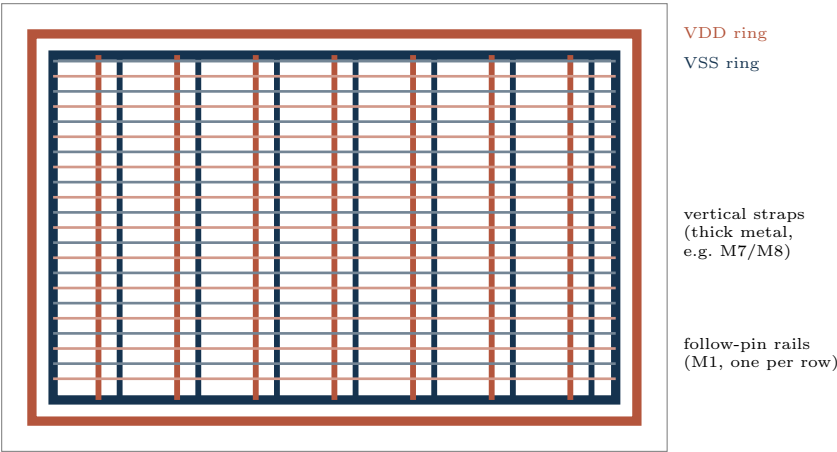
CHAPTER 7

Power Planning

What this stage decides

The metal network that delivers V_{DD} and V_{SS} to every cell: rings, stripes/straps, follow-pins (rails), macro connections, and the special via stacks. It determines IR drop, electromigration margin, and how many routing tracks remain for signals.

7.1 Structure



Element	Layers	Purpose
Core ring	Top two thick metals	Distributes from pads to the whole core.
Block/macro ring	Thick metals	Local supply around a macro.
Vertical straps	Thick metal (M7/M8)	Carry bulk current down the die.
Horizontal straps	Next thick metal	Form a mesh with the vertical straps.
Follow-pin rails	M1	Run along every row, feed each cell's power pins.
Via stacks	All	Connect mesh → rails; the usual IR bottleneck.

7.2 IR drop

Static (average) IR drop:

$$V_{drop} = I_{avg} \times R_{grid}$$

Dynamic IR drop adds the transient term from simultaneous switching:

$$V_{drop,dyn} = IR + L \frac{dI}{dt}$$

mitigated by decoupling capacitance, which supplies local charge before the grid can respond.

Why IR drop is a timing problem

Cell delay rises roughly 1–2% per 1% of V_{DD} lost — and worse at low voltage. A 5% IR drop can therefore eat 5–10% of your timing margin *everywhere at once*. A timing signoff run at nominal V_{DD} over a grid with 8% drop is simply wrong.

Budget	Typical target	Notes
Static IR drop	$\leq 1\text{--}2\%$ of V_{DD}	Per rail (VDD and VSS separately)
Dynamic IR drop	$\leq 4\text{--}5\%$ of V_{DD}	Worst instantaneous
Combined	$\leq 5\text{--}7.5\%$	Must match the STA voltage assumption

Example 7.1 — Grid sizing from an IR budget

A block draws $I_{total} = 520\text{ mA}$ at $V_{DD} = 0.90\text{ V}$. The vertical strap layer (M8) has sheet resistance $R_s = 0.025\ \Omega/\square$. Straps run the full $1000\mu\text{m}$ core height, are $3\mu\text{m}$ wide, and current is delivered from the top edge. Assume each strap carries an equal share of the current and, as a first-order model, that current is uniformly tapped along its length (giving an effective resistance of $R/2$ and an effective drop of $IR/2$). Static IR budget is 1.5% of V_{DD} . How many straps are needed?

Solution

Budget: $0.015 \times 0.90 = 13.5\text{ mV}$.

Resistance of one strap: number of squares = $L/W = 1000/3 = 333.3$, so $R_{strap} = 333.3 \times 0.025 = 8.33\ \Omega$.

With n straps, current per strap = $0.520/n\text{ A}$. Effective drop = $\frac{1}{2}I_{strap}R_{strap}$:

$$\frac{1}{2} \cdot \frac{0.520}{n} \cdot 8.33 \leq 0.0135 \Rightarrow \frac{2.166}{n} \leq 0.0135 \Rightarrow n \geq 160.4$$

So **n = 161** straps, i.e. a pitch of $1000/161 \approx 6.2\mu\text{m}$ if the core is also $1000\mu\text{m}$ wide.

Reality check. 161 straps of $3\mu\text{m}$ on a $1000\mu\text{m}$ width consumes $483\mu\text{m}$ of M8 — 48% of that layer. That is plausible for a thick top layer used only for power, but if M8 is also needed for the clock or long buses, you must instead:

- widen straps (resistance $\propto 1/W$, so $6\mu\text{m}$ straps halve R and halve the count needed to ≈ 81 , consuming the same total metal — *no gain* in area, but fewer via stacks and better EM);
- feed from *both* top and bottom edges: each strap now behaves as two $500\mu\text{m}$ halves, cutting effective resistance by $4\times$, so $n \geq 40$. **This is the real answer;**
- add a second orthogonal strap layer to form a true mesh, so no cell sits far from a low-resistance node.

Lesson: topology beats brute force in power grid design.

7.3 Electromigration

Metal atoms migrate under high current density, eventually opening the wire. Black's equation gives median time to failure:

$$\text{MTTF} = \frac{A}{J^n} e^{E_a/kT}, \quad n \approx 2$$

Design rule form: current density $J = I/(W \cdot t)$ must stay below a foundry-specified J_{max} that depends on layer, temperature, and duty cycle.

Current type	Where it bites
DC (average)	Power/ground rails and straps — unidirectional, worst EM case.
RMS	Self-heating (Joule heating) limits, especially thin wires.
Peak / AC	Signal nets, especially clock nets with high toggle rates and large drivers.

Fixes: widen the wire, use multi-cut vias, split the net between two parallel wires, downsize the driver, reduce toggle rate, move to a thicker layer.

Example 7.2 — EM check on a clock net

A clock buffer drives a net on M4. Peak current = 1.8 mA, wire width = 0.14 μm, thickness = 0.22 μm. Foundry $J_{max,peak} = 2.5 \text{ mA}/\mu\text{m}^2$ at 105°C. Does it pass? If not, what is the minimum width?

Solution

Cross-section = $0.14 \times 0.22 = 0.0308 \mu\text{m}^2$.

$$J = \frac{1.8}{0.0308} = 58.4 \text{ mA}/\mu\text{m}^2 \gg 2.5$$

Fails by 23×. Minimum cross-section = $1.8/2.5 = 0.72 \mu\text{m}^2$, so minimum width = $0.72/0.22 = 3.27 \mu\text{m}$ — i.e. about 23× minimum width, which is unroutable on M4.

Correct fix: this is not a width problem, it is a *driver* problem. Either (a) split the fanout so no single buffer sources 1.8 mA, (b) move the net to a thick upper layer (M7 with $t = 0.8 \mu\text{m}$ needs 0.9 μm width — feasible as a non-default rule), or (c) both, which is what real clock distribution does: upper-layer, wide, shielded NDR routing with distributed drivers.

7.4 Decap, tap, and endcap insertion order

1. Endcap and tap cells: immediately after floorplan (before placement).
2. Decap cells: after placement, in whitespace near high-activity regions.
3. Filler cells: last, after *all* ECOs, immediately before final GDS.

Caution

Inserting fillers before ECO means every ECO must first delete fillers, and if you forget, the ECO cell has nowhere to go. Always: ECO first, filler last.

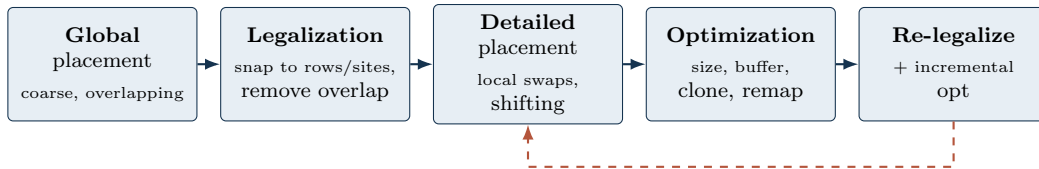
CHAPTER 8

Placement

What this stage decides

The (x, y) location and orientation of every standard cell. Objectives: minimize wirelength, meet timing, keep congestion routable, and respect density limits — simultaneously.

8.1 Sub-stages



8.2 The objective: wirelength models

Exact routed wirelength is unknown during placement, so proxies are used.

8.2.1 Half-perimeter wirelength (HPWL)

For a net with pins at (x_i, y_i) :

$$\text{HPWL} = \left(\max_i x_i - \min_i x_i \right) + \left(\max_i y_i - \min_i y_i \right)$$

Exact for 2- and 3-pin nets; a lower bound for larger nets. Fast, and the industry default.

8.2.2 Other models

Model		Description
Clique / quadratic		Sum of squared pairwise distances; differentiable, enables analytical placement.
Star model		Add a virtual centre pin; $O(p)$ instead of $O(p^2)$.
Rectilinear minimal (RSMT)	Steiner tree	The true optimum tree; NP-hard, but FLUTE gives near-exact results fast for ≤ 9 pins.
Steiner ratio		$\text{RSMT} \leq \text{MST}$, with $\text{RSMT}/\text{MST} \geq 2/3$ in the rectilinear plane.

Example 8.1 — HPWL and the effect of one move

A 4-pin net has pins at (10, 20), (60, 25), (35, 80), (15, 30) in microns. Compute HPWL. Then the cell holding pin (35, 80) is moved to (35, 40). Recompute, and state the percentage improvement.

Solution

Before: x range = [10, 60] $\Rightarrow$ 50; y range = [20, 80] $\Rightarrow$ 60. HPWL = 110 μ m.

After: x range unchanged = 50; y range = [20, 40] $\Rightarrow$ 20. HPWL = 70 μ m.

Improvement: $(110 - 70)/110 = 36.4\%$.

Follow-through. With $c = 0.20$ fF/ μ m, the net capacitance dropped by $40\mu\text{m} \times 0.20 = 8$ fF. If the driver had $R_{drv} = 2$ k Ω , the delay saving is roughly $0.69 \times 2000 \times 8 \times 10^{-15} = 11$ ps — from moving one cell. Multiply by thousands of cells and you see why global placement is where the wirelength battle is won.

8.3 Timing-driven placement

Pure wirelength minimization is timing-blind: it will happily lengthen a critical net to shorten ten non-critical ones. Two remedies:

1. **Net weighting.** Assign each net a weight w_n derived from its slack, e.g. $w_n = \left(1 + \frac{|\text{slack}_n|}{T}\right)^\alpha$, and minimize $\sum_n w_n \text{HPWL}_n$. Simple, robust, but indirect.
2. **Path-based / net constraints.** Budget the cycle time across the stages of each critical path and impose a maximum length per net. More accurate; harder to converge.

The weighting trap

Over-weighting critical nets pulls their cells together and *pushes everything else outward*, creating new critical paths and local congestion. This is why tools iterate: weight $\rightarrow$ place $\rightarrow$ re-time $\rightarrow$ re-weight. Do not hand-set an enormous weight on one net.

8.4 Congestion-driven placement

Congestion is measured on a grid of *gcells*. For each gcell edge:

$$\text{Overflow} = \max(0, \text{Demand} - \text{Supply}), \quad \text{Congestion \%} = \frac{\text{Demand}}{\text{Supply}} \times 100$$

Example 8.2 — Reading a congestion map

A gcell is $10\mu\text{m} \times 10\mu\text{m}$. M2 pitch is $0.10\mu\text{m}$ and M4 pitch is $0.20\mu\text{m}$; both are vertical. 12% of tracks are blocked by power straps. Global route estimates 155 vertical wires must cross this gcell. Compute supply, demand, overflow, and decide whether it is routable.

Solution

Raw vertical tracks: M2 gives $10/0.10 = 100$; M4 gives $10/0.20 = 50$. Total = 150.

After 12% blockage: $150 \times 0.88 = 132$ usable tracks = **supply**.

Demand = 155. **Overflow** = $155 - 132 = 23$ tracks. **Congestion** = $155/132 = 117\%$.

Not routable as placed. Ranked fixes:

1. Add a *partial* placement blockage of $\sim 30\%$ over this gcell cluster; the tool spreads the

cells and demand falls roughly proportionally.

2. Check whether one high-fanout net (e.g. a reset or a scan enable) is responsible — if so, buffer/clone it into a tree rather than a star.
3. Enable via-pillar / pin-access-aware placement if pin access is the real limiter.
4. Only as a last resort, reduce global utilization (grows die).

Rule of thumb: overflow of 0 is required for a clean route; a few gcells at 1–2 overflow can usually be absorbed by the detailed router, > 10 cannot.

8.5 Placement-stage optimization moves

This is the heart of the “optimization of physical design” theme. Each move below is a distinct, named transform your tool applies — and each one you should be able to apply by hand in an ECO.

Move	What it does	Costs / risks
Gate sizing (up)	Larger drive $\Rightarrow$ lower output resistance $\Rightarrow$ faster	Area, leakage, and <i>increases load on the previous stage</i>
Gate sizing (down)	Smaller cell on non-critical paths	Recovers area and leakage; may create new criticality
V_t swap (LVT)	Same footprint, lower V_{th} , faster	Leakage can rise 5–10 $\times$ per cell
V_t swap (HVT)	Slower, far less leakage	Only on paths with slack
Buffer insertion	Splits a long net; restores slew; isolates load	Adds area, adds intrinsic delay, adds power
Buffer removal	Deletes redundant buffers on slack paths	Recovers area/power
Cloning / replication	Duplicates a high-fanout driver, splits its sinks	Adds area; must keep logical equivalence
Logic restructuring	Re-associates a gate tree so the late-arriving signal has the shortest path	Netlist changes; LEC needed
Remapping	Two 2-input ANDs $\rightarrow$ one 3-input AND, or vice versa	Changes depth and area
Pin swapping	Swap logically equivalent inputs so the critical signal uses the fastest pin, or so a high-activity net uses the low-power pin	Free; almost always worth doing
Cell moving / relocation	Physically shift a cell toward its critical source/sink	May disturb legality and neighbours
Redundancy removal	Deletes logic proven unobservable	Requires equivalence proof
Scan chain re-order	Re-sequences scan connections by physical proximity	Huge wirelength win, must preserve chain count/order constraints

8.5.1 Pin swapping, concretely

A 2-input NAND has a *near* input (closer to output, lower delay) and a *far* input (stacked, higher delay) because of the series NMOS stack. If the critical signal is on the far pin and a non-critical one on the near pin, swapping them is free timing.

Example 8.3 — Pin swap value

For NAND2X2: arc $A \rightarrow Y = 48$ ps, arc $B \rightarrow Y = 61$ ps. Signal `crit` arrives at $t = 700$ ps on pin B; signal `slow_en` arrives at $t = 520$ ps on pin A. Required time at Y is 760 ps. Evaluate before and after a swap.

Solution

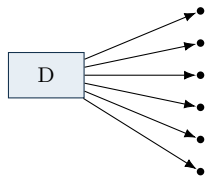
Before: arrival at Y = $\max(700 + 61, 520 + 48) = \max(761, 568) = 761$ ps. Slack = $760 - 761 = -1$ ps (violating).

After swap (`crit` on A, `slow_en` on B): arrival = $\max(700 + 48, 520 + 61) = \max(748, 581) = 748$ ps. Slack = $760 - 748 = +12$ ps (met).

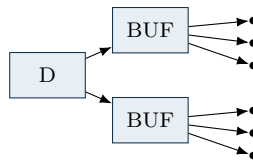
Cost: zero. No area, no power, no new cell. This is why pin swapping is the first move in every optimization engine — and why you should check for it manually before upsizing anything.

8.5.2 Cloning versus buffering

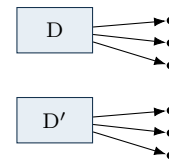
high fanout: 6 sinks



buffered



cloned



	Buffering	Cloning
Adds	1 stage of delay to <i>every</i> sink	0 extra stages
Best when	Sinks are far away; slew is the issue	Sinks form two spatially distinct clusters
Needs	Nothing	The driver's own inputs to be re-routed to the clone
Effect on driver	Load reduced to a few buffer inputs	Load halved

8.6 Scan chain reordering

Synthesis stitches scan chains by netlist order, which is physically random. After placement, re-order the chain to follow physical proximity.

Example 8.4 — Scan reorder saving

A scan chain has 500 flops. Before reorder, average scan net length is $180\mu\text{m}$. After proximity-based reorder, it is $22\mu\text{m}$. With $c = 0.20\text{ fF}/\mu\text{m}$, estimate the wirelength and switching-capacitance saving.

Solution

Wirelength: $500 \times (180 - 22) = 500 \times 158 = 79,000\mu\text{m} = 79\text{ mm}$ saved.

Capacitance: $79,000\mu\text{m} \times 0.20\text{ fF}/\mu\text{m} = 15,800\text{ fF} = 15.8\text{ pF}$ removed from the scan network.

At $V_{DD} = 0.9\text{ V}$ and scan-shift frequency 100 MHz with activity factor 0.5 :

$$P = \alpha CV^2 f = 0.5 \times 15.8 \times 10^{-12} \times 0.81 \times 10^8 = 0.64\text{ mW} \text{ saved in test mode}$$

More importantly, 79 mm of routing demand disappears from the congestion picture — scan nets are notorious congestion contributors precisely because they are long and numerous.

Caution: reordering must preserve chain length balance, must not cross clock domains, and must respect any lockup-latch placement.

8.7 Placement quality checklist

- Legality: 0 overlaps, all cells on sites/rows, correct orientation.
- Density map: no region above the target; check the max-density number, not just the average.
- Congestion: overflow ≈ 0 in trial route.
- Timing: setup WNS within a plausible distance of the target (post-CTS will change it); DRVs at zero.
- Pin access: no cells with unreachable pins (advanced nodes).
- Cell distribution: no “cell pileup” against a macro edge or die corner.

CHAPTER 9

Clock Tree Synthesis

What this stage decides

How the clock physically reaches every sequential element. Before CTS the clock is *ideal* (zero delay, zero skew, propagated by assumption). After CTS it is *propagated* — real buffers, real wires, real skew. Almost every timing number changes.

9.1 Definitions, precisely

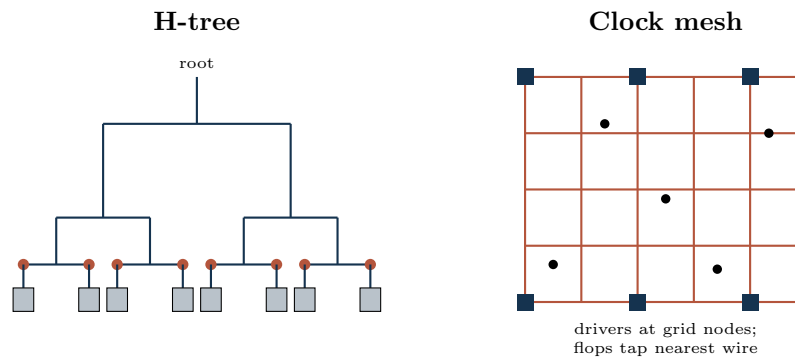
Term	Definition
Insertion delay (latency)	Clock source $\rightarrow$ flop clock pin delay.
Global skew	max insertion delay – min insertion delay, over all flops.
Local skew	Skew between flops that are in a launch/capture relationship. Only this one actually affects timing.
Useful skew	Deliberately non-zero local skew used to borrow slack.
Jitter	Cycle-to-cycle variation of the clock source (PLL). Not fixable by CTS.
Clock uncertainty	Budgeted allowance = jitter + margin + (pre-CTS) an estimate of skew.
Duty cycle distortion	Asymmetry introduced by unbalanced rise/fall in the tree.
Transition (slew)	Clock edge rate at each pin; tight limits (e.g. < 80 ps) because slow clock edges cost setup <i>and</i> increase short-circuit power.

Do not chase global skew

A team that reports “global skew = 3 ps” has usually over-buffered the tree, burned power, and inflated insertion delay — all to balance flops that never talk to each other. **Local skew is the metric. Global skew is a vanity number.**

9.2 Tree topologies

Topology	Description	Trade-off
Buffer tree	Recursive fanout tree built by the tool	Lowest power, most common, skew depends on balancing quality
H-tree	Geometrically symmetric recursive H	Excellent skew by construction; wasteful unless sinks are uniform
Clock mesh / grid	A driven grid over the die, flops tap the nearest node	Very low skew, extremely variation-tolerant, <i>very</i> high power. Used in high-performance CPUs
Spine / fish-bone	Central spine with ribs	Middle ground; good for elongated blocks
Multi-source CTS	Several drivers into a common structure	Reduces insertion delay on very large dies



9.3 The CTS algorithm in outline

1. **Identify sinks:** all clock pins, including gated branches and macro clock inputs.
2. **Cluster sinks** by physical proximity and required latency.
3. **Build topology** (DME — Deferred Merge Embedding — or tool-proprietary): recursively merge sub-trees at points of equal delay.
4. **Insert and size buffers/inverters** to meet slew and skew targets.
5. **Route the clock nets**, usually with non-default rules.
6. **Balance:** add delay to short branches, restructure long ones.
7. **Post-CTS optimization:** fix the setup violations that appeared and the (many) hold violations that CTS created.

9.4 Non-default routing rules for clocks

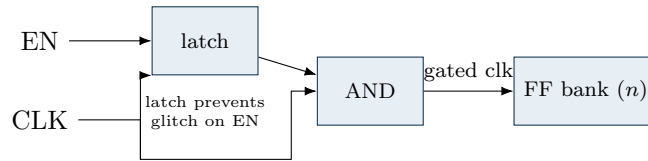
Rule	Reason
Double width	Halves resistance $\Rightarrow$ less delay, better EM.
Double spacing	Reduces coupling capacitance and crosstalk-induced jitter.
Shielding (VSS on both sides)	Isolates the clock from aggressors entirely; costs 2 extra tracks per clock wire.
Upper-layer preference	Thick metals have far lower R .
Multi-cut vias	Lower via resistance, better EM and yield.

Why the clock deserves special treatment

The clock is (i) the highest-toggle-rate net on the chip — $\alpha = 2$ per cycle vs. ≈ 0.1 for data; (ii) connected to *every* flop, so its delay variation multiplies across the design; and (iii) the one net where crosstalk causes *jitter*, which eats setup *and* hold margin at the same time. Spending metal on the clock is almost always the right call.

9.5 Clock gating

An Integrated Clock Gating cell (ICG) stops the clock to a register bank when its enable is inactive:



Example 9.1 — Value of clock gating

A design has 40,000 flops. Each flop's clock pin capacitance is 2.0 fF; the clock tree wiring and buffers add a further 1.6 $\times$ that total. $V_{DD} = 0.9$ V, $f = 800$ MHz. Measured average enable activity is 35%, i.e. on average 65% of flops have their clock stopped. Compute clock power before and after gating, assuming ICG cells themselves cost 8% of the saved power.

Solution

Flop pin capacitance total: $40,000 \times 2.0 \text{ fF} = 80,000 \text{ fF} = 80 \text{ pF}$.

Including tree: $C_{clk} = 80 \text{ pF} \times (1 + 1.6) = 208 \text{ pF}$.

Clock toggles *twice* per cycle, so $\alpha_{clk} = 1$ in the convention $P = \alpha CV^2 f$ with α counting full charge/discharge cycles per clock period:

$$\begin{aligned}
 P_{ungated} &= 1 \times 208 \times 10^{-12} \times (0.9)^2 \times 800 \times 10^6 = 208 \times 10^{-12} \times 0.81 \times 8 \times 10^8 \\
 &= 134.8 \times 10^{-3} \text{ W} = \mathbf{134.8 \text{ mW}}
 \end{aligned}$$

With gating, only 35% of the leaf-level capacitance toggles. Assume the trunk (say 30% of C_{clk}) still toggles always, and the gated leaves (70%) toggle 35% of the time:

$$C_{eff} = 208(0.30) + 208(0.70)(0.35) = 62.4 + 50.96 = 113.4 \text{ pF}$$

$$P_{gated} = 113.4 \times 10^{-12} \times 0.81 \times 8 \times 10^8 = 73.5 \text{ mW}$$

Saving = $134.8 - 73.5 = 61.3$ mW. ICG overhead = $0.08 \times 61.3 = 4.9$ mW.

Net saving = 56.4 mW, **i.e. 42% of clock power.**

Additional effect: gated flops also stop their internal clock-driven switching, so real savings are typically larger than this estimate. This is why clock gating is the single highest-yield dynamic-power technique.

9.6 Useful skew, quantitatively

Example 9.2 — Borrowing slack with skew

Two back-to-back stages, $T = 1000$ ps, $T_{setup} = 50$ ps, $T_{cq} = 90$ ps, uncertainty = 30 ps. Stage 1 (FF1→FF2) has $T_{comb} = 880$ ps. Stage 2 (FF2→FF3) has $T_{comb} = 620$ ps. All skews initially zero. Find both slacks, then find the skew on FF2's clock that maximizes the minimum slack.

Solution

Baseline.

$$S_1 = 1000 + 0 - 50 - 30 - 90 - 880 = -50 \text{ ps} \quad (\text{violating})$$

$$S_2 = 1000 + 0 - 50 - 30 - 90 - 620 = +210 \text{ ps}$$

Now delay FF2's clock by δ . FF2 is the *capture* flop of stage 1 (so skew = $+\delta$, helping) and the *launch* flop of stage 2 (so its skew is $-\delta$, hurting):

$$S_1(\delta) = -50 + \delta, \quad S_2(\delta) = 210 - \delta$$

Maximize $\min(S_1, S_2)$: set them equal.

$$-50 + \delta = 210 - \delta \Rightarrow 2\delta = 260 \Rightarrow \boxed{\delta = 130 \text{ ps}}$$

Both slacks become +80 ps. **The violation is fixed with zero added gates.**

Now check hold. Stage 1's hold slack *degrades* by $\delta = 130$ ps because the capture clock is later. If stage 1's baseline hold slack was +60 ps, it is now -70 ps and needs delay buffers. This is the price of useful skew, and it is why tools apply it in a constrained, slack-aware way ("concurrent clock and data optimization," CCD/CCOpt).

Practical bound: keep $|\delta|$ below roughly 10–15% of T unless you have verified hold across all corners.

9.7 Post-CTS hold fixing

CTS is where hold violations are born, in the hundreds or thousands. Fixes in order of preference:

1. **Fix at the clock tree:** rebalance so skew is less hostile. Free if it does not break setup.
2. **Downsize** the launch-path cells (slower data = better hold).
3. V_t **swap to HVT** on the short path — slows it *and* saves leakage. Best-of-both move.
4. **Insert delay cells** (specialised high-delay, low-area buffers). Standard, but adds area and power.
5. **Reroute** the data net on a longer path (detour) — rarely used.

Caution

Never fix hold before CTS is final and never fix hold using the pre-route estimate alone. Each hold buffer you add on an estimate you will have to re-verify, and hold fixes inserted early become hold *over*-fixes that cost setup after routing.

Routing

What this stage decides

The exact metal geometry of every signal net: which layers, which tracks, which vias. Objectives: complete every connection, break no design rule, meet timing after real parasitics, and keep crosstalk within budget.

10.1 Sub-stages

Stage	What happens
Global routing	The die is divided into gcells. Each net is assigned a <i>sequence of gcells</i> and a layer range, but no exact track. Congestion is resolved here.
Track assignment	Each net segment is assigned an actual track within its gcells; long straight runs are created.
Detailed routing	Exact geometry, vias, and DRC-clean connections. Iterative rip-up-and-reroute.
Search and repair	Remaining violations targeted individually.
Post-route optimization	Timing and SI fixes with real parasitics.

10.2 Routing algorithms in brief

Algorithm	Idea
Lee / maze router	BFS wavefront from source to target on a grid; guarantees shortest path if one exists. $O(N)$ memory and time in grid size — expensive.
A*	Maze router with a distance heuristic; far faster, same optimality under an admissible heuristic.
Line-probe (High-tower/Mikami)	Shoot lines from source and target, look for intersections. Memory-light, not always optimal.
Channel routing (left-edge)	For rectangular channels: build a vertical constraint graph and a horizontal interval graph, then assign tracks.
Steiner tree (FLUTE, RSMT)	Multi-pin optimal/near-optimal topology.
Pattern routing	Restrict to L-shapes and Z-shapes for speed in global route.
Negotiated congestion (PathFinder)	Iteratively re-route with rising penalties on over-used resources until conflicts vanish. The basis of modern global routers.

10.3 The antenna effect

During plasma etching, a long metal segment connected to a gate but not yet connected to a diffusion collects charge; the resulting voltage can rupture the thin gate oxide.

$$\text{Antenna ratio} = \frac{\text{Area of connected metal (or poly)}}{\text{Area of connected gate oxide}} \leq \text{AR}_{\max}$$

Fix	How it works
Layer jumping (via up/down)	Break the long segment on the offending layer by routing part of it on a higher layer, which is processed later.
Diode insertion	Add a reverse-biased diode to bleed charge to substrate. Costs area and adds capacitance.
Buffer insertion	Splits the net, so each piece has its own diffusion.
Driver upsizing	A larger driver has more diffusion area, raising the allowed metal area.

Example 10.1 — Antenna ratio

A net on M1 has length $180\mu\text{m}$ and width $0.09\mu\text{m}$. It drives a single gate of $W/L = 0.30\mu\text{m}/0.03\mu\text{m}$. Foundry limit for M1 is $\text{AR}_{\max} = 400$. Does it violate? What is the maximum legal M1 length?

Solution

Metal area = $180 \times 0.09 = 16.2\mu\text{m}^2$.

Gate area = $0.30 \times 0.03 = 0.009\mu\text{m}^2$.

$\text{AR} = 16.2/0.009 = \mathbf{1800}$. Limit is 400 — **violates by 4.5×**.

Maximum legal metal area = $400 \times 0.009 = 3.6\mu\text{m}^2$, hence maximum M1 length = $3.6/0.09 = \mathbf{40\mu\text{m}}$.

Fix: route only $\leq 40\mu\text{m}$ on M1, then jump to M2 for the remaining $140\mu\text{m}$ (M2's own budget is then computed against the same gate). Or insert a diode at the gate, which raises the effective allowance. Layer jumping is preferred — it is free.

10.4 Crosstalk: noise and delta delay

Two distinct failures come from the same coupling capacitor.

10.4.1 Crosstalk noise (glitch)

An aggressor switching while the victim is *static* injects a glitch of approximate peak

$$V_{glitch} \approx V_{DD} \frac{C_c}{C_c + C_v} \cdot \frac{\tau_v}{\tau_a + \tau_v}$$

where C_v is victim ground capacitance, and τ are the respective time-constants. If V_{glitch} exceeds the victim gate's noise margin, a false logic value can propagate — fatal on a reset, set, or clock net.

10.4.2 Crosstalk delay (delta delay)

An aggressor switching *simultaneously* in the opposite direction increases the effective coupling capacitance up to $2C_c$ (Miller), slowing the victim; in the same direction it speeds it up. Both matter: the first hurts setup, the second hurts hold.

Example 10.2 — Delta delay budget

A victim net has $C_{gnd} = 12\text{ fF}$ and $C_c = 9\text{ fF}$ shared with one aggressor. The victim driver has $R_{drv} = 2.4\text{ k}\Omega$. Compute nominal delay, worst-case (opposite-direction) delay, and best case (same direction). Assume the $0.69RC$ model.

Solution

Nominal ($k_{MF} = 1$): $C_{eff} = 12 + 9 = 21\text{ fF}$.

$$t = 0.69 \times 2400 \times 21 \times 10^{-15} = 34.8\text{ ps}$$

Worst case ($k_{MF} = 2$): $C_{eff} = 12 + 18 = 30\text{ fF}$.

$$t = 0.69 \times 2400 \times 30 \times 10^{-15} = 49.7\text{ ps}$$

Best case ($k_{MF} = 0$): $C_{eff} = 12\text{ fF}$.

$$t = 0.69 \times 2400 \times 12 \times 10^{-15} = 19.9\text{ ps}$$

Delta delay range: -14.9 ps to $+14.9\text{ ps}$ — a $\pm 43\%$ swing on this net.

The setup check must use 49.7 ps ; the hold check must use 19.9 ps . On a path with 15 such nets, that is over 200 ps of setup pessimism you either budget for or engineer away.

Engineering it away: note $C_c/(C_c + C_{gnd}) = 9/21 = 43\%$ — an extremely high coupling ratio, typical of a long minimum-spaced parallel run. Doubling the spacing roughly halves C_c to 4.5 fF , bringing the ratio to 27% and the swing to $\pm 7.5\text{ ps}$. That is what an NDR buys you.

10.4.3 Crosstalk mitigation, ranked

1. **Increase spacing** (NDR) on victim or aggressor.
2. **Shield** with a grounded wire (full isolation; costs 2 tracks).
3. **Reduce parallel run length** by staggering layers or detouring.
4. **Upsize the victim driver** — lower R_{drv} means the victim resists being pulled.
5. **Downsize/slow the aggressor** — lower dV/dt injects less charge.
6. **Buffer the victim** to shorten the exposed segment.
7. **Layer swap** to a layer where the aggressor is not adjacent.

10.5 Via optimization

- **Multi-cut / redundant vias**: two or more cuts in place of one. Lower resistance, dramatically better yield (a single cut is the most common open defect), better EM. Target > 95% multi-cut conversion.
- **Via pillars**: stacked multi-cut vias on critical nets to reduce the via-resistance contribution to delay.
- **Minimize via count** on timing-critical nets: each via is both resistance and a DRC risk.

10.6 Routing DRC categories

Rule	Meaning
Min width	Wire narrower than allowed.
Min spacing	Two shapes closer than allowed (may depend on width and parallel run length — “PRL spacing”).
Min area	A metal island smaller than the minimum printable area.
Min enclosed area	A hole in metal too small.
Via enclosure	Metal must overhang the via cut by a minimum.
End-of-line (EOL) spacing	Special, larger spacing at a wire tip.
Cut spacing	Between via cuts.
Notch	A concave step smaller than the minimum.
Density (min/max)	Metal fill ratio per window — fixed by dummy metal fill.
Antenna	See above.

10.7 Post-route optimization

With real SPEF, the tool now re-optimizes under a constraint no earlier stage had: *do not disturb the routing more than necessary*.

- Same-footprint V_t swaps (zero routing disturbance) — always first.

-
- Same-footprint sizing where the library provides it.
 - Local buffer insertion into existing whitespace.
 - Metal-only ECO using pre-placed spare cells (for post-tapeout fixes).

CHAPTER 11

Sign-off and ECO

11.1 The sign-off matrix

Check	Tool class	Passing criterion
STA	PrimeTime / Tempus	$WNS \geq 0$ setup and hold, all modes, all corners, all path groups; DRVs zero.
DRC	Calibre / Pegasus / ICV	Zero violations against the foundry runset.
LVS	same	Layout netlist $\equiv$ schematic netlist. Zero discrepancies.
ERC	same	No floating gates, no shorted supplies, correct well ties.
Antenna	same	All ratios within limit.
IR / EM	Voltus / Red-Hawk	Static and dynamic drop within budget; no EM violation.
LEC	Conformal / Formality	Post-PD netlist logically equivalent to RTL.
DFT	Tessent / DFT-MAX	Coverage target met; chains intact after reorder.
Power	Voltus / PTPX	Total power within budget with realistic activity (VCD / SAIF).
DFM	—	Litho hotspots, CMP density, via redundancy, recommended rules.

11.2 Types of ECO

Type	Scope
Timing ECO	Size/swap/buffer to close remaining WNS or TNS. Pre-mask.
Functional ECO	A logic change late in the flow. Pre-mask if free placement allowed; post-mask if it must use spare cells.
Metal-only ECO	Uses only pre-placed spare cells and changes only metal layers — vastly cheaper than a full mask set.
Freeze-silicon ECO	Same as metal-only but after tapeout.

11.3 ECO discipline

1. Always start from the *routed, extracted* database.
2. Fix DRVs, then setup, then hold — in that order.
3. Prefer zero-disturbance fixes (V_t swap, same-footprint size).
4. Re-extract and re-time after *every* batch; do not stack unverified fixes.
5. Run LEC after any netlist change.
6. Re-insert fillers only at the very end.

Listing 11.1: A minimal ECO loop

```

1  proc eco_iterate {maxiter} {
2    for {set i 1} {$i <= $maxiter} {incr i} {
3      puts "=== ECO iteration $i ==="
4      optDesign -postRoute -drv
5      optDesign -postRoute -setup
6      optDesign -postRoute -hold -holdTarget 0.020
7      ecoRoute
8      extractRC
9      set wns [dbGet top.wns]
10     puts "iteration $i : WNS = $wns"
11     if {$wns >= 0} { puts "closed"; break }
12   }
13   verify_drc -report rpt/drc.rpt
14   verifyConnectivity -report rpt/conn.rpt
15 }

```

Part III

The Optimization Sweep

CHAPTER 12

Area Optimization

12.1 What “area” actually means

Four different numbers all get called area. Know which one you are being scored on.

Metric	Definition
Cell area	$\sum$ areas of instantiated standard cells. Excludes fillers and white-space. What synthesis reports.
Placed area	Cell area + macro area.
Core area	The region rows occupy. Cell area / core area = utilization.
Die area	Core + IO ring + seal ring + scribe margin. What you <i>pay</i> for.

Key idea

Reducing cell area does not reduce die area unless you also shrink the core. A contest that scores “area” usually means *die* or *core*. Reducing cell count and then leaving the floorplan unchanged scores nothing.

12.2 Levers, ranked by impact

12.2.1 1. Architecture and RTL (largest, usually out of scope)

Resource sharing, narrower datapaths, memory instead of flops, arithmetic restructuring. If the contest gives you RTL, this is where the biggest wins are — one shared multiplier instead of four is a -75% on that block.

12.2.2 2. Synthesis constraints

- Do not over-constrain the clock. Synthesizing at $1.4\times$ the required frequency inflates every cell in the design. Constrain to the target plus a small margin (5–10%), no more.
- `set_max_area 0` plus `compile_ultra -area_high_effort`.
- Enable *ungrouping* so the optimizer can work across hierarchy boundaries — boundary optimization removes constant/duplicate logic between modules.
- Enable *register retiming* where legal: moves flops across combinational logic, often reducing both area and depth.

12.2.3 3. Gate downsizing on slack paths

The single biggest post-synthesis area lever. Every path with positive slack is carrying cells larger than they need to be.

Example 12.1 — Downsizing recovery

A block has 120,000 cells. Timing analysis shows 62% of cells lie only on paths with slack > 200 ps. Suppose downsizing one step reduces cell area by an average 32% and costs 45 ps of delay on the affected stage. Estimate the area recovered and confirm timing safety.

Solution

Eligible cells: $0.62 \times 120,000 = 74,400$.

Not all can be downsized (some are already minimum size). Assume 70% of the eligible set has a smaller variant: 52,080 cells.

If average cell area is $1.35 \mu\text{m}^2$, eligible area $= 52,080 \times 1.35 = 70,308 \mu\text{m}^2$. Recovery at 32%: **22,499 μm^2** .

Original total cell area $= 120,000 \times 1.35 = 162,000 \mu\text{m}^2$, so this is a **13.9%** cell-area reduction.

Timing safety. A path may contain several downsized stages. If a path has 4 downsized stages at 45 ps each, it loses 180 ps — within the 200 ps slack, but only just. Real tools handle this correctly by recomputing slack after each move (*incremental* downsizing) rather than batch-applying the transform. If you script this by hand, downsize in small batches and re-time.

Bonus: the leakage saving is roughly proportional to the area saving for same- V_t cells — so this move buys area *and* power. It is the best-value transform in the flow.

12.2.4 4. Buffer removal

Post-CTS and post-route designs accumulate buffers that are no longer needed after other optimizations changed the surrounding context. A buffer-removal pass on positive-slack nets typically recovers 2–5% of cell area.

12.2.5 5. Utilization increase

Raising utilization from 65% to 72% shrinks core area by $1 - 65/72 = 9.7\%$. Free, until congestion bites. Find the breaking point empirically with the trial-route loop from Chapter 6.

12.2.6 6. Floorplan shape and macro packing

Whitespace trapped between misaligned macros is pure waste. Aligning macro edges and closing narrow channels with blockages can recover several percent of core.

12.2.7 7. Library choice

A 7-track library is roughly 20% smaller in cell height than a 9-track library, at a cost in drive strength. For an area-dominated, timing-relaxed block, switching track height is a large single-step win. Mixed-height libraries (some vendors) let you use tall cells only on critical paths.

12.3 Area optimization procedure

Recipe

1. Report cell area, macro area, core area, utilization. Know your baseline.
2. Run area-recovery (downsizing + buffer removal) with a slack guard-band of $\sim 50\text{--}100\text{ ps}$ so you do not eat all your margin.
3. Re-time. Confirm WNS still positive.
4. Shrink the core by the recovered fraction; re-place; re-route trial.
5. If congestion is clean, repeat from step 2 until either WNS or congestion becomes the binding constraint.
6. Record the constraint that stopped you — that is your answer to “why not smaller?”

The area/wirelength paradox

Shrinking the core shortens wires (good) but raises congestion, which forces detours (bad). There is a minimum-total-wirelength utilization, usually a little *above* the minimum-congestion utilization. Plot HPWL vs. utilization for your design; the curve is U-shaped and its minimum is your answer.

CHAPTER 13

Timing Optimization

13.1 The complete lever list

#	Lever	Stage	Costs
1	Pin swap	Place, ECO	None
2	V_t swap to LVT	Place, post-route ECO	Leakage
3	Gate upsizing	Place, CTS, route	Area, power, upstream load
4	Buffer insertion	Place, route	Area, power, one stage of delay
5	Cell relocation	Place	Legality, neighbours
6	Cloning / fanout split	Place	Area
7	Logic restructuring	Synth, place	Netlist change, LEC
8	Remapping	Synth, place	Netlist change
9	Useful skew	CTS	Hold risk
10	Register retiming	Synth	Verification complexity
11	Layer promotion (NDR)	Route	Track resources
12	Wire widening / spacing	Route	Track resources
13	Via pillars	Route	Area on upper layers
14	Crosstalk mitigation	Route	Tracks
15	Floorplan change	Floorplan	Full re-run
16	Multicycle / false path	Constraints	<i>Must be functionally proven</i>
17	Higher V_{DD} / better IR	Power plan	Power, metal
18	Pipeline insertion	RTL	Latency, verification

13.2 Logical effort: sizing without guessing

Logical effort turns gate sizing from art into arithmetic. Delay of a stage:

$$d = g h + p$$

- g = **logical effort**: how much worse this gate is than an inverter at driving a load. Inverter $g = 1$, NAND2 $g = 4/3$, NOR2 $g = 5/3$, NAND3 $g = 5/3$, NOR3 $g = 7/3$, XOR2 $g = 4$.
- h = **electrical effort** (fanout) $= C_{out}/C_{in}$.
- p = **parasitic delay**: inverter 1, NAND2 2, NOR2 2, NAND n /NOR n $= n$.

For an N -stage path with path logical effort $G = \prod g_i$, path branching effort $B = \prod b_i$, and path electrical effort $H = C_{out}/C_{in}$:

$$F = GBH, \quad \hat{f} = F^{1/N} \text{ (optimal effort per stage)}, \quad D_{min} = NF^{1/N} + P$$

and the best number of stages satisfies $\hat{f} \approx 3.6$ (often rounded to 4) when the inverter parasitic is $p_{inv} = 1$.

Example 13.1 — Sizing a path by logical effort

A path drives a $64\times$ load from a $1\times$ input through $\text{INV} \rightarrow \text{NAND2} \rightarrow \text{INV} \rightarrow \text{NAND2}$. No branching. Find the optimal per-stage effort, the size of each gate, and the minimum delay in units of τ .

Solution

$$G = 1 \times \frac{4}{3} \times 1 \times \frac{4}{3} = \frac{16}{9} = 1.778.$$

$$B = 1. \quad H = 64. \quad F = 1.778 \times 64 = 113.8. \quad N = 4.$$

$$\hat{f} = 113.8^{1/4} = 3.267$$

$$P = 1 + 2 + 1 + 2 = 6.$$

$$D_{min} = 4(3.267) + 6 = 13.07 + 6 = \mathbf{19.07 \tau}$$

Gate sizes. Work backwards from the load. For stage i , $C_{in,i} = g_i C_{out,i} / \hat{f}$.

$$\text{Stage 4 (NAND2), } C_{out} = 64 : \quad C_{in,4} = \frac{4}{3}(64)/3.267 = 26.1$$

$$\text{Stage 3 (INV), } C_{out} = 26.1 : \quad C_{in,3} = 1(26.1)/3.267 = 7.99$$

$$\text{Stage 2 (NAND2), } C_{out} = 7.99 : \quad C_{in,2} = \frac{4}{3}(7.99)/3.267 = 3.26$$

$$\text{Stage 1 (INV), } C_{out} = 3.26 : \quad C_{in,1} = 1(3.26)/3.267 = 1.00 \checkmark$$

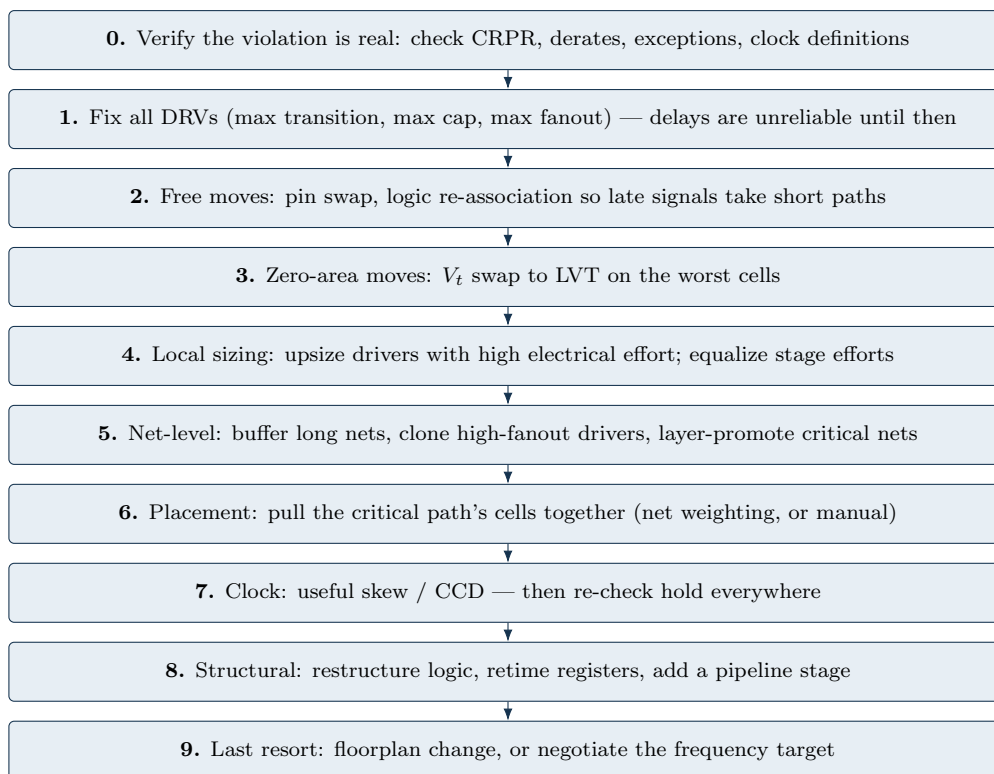
The check closing on 1.00 confirms the arithmetic.

Now the design question: is 4 stages right? $\hat{f} = 3.27$ is close to the ideal 3.6–4, so yes. If we tried $N = 6$ (adding two inverters), G stays 1.778, F stays 113.8, $\hat{f} = 113.8^{1/6} = 2.22$, $P = 8$, $D = 6(2.22) + 8 = 21.3 \tau$ — worse. If $N = 2$ were possible, $\hat{f} = 10.7$, $D = 2(10.7) + 3 = 24.4 \tau$ — also worse. Four is right.

The practical takeaway from logical effort

Stage effort should be roughly equal along a path, and around 4. If your timing report shows one gate driving a fanout of 20 and the next driving a fanout of 2, the path is mis-sized regardless of what the total delay is. Equalize the efforts and you get free speed.

13.3 Ordering your timing fixes



13.4 TNS vs. WNS: a strategic distinction

- **WNS** is one path. Fixing it may take a big structural change and improves the score by one number.
- **TNS** is $\sum_{\text{endpoints}} \min(0, \text{slack})$. It measures *how much* of the design is failing.
- **NVP** (number of violating paths / endpoints) measures *how many*.

Diagnose from the shape of the histogram

- **Small WNS, huge TNS, huge NVP:** a systemic problem — clock too aggressive, uncertainty over-budgeted, or the whole design is mis-sized. Attack globally.
- **Huge WNS, small TNS, tiny NVP:** one or two pathological paths. Attack them individually — often a single unbuffered long net, a badly placed macro pin, or a genuinely deep logic cone that needs pipelining.
- **Both large:** you are far from closure. Go back to floorplan.

Example 13.2 — Reading a slack histogram

A post-route report gives: WNS = −38 ps; TNS = −4,120 ps; 214 violating endpoints. Average violation = $4120/214 = 19.3$ ps. Recommend a strategy.

Solution

The average violation (19 ps) is half the worst (38 ps) and there are 214 of them — a *broad, shallow* failure. This is not a path problem, it is a margin problem.

Ranked strategy:

1. **Check clock uncertainty.** If it is still carrying a pre-CTS skew estimate (say 80 ps) after CTS achieved 25 ps local skew, removing the stale allowance recovers 55 ps on *every* path — fixing all 214 at once, for free.
2. **Check CRPR is on.** Typically worth 20–60 ps globally.
3. **Check derate model.** Moving from flat OCV to AOCV/POCV commonly recovers this magnitude.
4. Only then, **batch V_t swap:** 214 endpoints, average 19 ps, and an LVT swap gives ~25–30% on one cell. Roughly 200–400 swaps. Estimate leakage cost before committing.

The lesson: when TNS is broad and shallow, look at the *analysis setup* before you look at the *design*. Half of all “timing problems” in competitions are pessimism problems.

13.5 Hold optimization

Fix	Delta on hold	Side effect
Rebalance clock tree	Large	May break setup
HVT swap on short path	Moderate, free area	Saves leakage — best move
Downsize launch cells	Moderate	Saves area and power
Insert delay cells	Precisely controllable	Area, power, congestion
Detour data route	Small	Manual, fragile

Example 13.3 — Hold fixing cost

A design has 1,450 hold violations with an average of -42 ps. Delay cells provide 28 ps each, occupy $2.7 \mu\text{m}^2$, and leak 18 nW. Estimate the cost of a pure delay-cell fix, then propose a cheaper mix.

Solution

Cells needed per violation: $\lceil 42/28 \rceil = 2$.

Total cells = $1450 \times 2 = 2,900$.

Area = $2900 \times 2.7 = 7,830 \mu\text{m}^2$.

Leakage = $2900 \times 18 \text{ nW} = 52.2 \mu\text{W}$.

Cheaper mix. Note many of the 1450 violations will share paths and common launch cells.

1. First, HVT-swap the launch-path cells. If HVT adds ~ 35 ps per cell and 900 of the violations have a swappable launch cell, that fixes them at *zero area* and *negative* leakage cost.
2. Downsize where slack allows: another ~ 200 fixed, recovering area.
3. Remaining ~ 350 need delay cells: 700 cells, $1,890 \mu\text{m}^2$, $12.6 \mu\text{W}$.

Result: area cost falls from 7,830 to $1,890 \mu\text{m}^2$ (a 76% reduction) and total leakage actually improves because the HVT swaps more than offset the remaining delay cells.

This is the kind of reasoning that wins a competition: not “I fixed hold” but “I fixed

hold at one-quarter the area cost, and here is the arithmetic.”

13.6 Common timing pathologies and their signatures

Symptom in report	Likely cause	Fix
One net with huge delay, low fanout	Long unbuffered route, often across a macro	Buffer it; check floorplan and blockages
Large delay on a 2-input gate	Slow input transition upstream	Fix the DRV first
Path with 30+ stages	Deep combinational cone	Restructure or pipeline
Delay dominated by clock network	Excessive insertion delay	Reduce tree depth; multi-source CTS
Big gap between pre- and post-route	Parasitics were badly estimated	Fix congestion; check NDRs
Violations only in one corner	Corner-specific library or derate issue	Check the MMMC setup, not the design
Violation appears only with SI on	Crosstalk delta delay	Spacing, shielding, driver upsizing
Hold fails only after CTS	Skew, as expected	Post-CTS hold fixing (normal)
Setup fails only in test mode	Scan path not constrained/optimized	Constrain and optimize test mode explicitly

CHAPTER 14

Power Optimization

14.1 The three components

$$P_{total} = \underbrace{\alpha C_L V_{DD}^2 f}_{\text{switching (dynamic)}} + \underbrace{I_{sc} V_{DD}}_{\text{short-circuit}} + \underbrace{I_{leak} V_{DD}}_{\text{static (leakage)}}$$

Component	Driven by	Reduced by
Switching	Activity, capacitance, V_{DD}^2 , frequency	Clock gating, V_{DD} scaling, wirelength reduction, downsizing, encoding
Short-circuit	Input slew (both devices on together)	Slew control, driver sizing — usually < 10% of dynamic
Leakage	V_{th} , temperature, W , count	HVT cells, power gating, stack forcing, body bias, downsizing

The V_{DD}^2 lever

Dynamic power scales with the *square* of supply. A 10% voltage reduction cuts dynamic power by 19% — and costs roughly 15–25% in speed. This is why DVFS exists, and why voltage is always the first thing a power-constrained design negotiates.

14.2 Leakage mechanisms

1. **Subthreshold conduction** — dominant. $I \propto e^{-qV_{th}/nkT}$, so it doubles roughly every 8–10°C.
2. **Gate tunnelling** — through thin oxide; largely solved by high- κ dielectrics.
3. **Junction / GIDL** — reverse-biased diode and gate-induced drain leakage.

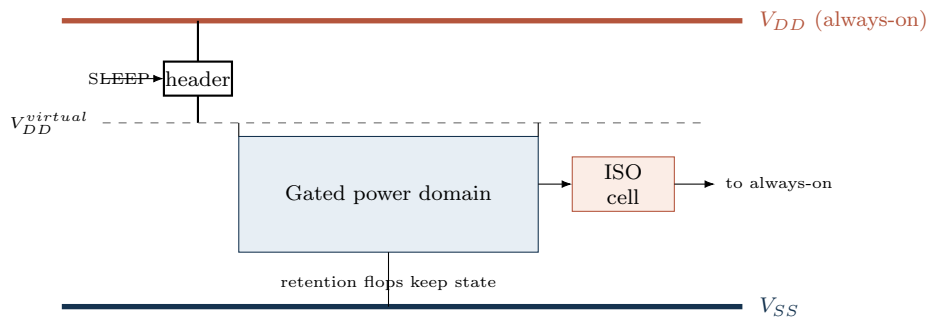
14.2.1 Stacking effect

Two series-off transistors leak far less than one (the intermediate node floats up, giving the top device a negative V_{GS} and reverse body bias). A NAND2 with both inputs low leaks perhaps 1/5 to 1/10 of a NAND2 with one input low. Hence *state-dependent* leakage tables in Liberty, and *input vector control* as a low-power technique.

14.3 The technique catalogue

Technique	Targets	Cost / caveat
Clock gating	Dynamic (largest single win)	ICG area; enable logic; testability — needs test-mode bypass
Data gating / operand isolation	Dynamic	Extra AND gates on datapath inputs
Multi- V_t (V_t swapping)	Leakage	Timing; requires the library to have multiple flavours
Multi- V_{DD} / voltage islands	Dynamic (V^2)	Level shifters, separate grids, complex CTS
DVFS	Dynamic + leakage	Needs regulator, control loop, multi-corner signoff
Power gating (MTCMOS)	Leakage (near-total)	Sleep transistors, IR drop through them, wake-up latency, retention flops, isolation cells
Retention registers	State preservation with gating	$\sim 2\times$ flop area on retained flops
Body biasing	Leakage / speed tuning	Needs triple-well, extra rails
Downsizing	Both	Timing
Wirelength reduction	Dynamic	The placement work you already did
Slew control	Short-circuit	DRV fixing you already do
Bus encoding (gray, bus-invert)	Dynamic on wide buses	Encoder/decoder logic
Memory banking / partial access	Dynamic in SRAM	Architecture change
Glitch reduction (path balancing)	Dynamic (glitches can be 10–40% of switching power in arithmetic)	Restructuring

14.4 Power gating anatomy



	Header (PMOS, cuts V_{DD})	Footer (NMOS, cuts V_{SS})
Size	Larger (PMOS weaker per width)	Smaller for the same on-resistance
Noise	Virtual V_{DD} droop	Virtual ground bounce
Typical	More common (protects supply)	Used where PMOS area is prohibitive

Example 14.1 — Sizing a power-gating switch

A gated domain draws $I_{peak} = 95$ mA. The design allows a maximum 25 mV drop across the header switch. Each header cell contributes an on-resistance of $R_{on} = 48 \Omega$ and occupies $3.4 \mu\text{m}^2$. How many are needed, and what is the area overhead if the domain is $180,000 \mu\text{m}^2$?

Solution

Required total on-resistance:

$$R_{total} \leq \frac{25 \times 10^{-3}}{95 \times 10^{-3}} = 0.2632 \Omega$$

With n headers in parallel, $R_{total} = 48/n$:

$$\frac{48}{n} \leq 0.2632 \Rightarrow n \geq 182.4 \Rightarrow \mathbf{n = 183}$$

Area = $183 \times 3.4 = 622 \mu\text{m}^2$, i.e. **0.35%** of the domain — very acceptable.

But the real constraint is usually rush current, not steady state. On wake-up, all 183 switches turning on at once slams the always-on grid with a huge dI/dt , causing a supply glitch that can corrupt *neighbouring* domains. Real designs use a *daisy-chained* switch enable so the headers turn on in a staggered sequence over hundreds of nanoseconds. Budget the wake-up latency accordingly.

14.5 Multi- V_t optimization as a knapsack problem

You have N cells. Each can be HVT (slow, low leak), SVT, or LVT (fast, high leak). You want to minimize $\sum_i \text{leak}(i)$ subject to $\text{WNS} \geq 0$.

The greedy algorithm that tools use:

1. Start with everything LVT (or as synthesized). Timing is met, leakage is awful.
2. Compute, for every cell, the ratio $\eta_i = \frac{\Delta \text{leakage saved}}{\Delta \text{delay added}}$.
3. Swap the cell with the highest η_i that has enough slack.
4. Re-time incrementally. Repeat until no cell can be swapped without violating.

Example 14.2 — Greedy V_t recovery

Five cells on a path with 150 ps of slack. Swapping to HVT adds delay and saves leakage as follows.

Cell	Δ delay (ps)	Leakage saved (nW)	η (nW/ps)
A	20	180	9.0
B	35	210	6.0
C	15	60	4.0
D	50	500	10.0
E	25	50	2.0

Which cells should be swapped to maximize leakage saving within the 150 ps budget?

Solution

This is a 0/1 knapsack with capacity 150 ps.

Greedy by η : D (10.0), A (9.0), B (6.0), C (4.0), E (2.0).

- Take D: cost 50, remaining 100, saved 500.
- Take A: cost 20, remaining 80, saved 680.
- Take B: cost 35, remaining 45, saved 890.
- Take C: cost 15, remaining 30, saved 950.
- Take E: cost 25, remaining 5, saved **1000**.

Total cost = 145 ps $\leq$ 150. All five fit! Saving = 1000 nW = 1 μ W on this path.

Now make it interesting. Suppose slack were only 70 ps.

- Greedy: D (50, 500), then A (20, 180) $\Rightarrow$ cost 70, saved **680**.
- Optimal (exhaustive): B+A+C = 35 + 20 + 15 = 70 ps, saved 210 + 180 + 60 = 450. Worse.
D+A = 70, saved 680.
D+C = 65, saved 560.
A+B+C = 70, saved 450.
So greedy is optimal here at **680**.

Lesson: greedy-by-ratio is not *guaranteed* optimal for 0/1 knapsack, but on real designs with thousands of cells and fine granularity it is within a fraction of a percent, which is why every tool uses it. What matters far more than the algorithm is *re-timing incrementally*: after swapping D, the slack on the path is genuinely 100 ps, not the stale 150.

14.6 Power analysis accuracy

Activity source	Accuracy	Use
Default toggle rate (e.g. 0.1)	Very poor	Early sanity only
Propagated from primary inputs	Poor–moderate	Relative comparisons
SAIF from RTL simulation	Good	Standard practice
VCD/FSDB gate-level sim	Best; includes glitches	Sign-off; huge files

Caution

Reporting power with default activity and calling it an optimization result is meaningless — you are comparing two guesses. Use the *same* activity file for before and after, and say which one you used.

14.7 A complete power optimization worked example

Example 14.3 — Full power budget

A block: 95,000 combinational cells, 28,000 flops, $f = 600$ MHz, $V_{DD} = 0.85$ V, $T_j = 105^\circ\text{C}$. Measured baseline: switching = 182 mW, internal = 64 mW, leakage = 41 mW. Clock network is 38% of switching power. Average data activity $\alpha = 0.12$. Target: 30% total reduction with no timing loss.

Solution

Baseline total = $182 + 64 + 41 = \mathbf{287}$ mW. Target = 200.9 mW, i.e. reduce by 86.1 mW.

Step 1 — Clock gating. Clock switching = $0.38 \times 182 = 69.2$ mW. If enable activity averages 40% and 70% of the clock capacitance is gateable:

$$\Delta P = 69.2 \times 0.70 \times (1 - 0.40) = 29.1 \text{ mW saved}$$

minus $\sim 8\%$ ICG overhead $\Rightarrow$ net **26.8** mW. Flops also stop internal switching: credit $\sim 0.70 \times 0.60$ of the flop share of internal power. If flops are 55% of internal power (35.2 mW), that is $35.2 \times 0.42 = \mathbf{14.8}$ mW. **Subtotal:** 41.6 mW.

Step 2 — Multi- V_t leakage recovery. Suppose 60% of cells sit on paths with > 100 ps slack and HVT cuts their leakage by 80%:

$$\Delta P = 41 \times 0.60 \times 0.80 = \mathbf{19.7} \text{ mW}$$

Step 3 — Area recovery / downsizing. A 12% cell-area reduction on slack paths reduces both leakage ($\sim$ proportionally on the affected cells) and switching (lower input capacitance). Estimate 6% of the remaining switching and 8% of remaining leakage:

$$\Delta P \approx 0.06(182 - 29.1) + 0.08(41 - 19.7) = 9.2 + 1.7 = \mathbf{10.9} \text{ mW}$$

Step 4 — Wirelength reduction. A placement pass that cuts total net capacitance by 8% cuts the wire component of switching. If wires are 45% of net switching capacitance:

$$\Delta P \approx (182 - 29.1) \times 0.45 \times 0.08 = \mathbf{5.5} \text{ mW}$$

Step 5 — Slew tightening to cut short-circuit (part of internal): say 4% of internal = **2.0** mW.

Action	Saving (mW)	Cumulative
Clock gating (clock net)	26.8	26.8
Clock gating (flop internal)	14.8	41.6
Multi- V_t HVT swap	19.7	61.3
Downsizing	10.9	72.2
Wirelength reduction	5.5	77.7
Slew tightening	2.0	79.7
Total	79.7	
Final power	$287 - 79.7 = \mathbf{207.3}$ mW (-27.8%)	

Short of the 30% target by 2.2%. Honest options:

- Voltage: dropping V_{DD} from 0.85 to 0.82 V (-3.5%) cuts dynamic by $1 - (0.82/0.85)^2 = 7.0\% \Rightarrow$ another ~ 10 mW. Costs $\sim 5\%$ speed — only viable if you have timing margin.
- Operand isolation on the datapath: typically another 2–5%.
- Memory access optimization if SRAMs are in the budget.

What to present: the table above, plus a statement of what each move cost in area and timing. A judge values the accounting as much as the number.

Congestion and Routability Optimization

15.1 Definitions and measurement

Supply_e = tracks available on edge e , Demand_e = nets that must cross e

$$\text{Overflow} = \sum_e \max(0, D_e - S_e), \quad \text{OF}\% = \frac{\#\{e : D_e > S_e\}}{\#\{e\}}$$

Metric	Interpretation
Overflow = 0	Global route believes the design is routable.
$0 < \text{OF} < 0.3\%$	Usually absorbable by detailed routing.
$\text{OF} > 1\%$	Expect thousands of DRCs; fix at placement.
Hot-spot map	Where, not just how much. Always look at the map.
Pin density	At advanced nodes, pin <i>access</i> fails before track supply does.

15.2 Root causes, in the order you should check them

1. **Utilization too high globally.** Check the number first.
2. **Local density hot-spot.** Timing-driven placement clustered a critical cone.
3. **High-fanout nets.** Reset, scan enable, test mode, clock enables. Each is one net with thousands of sinks and it saturates its region.
4. **Macro channels too narrow.** Everything must funnel through.
5. **Macro pin clustering.** All 128 pins of an SRAM on one edge creates a wall of demand.
6. **Complex cells with poor pin access.** Multi-height, many-input cells (AOI/OAI, full adders) placed adjacently.
7. **Power grid too dense.** Straps consume signal tracks.
8. **Layer restriction.** Someone blocked M6–M8; global route has nowhere to go.
9. **Blockage error.** A routing blockage left over from an experiment.

15.3 Fixes, ranked by cost

#	Fix	Notes
1	Partial placement blockage	Local, cheap, surgical. Set 30–50% max density over the hot-spot.
2	Cell padding / spacing	Adds a keep-out around specific cell types (e.g. all AOI22) to improve pin access.
3	High-fanout net synthesis (HFNS)	Convert star topology into a balanced buffer tree with spatially clustered branches.
4	Net weighting reduction	Lower the timing weight in the hot region so the placer stops clustering.
5	Macro halo increase	Keeps cells away from pin-dense macro edges.
6	Macro re-orientation	Turn pins toward the open core.
7	Macro re-placement	Widen the channel or move the macro to the edge.
8	Layer promotion for long nets	Frees lower-layer tracks in the hot region.
9	Power grid thinning (local)	Only where IR analysis permits.
10	Utilization reduction	Grows die; use last.
11	Floorplan restructure	Most expensive; sometimes the only right answer.

15.4 High-fanout net synthesis

Example 15.1 — Buffering a reset

An asynchronous reset drives 18,000 flops spread over a $900\mu\text{m} \times 900\mu\text{m}$ core. Max fanout per buffer is 24, max transition 120 ps. Design the buffer tree: how many levels, how many buffers, and what is the latency?

Solution

Levels. With fanout $f = 24$ per level, the number of levels L must satisfy $24^L \geq 18,000$:

$$24^2 = 576, \quad 24^3 = 13,824, \quad 24^4 = 331,776$$

So $L = 4$ levels (since $24^3 < 18,000$). But we can use a smaller effective fanout at the top: $18,000/24 = 750$ leaf buffers; $750/24 = 32$ level-2; $32/24 = 2$ level-3; 2 into 1 root.

Buffer count:

$$750 + 32 + 2 + 1 = \mathbf{785 \text{ buffers}}$$

Area. At $1.9\mu\text{m}^2$ per buffer: $1,492\mu\text{m}^2$ — about 0.18% of an $810,000\mu\text{m}^2$ core. Negligible.

Latency. Four levels at, say, 55 ps per buffer stage plus wire delay. Average wire per leaf buffer to its 24 flops: if the core is $900^2\mu\text{m}^2$ and there are 750 leaf regions, each region is $\sim 1080\mu\text{m}^2$, i.e. $\sim 33\mu\text{m}$ square. Wire per leaf $\approx 24 \times 20\mu\text{m} = 480\mu\text{m}$ of branch. At $0.20\text{ fF}/\mu\text{m}$ that is 96 fF per leaf buffer — check this against max cap; it may force fanout down to 16.

Total latency $\approx 4 \times 55 + \text{wire} \approx 300\text{--}450$ ps.

Skew consideration. An *asynchronous* reset’s release must be synchronized; the ~ 400 ps spread across the tree means reset de-assertion is not simultaneous. Standard practice: synchronize reset release to the clock (reset synchronizer) and treat the reset tree with the same balance discipline as a clock tree — some flows literally run CTS on it.

Congestion effect. Before HFNS, 18,000 nets radiate from one point — that region is unroutable. After, demand is spatially distributed. This single transform routinely turns a failing design into a passing one.

15.5 Pin access at advanced nodes

Below roughly 16 nm, the binding routability constraint stops being track count and becomes whether the router can legally land a via on a pin. Contributors:

- Very few M1 tracks per cell.
- Unidirectional M1/M2 with fixed colouring (multi-patterning).
- Adjacent cells whose pins fall on the same track.

Fixes: cell padding, pin-access-aware detailed placement, via-pillar-friendly libraries, and simply avoiding certain cell adjacencies (the tool has a “pin access cost” table).

15.6 Congestion vs. timing: the tension

Key idea

Timing-driven placement *creates* congestion by clustering. Congestion-driven placement *creates* timing violations by spreading. The tool balances them with an effort knob. Your job is to identify *which one is your binding constraint* and set the effort accordingly — and to recognize that if both are failing simultaneously, the problem is upstream (floorplan, utilization, or an unrealistic frequency target).

CHAPTER 16

Signal Integrity, IR Drop, and Reliability

16.1 The reliability checklist

Effect	Mechanism	Primary mitigation
Crosstalk noise	Capacitive coupling to a static victim	Spacing, shielding, victim driver upsizing
Crosstalk delay	Miller effect during simultaneous switching	Same, plus timing window analysis
Static IR drop	IR across the grid	More/wider straps, mesh, more via stacks
Dynamic IR drop	$L dI/dt$ on simultaneous switching	Decap, staggered clock edges, reduced peak current
Ground bounce	Return-path inductance	Solid ground mesh, more pads
Electromigration	Atom transport at high J	Wider wires, multi-cut vias, lower duty cycle
Self-heating	Joule heating in thin wires	RMS current limits, wider wires
Antenna	Plasma charging	Layer jumping, diodes
NBTI / PBTI aging	V_{th} shift over lifetime	Aging-aware derate, margin
Latch-up	Parasitic thyristor	Tap cells, guard rings, well spacing
ESD	External discharge	IO clamps, ESD rails
Soft errors (SER)	Particle strike flips a node	ECC, parity, TMR, larger node capacitance

16.2 Timing windows: the reason SI analysis is iterative

An aggressor only affects a victim if they switch *in the same time window*. But the delta delay changes arrival times, which changes the windows, which changes which aggressors overlap. So SI analysis iterates:

1. Compute nominal arrival windows.
2. Find overlapping aggressor/victim pairs.
3. Compute delta delay and delta noise.

4. Update arrival windows.
5. Repeat until convergence (typically 2–4 iterations).

Caution

Turning off timing windows makes SI analysis fast and *wildly* pessimistic — every aggressor is assumed to switch at the worst moment. If your SI-on numbers look catastrophic, check whether windows are enabled before you start re-routing.

16.3 Decoupling capacitance sizing

A switching event draws charge $Q = C_{sw}V_{DD}$ in time t_{sw} . If local decap supplies it, the decap's own voltage droops by

$$\Delta V = \frac{Q}{C_{decap}} = \frac{C_{sw}V_{DD}}{C_{decap}}$$

So to keep droop below a fraction δ of V_{DD} :

$$C_{decap} \geq \frac{C_{sw}}{\delta}$$

Example 16.1 — Decap budget

A region switches 28 pF of capacitance within one clock edge. Allowed local droop is 3% of V_{DD} . How much decap is needed? If a decap cell provides 12 fF and occupies $1.8 \mu\text{m}^2$, what area does that cost?

Solution

$$C_{decap} \geq \frac{28 \text{ pF}}{0.03} = 933 \text{ pF}$$

Number of cells = $933 \times 10^{-12} / 12 \times 10^{-15} = \mathbf{77,750}$ cells.

Area = $77,750 \times 1.8 = 139,950 \mu\text{m}^2$.

That is enormous — likely more than the logic it protects. Two conclusions:

1. Real decap cells provide much more capacitance per area than a 12 fF logic-sized cell; dedicated decap cells are wide MOS capacitors giving hundreds of fF each. Recompute with 400 fF/cell at $9 \mu\text{m}^2$: 2,333 cells, $21,000 \mu\text{m}^2$. Now reasonable.
2. Decap is *not* the whole answer to dynamic IR. The grid itself must respond. Staggering clock edges across banks (deliberate small skew) reduces peak dI/dt far more cheaply than decap.

Note the trade-off: decap cells leak. A design that fills all whitespace with decap can add several mW of leakage. Fill with decap where switching is dense, plain filler elsewhere.

16.4 Aging and margin

NBTI shifts PMOS V_{th} over years of operation, slowing the circuit. Modern flows apply an *aging derate* (e.g. +3% delay at end-of-life) or run explicitly aged libraries. If a competition mentions “lifetime” or “reliability,” this is what it means: you must close timing with the aged model, not the fresh one.

CHAPTER 17

Multi-Objective Optimization and the PPA Trade-off

17.1 Why single-objective thinking loses

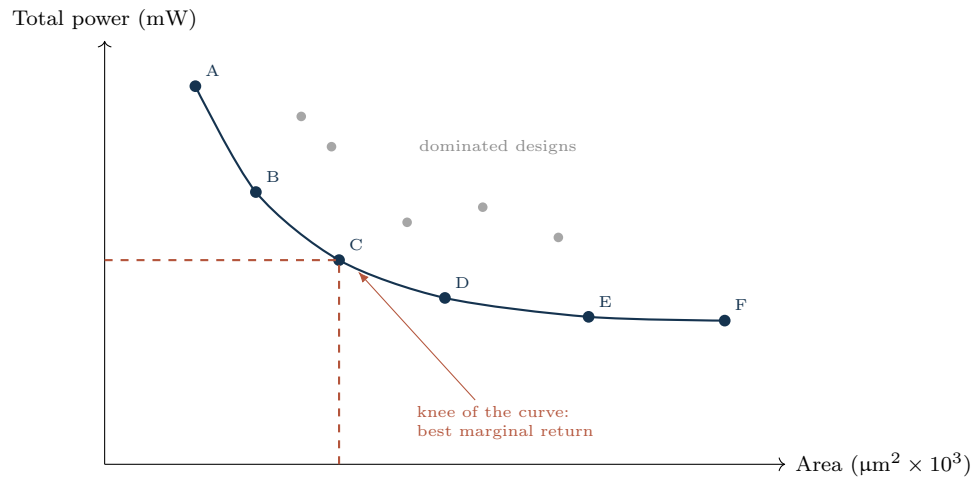
Each of the previous chapters optimized one thing. A real design — and a real competition — scores a weighted combination. The techniques interact, often adversarially.

Move	Timing	Area	Dyn. power	Leakage	Congestion
Upsize a gate	++	—	—	—	—
Downsize a gate	—	+	+	+	+
LVT swap	++	=	=	--	=
HVT swap	—	=	=	++	=
Insert buffer	+	—	—	—	—
Remove buffer	—	+	+	+	+
Clock gating	=	—	++	=	—
Useful skew	++	=	=	=	=
Raise utilization	±	++	+	=	--
Lower utilization	±	--	—	=	++
Widen power grid	+	=	=	=	--
Clock NDR / shielding	++	=	—	=	--
Scan reorder	=	=	+	=	++
HFNS on reset	+	—	—	—	++
Lower V_{DD}	--	=	++	+	=

++ strongly helps, + helps, = neutral, — hurts, -- strongly hurts, ± depends.

17.2 Pareto fronts

A design point is *Pareto-optimal* if no other reachable point is better in one objective without being worse in another. The set of such points is the Pareto front. Your goal is not “the best design” — it is *a point on the front that matches the scoring weights*.



Finding the knee

Between C and D you pay $1,400 \mu\text{m}^2$ to save 0.5 mW . Between D and E you pay $1,900 \mu\text{m}^2$ to save 0.25 mW — less than half the return for more than the cost. **The knee is where the marginal rate of substitution crosses the ratio implied by the scoring weights.** If the rulebook says $1 \text{ mW} \equiv 2,000 \mu\text{m}^2$, then C is your answer, not F.

17.3 Design of experiments: sweeping systematically

Do not tune one parameter at a time by intuition. Sweep.

Parameter	Typical sweep	What it reveals
Core utilization	60, 65, 70, 75, 80%	Area/congestion knee
Aspect ratio	0.7, 0.85, 1.0, 1.2, 1.5	Wirelength and CTS sensitivity
Synthesis clock margin	0, 5, 10, 15%	Area vs. closability
Max transition	60, 90, 120, 150 ps	Power/area vs. delay accuracy
V_t mix ratio	0, 20, 40, 60% LVT	Leakage/timing front
Placement congestion effort	low/med/high	Timing vs. routability
CTS target skew	20, 40, 60, 80 ps	Clock power vs. timing
Power strap pitch	20, 30, 40, 60 μm	IR vs. congestion

Sweep discipline

1. Fix a baseline and record every metric for it.
2. Change *one* parameter. Record all metrics again, including ones you did not expect to move.
3. Plot. Always plot — the knee is visible in a plot and invisible in a table of six numbers.
4. Combine only the settings that were individually good *and* whose interactions you have reasoned about.
5. Re-run the combination. Interactions are frequently non-additive.

Script this. A shell loop that launches 5 utilization runs overnight is worth more than a day of manual tuning.

17.4 A scoring-aware decision procedure

1. **Read the scoring function.** If none is given, assume: hard constraints (DRC/LVS/timing) then area, then power, then wirelength.
2. **Establish a legal baseline first.** A clean, unoptimized, mediocre-scoring design in hand beats a brilliant one that does not close. Save it. Tag it. Never lose it.
3. **Compute the exchange rate** between objectives from the weights.
4. **Apply free moves first** (pin swap, CRPR, exception cleanup, stale uncertainty removal). These are pure gain.
5. **Apply single-axis moves** where you have slack in the other axes.
6. **Sweep the two or three parameters with the largest leverage.**
7. **Stop when the marginal score gain per hour of work falls below the risk of breaking something.** This is a real engineering judgement and judges respect it.

The most common way strong teams lose

Over-optimizing until 30 minutes before the deadline, then discovering a DRC violation with no time to fix it. **Freeze your design one hour early.** Run the full verification suite. Submit the clean version. Then, if time remains, try the aggressive version — from a copy.

Part IV

Algorithms Inside the Tools

CHAPTER 18

Partitioning, Floorplan Representations, and Placement Mathematics

18.1 Why you should know these

You will not implement a router in a weekend contest. But knowing the algorithm tells you *what the tool can and cannot do*, which is exactly the knowledge that lets you set the right options and diagnose a bad result. When global placement produces a lopsided design, knowing that it solved a quadratic system with spreading forces tells you the fix is a density constraint, not a net weight.

18.2 Partitioning

18.2.1 The problem

Given a hypergraph $G = (V, E)$ of cells and nets, split V into k balanced blocks minimizing the number of nets cut. NP-hard; solved heuristically.

18.2.2 Kernighan–Lin (KL)

Bipartitioning by pairwise swaps.

1. For each cell a compute $D_a = E_a - I_a$ (external minus internal edge weight).
2. For each cross-partition pair (a, b) , gain $g_{ab} = D_a + D_b - 2c_{ab}$.
3. Swap the highest-gain pair, lock both, update all D values.
4. Repeat until all cells are locked, giving a sequence of gains $g_1, \dots, g_n$.
5. Find k maximizing $\sum_{i=1}^k g_i$. If positive, apply the first k swaps and start a new pass. If not, stop.

Complexity $O(n^3)$ per pass (or $O(n^2 \log n)$ with sorting). Handles only balanced bipartition of equal-size cells.

18.2.3 Fiduccia–Mattheyses (FM)

The practical improvement:

- Moves *one* cell at a time rather than swapping pairs — handles unequal cell sizes and area balance constraints.
- Uses a *bucket list* data structure indexed by gain, so selecting the best move is $O(1)$ and updating gains after a move is $O(\text{pins})$.

- Overall $O(P)$ per pass where P is the total pin count — linear.
- Operates on hypergraphs directly (a net with k pins is cut once, not $\binom{k}{2}$ times).

Example 17.1 — One FM move

Partition $A = \{1, 2, 3\}$, $B = \{4, 5, 6\}$. Nets: $n_1 = \{1, 2, 4\}$, $n_2 = \{2, 3\}$, $n_3 = \{3, 5, 6\}$, $n_4 = \{1, 4\}$. Compute the FM gain of moving cell 3 from A to B.

Solution

FM gain of moving cell $v = (\text{nets that become uncut}) - (\text{nets that become cut})$.

Nets containing cell 3: $n_2 = \{2, 3\}$ and $n_3 = \{3, 5, 6\}$.

n_2 : currently entirely in A (cells 2 and 3 both in A) $\Rightarrow$ uncut. Moving 3 to B makes it cut.

Gain contribution: -1 .

n_3 : cell 3 in A, cells 5 and 6 in B $\Rightarrow$ currently cut. Moving 3 to B puts all of n_3 in B $\Rightarrow$ uncut. **Gain contribution:** $+1$.

Total gain $= -1 + 1 = 0$.

A zero-gain move. FM *accepts* zero and even negative gain moves within a pass (that is the point of the “find best prefix k ” step) — this is how it escapes local minima, exactly like simulated annealing but deterministically.

Check balance: after the move $A = \{1, 2\}$ (2 cells), $B = \{4, 5, 6, 3\}$ (4 cells). If the balance constraint is ± 1 cell, this move is *illegal* and FM would not consider it. Balance constraints matter as much as gain.

18.2.4 Multilevel partitioning (hMETIS and successors)

Modern partitioners *coarsen* the hypergraph by merging strongly connected cells, partition the tiny coarsened graph optimally, then *uncoarsen* while refining with FM at each level. This is what makes million-cell partitioning tractable, and the same coarsen–solve–refine pattern reappears in placement.

18.3 Floorplan representations

To search over floorplans you need an encoding that (a) represents only legal non-overlapping floorplans, (b) is compact, and (c) admits local moves.

Representation	Encoding	Properties
Slicing tree / Polish expression	Binary tree of H and V cuts, e.g. 12H3V45HV	Only slicing floorplans; $O(n)$ area evaluation via shape curves; small solution space
Sequence pair	Two permutations (Γ_+, Γ_-)	Represents <i>all</i> floorplans including non-slicing; $O(n \log \log n)$ evaluation; $(n!)^2$ space
B*-tree	Ordered binary tree, left child = right neighbour, right child = above	Compact, $O(n)$ packing, admissible; very popular
O-tree	Similar, $2n$ bits + permutation	Compact

Representation	Encoding	Properties
Corner Block List	Triple of lists	$O(n)$
Transitive Closure Graph	Two constraint graphs	Direct geometric meaning

18.3.1 Polish expressions

A *normalized* Polish expression of n blocks has $2n - 1$ symbols with no two consecutive identical operators. Legal moves:

- M1: swap two adjacent operands.
- M2: complement a chain of operators ($H \leftrightarrow V$).
- M3: swap an adjacent operand and operator (must check normality).

Combined with simulated annealing, this was the classic Wong–Liu floorplanner.

Example 17.2 — Evaluating a Polish expression

Blocks: 1 (4×3), 2 (3×5), 3 (6×2). Evaluate the floorplan 12V3H. (Convention: V = side by side, H = one above the other.)

Solution

1 2 V: side by side.

$$W = 4 + 3 = 7, \quad H = \max(3, 5) = 5$$

Sub-block is 7×5 .

(12V) 3 H: stacked.

$$W = \max(7, 6) = 7, \quad H = 5 + 2 = 7$$

Final: $7 \times 7 = 49$ **area.** Block area = $12 + 15 + 12 = 39$, so dead space = 10, i.e. 20.4%.

Try 12H3V: 12H gives $W = \max(4, 3) = 4$, $H = 3 + 5 = 8$, so 4×8 . Then V with block 3: $W = 4 + 6 = 10$, $H = \max(8, 2) = 8$. Area = 80. **Much worse.**

Try 13V2H: 1,3 side by side: $W = 4 + 6 = 10$, $H = \max(3, 2) = 3$. Then H with 2: $W = \max(10, 3) = 10$, $H = 3 + 5 = 8$. Area = 80.

Try 23V1H: $W = 3 + 6 = 9$, $H = \max(5, 2) = 5$; then $W = \max(9, 4) = 9$, $H = 5 + 3 = 8$; area = 72.

So 12V3H at 49 is much the best of these — and this tiny search is exactly what simulated annealing automates over thousands of moves. Note also how sensitive area is to the encoding: a 63% spread from four permutations of three blocks.

18.3.2 Sequence pair decoding

Given (Γ_+, Γ_-) :

- If a precedes b in *both* sequences, a is **left of** b .
- If a precedes b in Γ_+ but follows in Γ_- , a is **below** b .

Build the horizontal and vertical constraint graphs from these relations; the longest path in each gives the packed width and height.

Example 17.3 — Decoding a sequence pair

$\Gamma_+ = (A, B, C)$, $\Gamma_- = (C, A, B)$. Blocks: $A = 3 \times 2$, $B = 2 \times 4$, $C = 4 \times 3$. Find the packed dimensions.

Solution

Pairwise relations:

- (A, B) : A before B in Γ_+ ; A before B in $\Gamma_- \Rightarrow$ **A left of B** .
- (A, C) : A before C in Γ_+ ; A after C in $\Gamma_- \Rightarrow$ **A above C** .
- (B, C) : B before C in Γ_+ ; B after C in $\Gamma_- \Rightarrow$ **B above C** .

Horizontal constraint graph (edges = “left of”): $A \rightarrow B$ only. Longest path: $w_A + w_B = 3 + 2 = 5$. But C is unconstrained horizontally, so $W = \max(5, w_C) = \max(5, 4) = 5$.

Vertical constraint graph (edges = “below”): $C \rightarrow A$, $C \rightarrow B$. Longest path: $h_C + \max(h_A, h_B) = 3 + \max(2, 4) = 7$. So $H = 7$.

Packed area $= 5 \times 7 = 35$, versus block area $6 + 8 + 12 = 26$. Dead space $= 9$ (25.7%).

Placement: C at $(0, 0)$ occupying 4×3 ; A at $(0, 3)$; B at $(3, 3)$.

18.4 Placement algorithms

18.4.1 Simulated annealing (TimberWolf lineage)

Accept a move that worsens cost with probability $e^{-\Delta C/T}$; lower T on a schedule. Extremely general and quality-competitive, but too slow for million-cell designs. Still used for macro placement, where n is small.

18.4.2 Min-cut / recursive bisection (Capo)

Recursively partition the netlist and the region together. Elegant, congestion-aware by construction, but struggles to optimize timing directly.

18.4.3 Quadratic placement

Model each net as springs; minimize

$$\Phi(x, y) = \frac{1}{2} \sum_{i,j} w_{ij} [(x_i - x_j)^2 + (y_i - y_j)^2]$$

Setting $\partial\Phi/\partial x = 0$ gives a sparse linear system

$$\mathbf{A} \mathbf{x} = \mathbf{b}_x, \quad \mathbf{A} \mathbf{y} = \mathbf{b}_y$$

where $\mathbf{A}$ is the (positive definite) Laplacian of the weighted netlist graph with fixed pads on the diagonal, and $\mathbf{b}$ carries the fixed pin positions. Solvable by conjugate gradient in near-linear time.

Why quadratic placement alone fails

The unconstrained minimum piles every movable cell on top of the others in the centre — a “ball of wool.” Real quadratic placers therefore alternate *solve* with *spreading*: add pseudo-nets pulling cells toward under-full bins, re-solve, repeat. This is the classic GORDIAN / FASTPLACE / KRAFTWERK family.

Example 17.4 — A 1-D quadratic placement

Three movable cells x_1, x_2, x_3 and two fixed pads at $x = 0$ and $x = 10$. Nets (all weight 1): (pad_0, x_1) , (x_1, x_2) , (x_2, x_3) , (x_3, pad_{10}) , (x_1, x_3) . Solve for the optimal positions.

Solution

Cost $\Phi = \frac{1}{2}[(x_1 - 0)^2 + (x_1 - x_2)^2 + (x_2 - x_3)^2 + (x_3 - 10)^2 + (x_1 - x_3)^2]$.

Partial derivatives set to zero:

$$\begin{aligned} \frac{\partial \Phi}{\partial x_1} &= x_1 + (x_1 - x_2) + (x_1 - x_3) = 0 & \Rightarrow 3x_1 - x_2 - x_3 &= 0 \\ \frac{\partial \Phi}{\partial x_2} &= (x_2 - x_1) + (x_2 - x_3) = 0 & \Rightarrow -x_1 + 2x_2 - x_3 &= 0 \\ \frac{\partial \Phi}{\partial x_3} &= (x_3 - x_2) + (x_3 - 10) + (x_3 - x_1) = 0 & \Rightarrow -x_1 - x_2 + 3x_3 &= 10 \end{aligned}$$

In matrix form:

$$\begin{pmatrix} 3 & -1 & -1 \\ -1 & 2 & -1 \\ -1 & -1 & 3 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 10 \end{pmatrix}$$

From eq. 2: $x_2 = (x_1 + x_3)/2$. Substituting into eq. 1:

$$3x_1 - \frac{x_1 + x_3}{2} - x_3 = 0 \Rightarrow 6x_1 - x_1 - x_3 - 2x_3 = 0 \Rightarrow 5x_1 = 3x_3 \Rightarrow x_1 = 0.6x_3$$

Then $x_2 = (0.6x_3 + x_3)/2 = 0.8x_3$. Substituting into eq. 3:

$$-0.6x_3 - 0.8x_3 + 3x_3 = 10 \Rightarrow 1.6x_3 = 10 \Rightarrow x_3 = 6.25$$

$$\boxed{x_1 = 3.75, \quad x_2 = 5.00, \quad x_3 = 6.25}$$

Observe the clustering: all three cells sit in the middle third of a 10-unit span, exactly the “ball of wool” pathology. A spreading step would now add pseudo-nets pulling x_1 toward 2 and x_3 toward 8 and re-solve.

18.4.4 Nonlinear / analytical placement (the modern standard)

Replace HPWL with a smooth approximation, typically log-sum-exp:

$$\text{HPWL}_x(e) \approx \gamma \left[\log \sum_{i \in e} e^{x_i/\gamma} + \log \sum_{i \in e} e^{-x_i/\gamma} \right]$$

which converges to the true HPWL as $\gamma \rightarrow 0$. Density is handled as a constraint via a penalty (e.g. the electrostatic analogy in `EPLACE` / `REPLACE`, where cells are charges and density is Poisson’s equation). Minimize with Nesterov’s accelerated gradient. This is what modern commercial and open-source placers do.

18.4.5 Legalization

- **Tetris** (greedy, left-to-right per row): simple and fast, poor displacement quality.
- **Abacus**: assigns cells to rows and solves an optimal *single-row* placement by dynamic programming to minimize total displacement. Standard.

- **Min-cost flow** formulations: highest quality, slowest.

18.4.6 Detailed placement

Local search: swap adjacent cells, mirror cells, slide within whitespace, and *optimal single-row reordering* (a small TSP-like DP over windows of 3–6 cells).

18.5 Routing algorithms

18.5.1 Lee’s maze router

BFS from source, labelling each grid cell with its distance, until the target is labelled; then backtrack. Guarantees the shortest path. $O(WH)$ time and memory for a $W \times H$ grid — prohibitive at chip scale, hence:

- **A***: add heuristic $h(n)$ = Manhattan distance to target; expands far fewer cells and remains optimal since Manhattan distance is admissible in a grid graph.
- **Coding schemes** (Akers): store 1, 1, 2, 2, 3, 3, ... mod 3 instead of full distances, cutting memory to 2 bits per cell.
- **Framing**: restrict search to a bounding box around source and target.

18.5.2 Steiner trees

For a multi-pin net, the optimal topology is a Rectilinear Steiner Minimal Tree. NP-hard, but:

- **Hanan’s theorem**: an optimal RSMT exists using only the grid formed by horizontal and vertical lines through the pins — reducing an infinite search to $O(n^2)$ candidate points.
- **FLUTE**: lookup-table based, exact for ≤ 9 pins, near-optimal and extremely fast beyond. Used inside almost every modern placer.
- **Bound**: $\text{RSMT} \geq \frac{2}{3} \text{RMST}$, so a minimum spanning tree is a 1.5-approximation.

Example 17.5 — MST vs. Steiner

Four pins at (0,0), (4,0), (0,3), (4,3). Compute rectilinear MST length and the optimal Steiner tree length.

Solution

Rectilinear (Manhattan) distances: (0,0)–(4,0): 4. (0,0)–(0,3): 3. (4,0)–(4,3): 3. (0,3)–(4,3): 4. Diagonals: 7 each.

MST: take the three cheapest edges forming a tree: $3 + 3 + 4 = 10$.

Steiner tree: run a vertical trunk at $x = 0$ from $y = 0$ to $y = 3$ (length 3), then two horizontal branches at $y = 0$ and $y = 3$ of length 4 each = 8. Total 11 — worse.

Better: a horizontal trunk at $y = 0$ (length 4) plus verticals at $x = 0$ and $x = 4$ of length 3 each: $4 + 3 + 3 = 10$. Same as MST.

For this symmetric rectangle no Steiner point helps — MST is already optimal. **Now move a pin**: put pins at (0,0), (4,0), (2,5). MST: (0,0)–(4,0) = 4; (4,0)–(2,5) = 7. Total 11. Steiner: trunk from (2,0) up to (2,5) = 5, plus horizontals (0,0)–(2,0) = 2 and (2,0)–(4,0) = 2. Total 9. **Steiner saves 18%** — and this three-pin case is exactly the situation FLUTE solves exactly and instantly.

18.5.3 Channel routing — the left-edge algorithm

For a rectangular channel with pins on top and bottom:

1. Build the **vertical constraint graph**: an edge $a \rightarrow b$ if net a has a top pin and net b a bottom pin at the same column.
2. Build the **horizontal constraint graph** (interval overlap).
3. Sort nets by leftmost column; assign each to the first track where it neither overlaps horizontally nor violates a vertical constraint.

Lower bound on tracks = *channel density* = max number of nets crossing any column. A cycle in the vertical constraint graph makes the channel unroutable without a *dogleg* (splitting a net across two tracks).

Example 17.6 — Channel density and tracks

Top pins (left to right): 1, 2, 0, 3, 2. Bottom pins: 0, 1, 3, 0, 3. (0 means no pin.) Find the net spans, the channel density, and a track assignment.

Solution

Columns 1–5. Net occurrences:

- Net 1: top col 1, bottom col 2 $\Rightarrow$ span $[1, 2]$.
- Net 2: top col 2, top col 5 $\Rightarrow$ span $[2, 5]$.
- Net 3: top col 4, bottom col 3, bottom col 5 $\Rightarrow$ span $[3, 5]$.

Density per column:

Column	1	2	3	4	5
Nets crossing	{1}	{1,2}	{2,3}	{2,3}	{2,3}
Density	1	2	2	2	2

Channel density = 2, so at least 2 tracks are needed.

Vertical constraints: column 2 has net 2 on top, net 1 on bottom $\Rightarrow 2 \rightarrow 1$ (net 2 must be above net 1). Column 5 has net 2 on top, net 3 on bottom $\Rightarrow 2 \rightarrow 3$. No cycles.

Left-edge assignment (sort by left endpoint: net 1 at 1, net 2 at 2, net 3 at 3):

- Track 1 (topmost): net 2 (must be above 1 and 3), span $[2, 5]$.
- Track 2: net 1 $[1, 2]$ and net 3 $[3, 5]$ — they do not overlap, so both fit on one track.

2 tracks — optimal, matching the density lower bound.

18.5.4 Negotiated congestion (PathFinder)

The idea that made modern global routing work:

1. Route every net by shortest path, allowing overuse.
2. Assign each resource a cost $c = (b + h) \cdot p$, where b is the base cost, h the *historical* congestion accumulated over iterations, and p the *present* sharing factor.
3. Rip up and reroute all nets with the updated costs.
4. Increase the penalties; repeat until no resource is overused.

Nets “negotiate”: a net with an easy alternative yields; a net with no alternative keeps the resource.

18.6 Clock tree algorithms

18.6.1 Deferred Merge Embedding (DME)

Builds an exact zero-skew tree in two passes:

- **Bottom-up:** for each internal node, compute the *merging segment* — the locus of points from which both subtrees have equal delay. Under the linear delay model this locus is a Manhattan arc (a 45° segment).
- **Top-down:** choose an actual embedding point on each merging segment, starting from the root, minimizing total wirelength.

DME gives exact zero skew with minimum wirelength *for a given topology*; choosing the topology (e.g. by nearest-neighbour matching, as in the Greedy-DME / CL method) is the harder part.

18.6.2 Zero-skew merge point

For two subtrees with delays t_1, t_2 and load capacitances C_1, C_2 , joined by a wire of total length L split as x and $L - x$, with unit resistance r and unit capacitance c , the Elmore-balanced condition is

$$t_1 + rx \left(\frac{cx}{2} + C_1 \right) = t_2 + r(L - x) \left(\frac{c(L - x)}{2} + C_2 \right)$$

Solving for x :

$$x = \frac{(t_2 - t_1) + rL \left(\frac{cL}{2} + C_2 \right)}{rL(cL + C_1 + C_2)}$$

If $x \notin [0, L]$, the segment is too short to balance and you must *snake* (elongate) the shorter branch.

Example 17.7 — Zero-skew merge

Two sinks: $t_1 = 0$, $C_1 = 20$ fF; $t_2 = 0$, $C_2 = 60$ fF. Wire between them $L = 200$ μm , $r = 0.2 \Omega/\mu\text{m}$, $c = 0.2$ fF/ μm . Where is the zero-skew tap point?

Solution

$rL = 0.2 \times 200 = 40 \Omega$. $cL = 0.2 \times 200 = 40$ fF.

$$x = \frac{0 + 40 \left(\frac{40}{2} + 60 \right)}{40(40 + 20 + 60)} = \frac{40(80)}{40(120)} = \frac{3200}{4800} = 0.667$$

So $x = 0.667$ in normalized units — careful: the formula’s x is a *length*. Redo with explicit lengths, x in μm , r per μm :

$$x = \frac{(t_2 - t_1) + rL \left(\frac{cL}{2} + C_2 \right)}{r(cL + C_1 + C_2)} \cdot \frac{1}{1}$$

Substituting $r = 0.2$, $L = 200$, $c = 0.2$:

$$x = \frac{0 + 40(20 + 60)}{0.2(40 + 20 + 60)} = \frac{3200}{0.2(120)} = \frac{3200}{24} = 133.3$$

$x = 133.3\mu\text{m}$ **from sink 1**, i.e. $66.7\mu\text{m}$ from sink 2.

Sanity check. Sink 2 has three times the capacitance of sink 1, so the tap point sits *closer to sink 2* — giving sink 2 less wire resistance to compensate for its larger load. That is exactly what we got (66.7 vs 133.3). ✓

Verify delays. Branch to sink 1: $R = 0.2(133.3) = 26.7\Omega$, wire $C = 0.2(133.3) = 26.7\text{fF}$. $t = 26.7(26.7/2 + 20) = 26.7(33.35) = 890\Omega\text{fF} = 0.89\text{ps}$. Branch to sink 2: $R = 0.2(66.7) = 13.3\Omega$, wire $C = 13.3\text{fF}$. $t = 13.3(13.3/2 + 60) = 13.3(66.65) = 887\Omega\text{fF} = 0.89\text{ps}$. ✓Balanced.

18.7 Van Ginneken buffer insertion

The optimal buffering problem: given an RC tree and a set of legal buffer positions, place buffers to maximize the required time at the source.

Algorithm. Process the tree bottom-up. At each node maintain a set of non-dominated *options*, each a pair $(C, q) = (\text{downstream capacitance}, \text{required arrival time})$. At each step:

- **Add wire:** $C \ += \ c\ell$; $q \ -= \ r\ell(c\ell/2 + C)$.
- **Insert buffer:** new option $(C_b, q - R_b C - t_b)$.
- **Merge branches:** $C = C_1 + C_2$, $q = \min(q_1, q_2)$, over the cross-product of option sets.
- **Prune:** discard any option (C_i, q_i) if some (C_j, q_j) has $C_j \leq C_i$ and $q_j \geq q_i$.

Pick the option with maximum q at the root. Complexity $O(n^2)$ for n legal positions; $O(n \log n)$ with better data structures. Pruning is what keeps the option set small in practice.

Key idea

Van Ginneken is a *dynamic program*, and its guarantee is optimality for a *fixed* topology and *fixed* buffer library under the Elmore model. That is why real tools give you good but not perfect buffering: the topology itself is being chosen heuristically, and Elmore is not the real delay model.

18.8 Complexity summary

Problem	Complexity	Practical method
Hypergraph partitioning	NP-hard	Multilevel FM (hMETIS)
Floorplanning	NP-hard	SA over sequence pair / B*-tree
Global placement	NP-hard	Analytical + density penalty
Legalization	Polynomial (per row)	Abacus DP
RSMT	NP-hard	FLUTE lookup
Global routing	NP-hard	Negotiated congestion
Detailed routing	NP-hard	Rip-up and reroute with A*
Zero-skew tree	Polynomial	DME
Buffer insertion (fixed topology)	Polynomial	Van Ginneken DP
Gate sizing (continuous)	Convex (geometric program)	Lagrangian relaxation
Gate sizing (discrete library)	NP-hard	Greedy + sensitivity
V_t assignment	NP-hard (knapsack-like)	Greedy by leakage/delay ratio

The one structural insight

Almost everything in physical design is NP-hard, and every tool is a heuristic. That means: (a) results depend on seeds, effort levels, and ordering; (b) running the same flow twice with different options genuinely gives different answers; and (c) **sweeping options is not laziness, it is the correct method.** Teams that run one flow and tune by hand lose to teams that run twelve flows and pick the best.

Part V

Practice

CHAPTER 19

Tools, Flows, and Scripts

19.1 The tool landscape

Task	Cadence	Synopsys	Open source
Synthesis	Genus	Design Compiler / Fusion Compiler	Yosys
Place & route	Innovus	IC Compiler II / Fusion Compiler	OpenROAD
STA	Tempus	PrimeTime	OpenSTA
Extraction	Quantus	StarRC	OpenRCX
Physical verification	Pegasus	IC Validator	KLayout, Magic
Power	Voltus	PrimePower / RedHawk-SC	—
Equivalence	Conformal	Formality	Yosys (eqy)
Full RTL-to-GDS	—	—	OpenLane / OpenROAD-flow-scripts

If you have no licences

Install **OpenLane** with the SkyWater 130 nm PDK. It is a complete, free, reproducible RTL-to-GDSII flow: Yosys → OpenROAD → Magic/KLayout → OpenSTA. Every concept in this book is exercisable in it, and the configuration file exposes precisely the knobs discussed in Part III. A single Docker command gets you running.

19.2 An annotated Innovus-style flow

Listing 19.1: Complete block-level PD flow, commented

```
1 #=====
2 # 0. SETUP
3 #=====
4 set DESIGN top
5 set STAGE_DIR ./dbs
```

```

6  file mkdir $STAGE_DIR rpt
7
8  set init_verilog      "netlist/${DESIGN}.v"
9  set init_design_netlisttype "Verilog"
10 set init_top_cell     $DESIGN
11 set init_lef_file     "lef/tech.lef lef/stdcells.lef lef/sram.lef"
12 init_design
13
14 #=====
15 # 1. FLOORPLAN
16 #=====
17 # aspect ratio 1.0, core utilization 0.70, 40um core-to-die on all sides
18 floorPlan -site core -r 1.0 0.70 40 40 40 40
19
20 # macro placement (manual is almost always better than automatic)
21 placeInstance u_sram0 60.0 600.0 MX
22 placeInstance u_sram1 60.0 120.0 R0
23 placeInstance u_sram2 700.0 600.0 MY
24 placeInstance u_sram3 700.0 120.0 R180
25
26 # keep standard cells off the macro pin edges
27 addHaloToBlock 12 12 12 12 -allBlock
28
29 # close the narrow channel between sram0 and sram2 if < 30 tracks wide
30 createPlaceBlockage -box 420 600 640 880 -type hard
31
32 # well taps and endcaps: MUST be before placement
33 addEndCap -preCap ENDCAP_L -postCap ENDCAP_R -prefix ENDCAP
34 addWellTap -cell TAPCELL -cellInterval 30 -prefix WELLTAP -checkerBoard
35
36 #=====
37 # 2. POWER PLAN
38 #=====
39 globalNetConnect VDD -type pgpin -pin VDD -all
40 globalNetConnect VSS -type pgpin -pin VSS -all
41
42 # core ring on the two thick top layers
43 addRing -nets {VDD VSS} -type core_rings -follow core \
44         -layer {top M8 bottom M8 left M7 right M7} \
45         -width 5 -spacing 2 -offset 2
46
47 # vertical straps: pitch is the IR-vs-congestion knob (sweep it!)
48 addStripe -nets {VDD VSS} -layer M8 -direction vertical \
49         -width 2.4 -spacing 1.8 -set_to_set_distance 40 \
50         -start_from left
51
52 # horizontal straps to form the mesh
53 addStripe -nets {VDD VSS} -layer M7 -direction horizontal \
54         -width 2.4 -spacing 1.8 -set_to_set_distance 60
55
56 # follow-pin rails on M1, one per row
57 sroute -connect {corePin blockPin padPin} -nets {VDD VSS} \
58         -allowJogging 0 -allowLayerChange 1
59
60 #=====
61 # 3. PLACEMENT
62 #=====
63 setPlaceMode -place_global_place_io_pins true \
64             -place_detail_legalization_inst_gap 2 \
65             -place_global_cong_effort high      ;# <-- congestion knob
66 setOptMode -fixCap true -fixTran true -fixFanoutLoad true
67
68 place_opt_design

```

```

69 # equivalent long form:
70 #   place_design ; optDesign -preCTS
71
72 # scan reorder: huge wirelength win, do it here
73 setScanReorderMode -keepScanChainCount true
74 scanReorder
75
76 timeDesign -preCTS -expandedViews -outDir rpt/preCTS
77 report_congestion > rpt/preCTS_cong.rpt
78 summaryReport -noHtml -outfile rpt/preCTS_summary.rpt
79 saveDesign ${STAGE_DIR}/place.enc
80
81 #=====
82 # 4. CTS
83 #=====
84 create_ccopt_clock_tree_spec
85 set_ccopt_property target_max_trans 0.080
86 set_ccopt_property target_skew 0.040
87 set_ccopt_property buffer_cells {CLKBUF8 CLKBUF12}
88 set_ccopt_property inverter_cells {CLKINV8 CLKINV12}
89 set_ccopt_property routing_top_min_fanout 1000
90
91 # non-default rule for the clock: wide + spaced + shielded
92 add_ndr -name CLK_NDR -width {M4 0.14 M5 0.14 M6 0.28} \
93         -spacing {M4 0.20 M5 0.20 M6 0.40}
94 set_ccopt_property -net_type trunk routing_rule CLK_NDR
95 set_ccopt_property -net_type leaf routing_rule CLK_NDR
96
97 ccopt_design ;# concurrent clock and data optimization
98
99 report_ccopt_clock_trees -filename rpt/clock_trees.rpt
100 report_ccopt_skew_groups -filename rpt/skew_groups.rpt
101 timeDesign -postCTS -hold -expandedViews -outDir rpt/postCTS
102 optDesign -postCTS -hold ;# hold violations are BORN here
103 saveDesign ${STAGE_DIR}/cts.enc
104
105 #=====
106 # 5. ROUTING
107 #=====
108 setNanoRouteMode -routeWithTimingDriven true \
109                 -routeWithSiDriven true \
110                 -drouteUseMultiCutViaEffort high \
111                 -routeWithViaInPin true
112 routeDesign
113
114 #=====
115 # 6. POST-ROUTE OPTIMIZATION AND SIGN-OFF
116 #=====
117 setExtractRCMode -engine postRoute -effortLevel high
118 extractRC
119
120 optDesign -postRoute -setup -hold -drv
121 optDesign -postRoute -si ;# crosstalk-aware
122
123 # area/power recovery on positive-slack paths
124 setOptMode -powerEffort high -leakageToDynamicRatio 0.5
125 optPower -postRoute
126
127 addFiller -cell {FILL1 FILL2 FILL4 FILL8 DCAP4 DCAP8} -prefix FILLER
128
129 verifyConnectivity -type all -report rpt/conn.rpt
130 verify_drc -report rpt/drc.rpt
131 verifyProcessAntenna -report rpt/antenna.rpt

```

```

132 report_power > rpt/power.rpt
133 timeDesign -signOff -expandedViews -outDir rpt/signoff
134
135 streamOut ${DESIGN}.gds -mapFile gds.map -stripes 1 -units 1000 \
136 -mode ALL
137 saveNetlist ${DESIGN}_pd.v

```

19.3 The equivalent OpenLane configuration

Listing 19.2: OpenLane config.tcl with the knobs that matter

```

1 set ::env(DESIGN_NAME) "top"
2 set ::env(VERILOG_FILES) "./src/*.v"
3 set ::env(CLOCK_PORT) "clk"
4 set ::env(CLOCK_PERIOD) "10.0" ;# ns
5
6 # ---- FLOORPLAN -----
7 set ::env(FP_SIZING) "relative"
8 set ::env(FP_CORE_UTIL) 45 ;# <-- SWEEP: 35 40 45 50 55
9 set ::env(FP_ASPECT_RATIO) 1
10 set ::env(FP_IO_HLENGTH) 4
11 set ::env(FP_PDN_VPITCH) 30 ;# <-- SWEEP: IR vs congestion
12 set ::env(FP_PDN_HPITCH) 30
13
14 # ---- PLACEMENT -----
15 set ::env(PL_TARGET_DENSITY) 0.50 ;# usually FP_CORE_UTIL/100 + 0.05
16 set ::env(PL_BASIC_PLACEMENT) 0
17 set ::env(PL_RESIZER_BUFFER_INPUT_PORTS) 1
18 set ::env(PL_RESIZER_HOLD_SLACK_MARGIN) 0.10
19 set ::env(PL_RESIZER_SETUP_SLACK_MARGIN) 0.05
20 set ::env(PL_RESIZER_MAX_WIRE_LENGTH) 600 ;# um; buffer anything longer
21
22 # ---- CTS -----
23 set ::env(CTS_TARGET_SKEW) 200 ;# ps
24 set ::env(CTS_ROOT_BUFFER) "sky130_fd_sc_hd__clkbuf_16"
25 set ::env(CTS_CLK_BUFFER_LIST) "sky130_fd_sc_hd__clkbuf_4 \
26 sky130_fd_sc_hd__clkbuf_8"
27
28 # ---- ROUTING -----
29 set ::env(GLB_RT_ADJUSTMENT) 0.15 ;# reserve 15% of tracks
30 set ::env(ROUTING_CORES) 8
31 set ::env(GRT_ALLOW_CONGESTION) 0
32
33 # ---- SYNTHESIS -----
34 set ::env(SYNTH_STRATEGY) "AREA 0" ;# or "DELAY 1" ... SWEEP THIS
35 set ::env(SYNTH_MAX_FANOUT) 10
36 set ::env(SYNTH_BUFFERING) 1
37 set ::env(SYNTH_SIZING) 1

```

19.4 A sweep harness

Listing 19.3: Sweep utilization and record every metric

```

1 #!/bin/bash
2 # sweep_util.sh -- run the flow across utilizations and tabulate results
3 DESIGN=top
4 echo "util,cell_area,die_area,wns,tns,power_mW,drc,wirelength" > sweep.csv
5

```

```

6  for U in 35 40 45 50 55 60; do
7      sed -i "s/FP_CORE_UTIL) ./FP_CORE_UTIL) $U/" config.tcl
8      sed -i "s/PL_TARGET_DENSITY) ./PL_TARGET_DENSITY) 0.$((U+5))/" config.tcl
9
10     flow.tcl -design $DESIGN -tag util_$U -overwrite > logs/util_$U.log 2>&1
11
12     R=runs/util_$U/reports
13     AREA=$(grep -m1 "Design area"    $R/synthesis/*stat* | awk '{print $3}')
14     WNS=$(grep -m1 "wns"              $R/signoff/*sta*   | awk '{print $2}')
15     TNS=$(grep -m1 "tns"              $R/signoff/*sta*   | awk '{print $2}')
16     PWR=$(grep -m1 "Total"            $R/signoff/*power*  | awk '{print $4}')
17     DRC=$(grep -c "violation"         $R/signoff/*drc*)
18     WL=$(grep -m1 "wire length"      $R/routing/*        | awk '{print $4}')
19
20     echo "$U,$AREA,,$WNS,$TNS,$PWR,$DRC,$WL" >> sweep.csv
21     echo "done util=$U wns=$WNS area=$AREA drc=$DRC"
22 done
23
24 column -s, -t sweep.csv

```

Do this on day one

Get *any* design through *any* full flow before you optimize anything. A team with a working baseline on day one and mediocre numbers beats a team with brilliant ideas and a broken flow on day three. Then run the sweep above overnight while you sleep. You will wake up with a Pareto curve.

CHAPTER 20

Reading Reports Like an Engineer

20.1 A timing report, dissected

Listing 20.1: Annotated report_timing output

```
1 Path 1: VIOLATED Setup Check with Pin u_ctrl/state_reg_2/CK
2 Endpoint: u_ctrl/state_reg_2/D (^) checked with leading edge of 'clk'
3 Beginpoint: u_dp/acc_reg_11/Q (v) triggered by leading edge of 'clk'
4 Path Groups: {clk}
5 Analysis View: setup_ss_125c
6
7 Other End Arrival Time          0.318      <-- capture clock insertion delay
8 - Setup                        0.052      <-- flop setup requirement
9 + Phase Shift                  1.000      <-- one clock period T
10 + CPPR Adjustment              0.021      <-- common path pessimism credited
    BACK
11 = Required Time                1.287
12 - Arrival Time                1.334
13 = Slack                       -0.047      <-- 47 ps short
14
15 # ----- launch clock path -----
16 Point                          Fanout  Cap    Trans  Incr    Arrival
17 clk (in)                      0.020  0.000  0.020  0.000  0.000
18 CLKBUF_root/Y                  18    0.180  0.041  0.081  0.081
19 CLKBUF_L1_3/Y                  12    0.121  0.038  0.069  0.150
20 CLKBUF_L2_7/Y                   9    0.094  0.035  0.062  0.212
21 CLKBUF_L3_22/Y                  4    0.048  0.028  0.051  0.263
22 acc_reg_11/CK                  0.028  0.034  0.297  <-- insertion
23 # ----- data path -----
24 acc_reg_11/Q                    3    0.031  0.045  0.112  0.409  <-- Tcq
25 U2891/Y (NAND2_X1)              2    0.029  0.061  0.058  0.467
26 U2892/Y (AOI22_X1)              1    0.018  0.052  0.071  0.538
27 U2893/Y (INV_X1)                 6    0.094  0.128  0.119  0.657  <-- SLEW! CAP!
28 U2894/Y (NAND3_X1)              1    0.014  0.049  0.093  0.750
29 U2895/Y (XOR2_X1)               2    0.145  0.171  0.204  0.954  <-- BIG net
30 U2896/Y (BUF_X2)                 1    0.022  0.048  0.058  1.012
31 U2897/Y (OAI21_X1)              1    0.016  0.044  0.062  1.074
32 ... (7 more stages) ...
33 state_reg_2/D                  0.041  0.000  1.334
```

20.1.1 How to read it in 30 seconds

1. **Slack** = -47 ps. Small. Not a structural problem.
2. **Look down the Incr column for outliers.** U2895 costs 204 ps — more than double any neighbour. Its load is 0.145 pF and its output transition is 171 ps. **That is your target.**

3. **Check upstream:** U2893 drives fanout 6 at 0.094 pF with a 128 ps transition — likely a max-transition DRV. Fixing it improves U2894 too.
4. **Count stages:** 13 logic stages in 1000 ps means ~ 77 ps per stage — reasonable, so the path is not pathologically deep.
5. **Insertion delay** 297 ps on a 1000 ps clock is high (30%). A shallower tree would help every path.
6. **CPPR is credited** (+21 ps) — good, it is enabled.

20.1.2 The fix, with estimated value

Action	Est. gain	Cost
Upsize U2895 (XOR2_X1 → X4)	60–90 ps	Area +3 μm^2 , load on U2894
Buffer the U2895 output net	30–50 ps	One buffer
Fix U2893 transition (upsize to X2)	20–35 ps	Small; also fixes a DRV
LVT swap U2893, U2895	40–60 ps	Leakage

Any *one* of these closes 47 ps. Choose based on what else you are optimizing — if leakage is scored, upsize; if area is scored, LVT swap.

20.2 A congestion report

Listing 20.2: Congestion summary

```

1 Total number of GCells      : 249,001
2 Overflow (H)                : 0.03% (74 gcells)
3 Overflow (V)                : 0.41% (1021 gcells)
4 Max H overflow              : 3
5 Max V overflow              : 11
6 Worst region                : (612,340) - (688,416)
7 Layer utilization:
8   M1 8.2%   M2 62.1%   M3 58.4%   M4 71.9%
9   M5 44.0%   M6 39.7%   M7 12.1%   M8 9.6%
```

Reading it: vertical overflow is 14 $\times$ horizontal, and M2/M4 (vertical layers here) are the loaded ones while M6–M8 sit nearly idle. The diagnosis is not “too dense” — it is **layer imbalance**. Fixes: relax the global-route layer adjustment on M5–M7, promote long nets upward, or check whether a routing blockage is wrongly excluding the upper layers. Max overflow of 11 in one localized region calls for a partial placement blockage there.

20.3 A power report

Listing 20.3: Power breakdown

Group	Internal	Switching	Leakage	Total	Percentage
Sequential	41.220	18.640	9.880	69.740	24.3%
Combinational	22.910	92.100	26.440	141.450	49.3%
Clock	0.000	69.180	1.220	70.400	24.5%

6	Macro	5.410	0.000	0.180	5.590	1.9%
7	-----					
8	Total	69.540	179.920	37.720	287.180	100.0%
9		24.2%	62.7%	13.1%		

Reading it:

- Clock is 24.5% of total and is almost *pure switching* (69.2 of 70.4). Clock gating is the highest-leverage move available.
- Combinational switching at 92 mW is the biggest single number. Attack via wirelength reduction, downsizing, and glitch reduction.
- Leakage is 13% — moderate. A multi- V_t pass is worth ~ 20 mW if slack allows.
- Macros are negligible here, so memory access optimization is not the lever.

20.4 A DRC report

Listing 20.4: Typical DRC summary — read the histogram, not the list

1	Verification Summary				
2	Cut.SP.1	(via cut spacing)	:	412	
3	M2.S.4	(EOL spacing)	:	238	
4	M3.W.1	(min width)	:	6	
5	M2.A.1	(min area)	:	89	
6	M4.S.2	(parallel-run spacing)	:	14	
7	Total		:	759	

The DRC triage rule

Do not fix 759 violations. Fix *four categories*. $412 + 238 = 650$ of the 759 are cut spacing and end-of-line — almost certainly one systematic cause, such as an over-aggressive via-in-pin setting or a track-offset mismatch. Identify the cause, change one setting, re-route, and watch 650 disappear at once. Only the residue needs individual attention.

20.5 Metrics table you should maintain for every run

Metric	Command	Why
Cell area, cell count	<code>report_design</code>	Baseline
Utilization	<code>report_design</code>	The main floorplan knob
WNS / TNS / NVP (setup)	<code>timeDesign -late</code>	Score
WNS / TNS (hold)	<code>timeDesign -early</code>	Must be ≥ 0
DRV count	<code>report_constraint</code>	Prerequisite for valid timing
Total wirelength	<code>report_route</code>	Power and congestion proxy
Overflow H/V	<code>report_congestion</code>	Routability
Total power (3 parts)	<code>report_power</code>	Score
DRC / LVS count	<code>verify_drc</code>	Pass/fail gate
Max IR drop	IR tool	Validity of timing
Clock insertion / skew	<code>report_ccopt</code>	Diagnostic
Runtime	—	Your own budget

Practice

Put these twelve numbers in a spreadsheet, one row per run, with a column for “what I changed.” By the end of a competition you will have thirty rows and the answer to every judge’s question will be in it. This single habit is worth more than any individual optimization technique.

CHAPTER 21

The Problem Bank: Simple to Complex

This chapter is the core of your practice. Problems are graded: **Tier 1** (P1–P10) are single-formula warm-ups; **Tier 2** (P11–P22) require two or three linked steps; **Tier 3** (P23–P32) are multi-constraint optimization decisions of the kind a competition actually poses. Work each one on paper before reading the solution.

21.1 Tier 1 — Fundamentals

P1 — Core sizing

A design has 62,000 cells averaging $1.28\text{ }\mu\text{m}^2$, plus two macros of $300\text{ }\mu\text{m} \times 180\text{ }\mu\text{m}$. Target utilization 68%, aspect ratio 1.25 (height/width). Find core width and height.

Solution

Cell area = $62,000 \times 1.28 = 79,360\text{ }\mu\text{m}^2$. Macro area = $2(300 \times 180) = 108,000\text{ }\mu\text{m}^2$. Total = $187,360\text{ }\mu\text{m}^2$. Core area = $187,360/0.68 = 275,529\text{ }\mu\text{m}^2$.

$W \cdot H = 275,529$ and $H = 1.25W \Rightarrow 1.25W^2 = 275,529 \Rightarrow W^2 = 220,424 \Rightarrow W = 469.5\text{ }\mu\text{m}$, $H = 586.8\text{ }\mu\text{m}$.

$470\text{ }\mu\text{m} \times 587\text{ }\mu\text{m}$. Note the macros alone are $108,000\text{ }\mu\text{m}^2$ and a $300\text{ }\mu\text{m}$ -wide macro barely fits in a $470\text{ }\mu\text{m}$ core — *check macro fit before accepting an aspect ratio*. Two $300\text{ }\mu\text{m}$ macros side by side need $600\text{ }\mu\text{m}$; they must be stacked vertically here.

P2 — Setup slack

$T = 800\text{ ps}$, $T_{cq} = 75\text{ ps}$, $T_{logic} = 640\text{ ps}$, $T_{setup} = 45\text{ ps}$, uncertainty = 35 ps , skew = -15 ps (capture clock *earlier*). Slack?

Solution

$$S = 800 + (-15) - 45 - 35 - 75 - 640 = -10\text{ ps}$$

Violating by 10 ps . The negative skew alone accounts for 15 ps — fixing the clock balance would close this path with no logic change.

P3 — Hold slack

$T_{cq,min} = 48$ ps, $T_{logic,min} = 35$ ps, skew = +40 ps, $T_{hold} = 25$ ps, hold uncertainty = 15 ps. Slack? How many 30 ps delay cells are needed?

Solution

$$S = 48 + 35 - 40 - 25 - 15 = +3 \text{ ps}$$

Met, barely. **Zero** delay cells needed — but with only 3 ps of margin this path will fail after any routing change or on a different corner. Add one delay cell (+30 ps $\Rightarrow$ +33 ps slack) as insurance if area permits.

P4 — Maximum frequency

Longest path: $T_{cq} = 90$ ps, logic = 1,240 ps, $T_{setup} = 55$ ps, uncertainty = 60 ps, skew = 0. What is f_{max} ?

Solution

$$T_{min} = 90 + 1240 + 55 + 60 = 1445 \text{ ps.}$$

$$f_{max} = \frac{1}{1.445 \times 10^{-9}} = \mathbf{692 \text{ MHz}}$$

Note the 60 ps uncertainty costs 4.2% of frequency by itself. After CTS, replacing the estimate with measured skew is often worth 20–40 MHz for free.

P5 — Dynamic power

$C_L = 4.2$ pF, $V_{DD} = 1.0$ V, $f = 500$ MHz, activity factor $\alpha = 0.15$. Dynamic power?

Solution

$$P = \alpha C V^2 f = 0.15 \times 4.2 \times 10^{-12} \times 1.0 \times 5 \times 10^8 = 3.15 \times 10^{-4} \text{ W} = \mathbf{0.315 \text{ mW}}$$

Now drop V_{DD} to 0.85 V: $P = 0.315 \times (0.85)^2 = 0.228$ mW, a 27.5% reduction from a 15% voltage cut. The square law at work.

P6 — HPWL

Net pins at (120, 340), (275, 180), (90, 410), (310, 295), (200, 150). Compute HPWL.

Solution

x : min 90, max 310 $\Rightarrow$ 220. y : min 150, max 410 $\Rightarrow$ 260. HPWL = **480** μm .

At 0.2 fF/ μm that is 96 fF of net capacitance — a substantial load. This net is a buffering candidate.

P7 — Elmore delay, chain

A wire modelled as three segments in series, each $R = 40 \Omega$, $C = 10$ fF, with a load $C_L = 25$ fF at the end. Compute Elmore delay to the load.

Solution

Using the Π /segment convention where each segment's capacitance sits at its far node:

$$\begin{aligned} T &= R_1(C_1 + C_2 + C_3 + C_L) + R_2(C_2 + C_3 + C_L) + R_3(C_3 + C_L) \\ &= 40(10 + 10 + 10 + 25) + 40(10 + 10 + 25) + 40(10 + 25) \\ &= 40(55) + 40(45) + 40(35) = 2200 + 1800 + 1400 = 5400 \text{ } \Omega \text{ fF} = \mathbf{5.4 \text{ ps}} \end{aligned}$$

Compare the lumped model: $R_{tot} = 120$, $C_{tot} = 55$, $0.5RC = 3.3 \text{ ps}$ — the distributed calculation is 64% larger. Lumping under-estimates; always distribute long wires.

P8 — Antenna ratio

Metal length $85 \mu\text{m}$, width $0.10 \mu\text{m}$, gate area $0.012 \mu\text{m}^2$, limit 500. Pass or fail? Max legal length?

Solution

Metal area = $8.5 \mu\text{m}^2$. $AR = 8.5/0.012 = \mathbf{708}$. **Fails**. Max legal metal area = $500 \times 0.012 = 6.0 \mu\text{m}^2 \Rightarrow \text{max length} = 6.0/0.10 = \mathbf{60 \mu\text{m}}$. Jump layers after $60 \mu\text{m}$.

P9 — IR drop

A rail carries 18 mA through a resistance of 0.55Ω . $V_{DD} = 0.9 \text{ V}$. Is the 2% static budget met?

Solution

$V_{drop} = 0.018 \times 0.55 = 9.9 \text{ mV}$. Budget = $0.02 \times 0.9 = 18 \text{ mV}$. **Met**, using 55% of budget. But remember VSS drops too. If the return path has similar resistance, the *effective* supply seen by the cell is $0.9 - 0.0099 - 0.0099 = 0.880 \text{ V}$, a 2.2% total drop — **over budget**. Always check both rails.

P10 — Track supply

A $20 \mu\text{m}$ gcell, M3 pitch $0.14 \mu\text{m}$, M5 pitch $0.28 \mu\text{m}$ (both horizontal), 15% blocked. Horizontal track supply?

Solution

M3: $20/0.14 = 142.9 \rightarrow 142$ tracks. M5: $20/0.28 = 71.4 \rightarrow 71$. Total = 213. After blockage: $213 \times 0.85 = \mathbf{181}$ tracks.

21.2 Tier 2 — Linked Reasoning

P11 — Buffer count for a long net

A net of $2,600 \mu\text{m}$ must be buffered. $r = 0.18 \Omega/\mu\text{m}$, $c = 0.22 \text{ fF}/\mu\text{m}$, buffer: $t_b = 30 \text{ ps}$, $R_b = 1.5 \text{ k}\Omega$, $C_b = 5 \text{ fF}$. Find k_{opt} , buffer pitch, and the resulting delay. Compare to unbuffered.

Solution

$$rc = 0.18 \times 0.22 = 0.0396 \text{ } \Omega \text{ fF}/\mu\text{m}^2 = 0.0396 \text{ ps}/\mu\text{m}^2.$$

$$t_b + 0.69R_bC_b = 30 + 0.69(1500 \times 5 \times 10^{-15}) = 30 + 0.69(7.5 \text{ ps}) = 30 + 5.18 = 35.18 \text{ ps}.$$

$$k_{opt} = 2600 \sqrt{\frac{0.345 \times 0.0396}{35.18}} = 2600 \sqrt{3.883 \times 10^{-4}} = 2600(0.01971) = \mathbf{51}$$

$$\text{Pitch} = 2600/51 = \mathbf{51\mu\text{m}}.$$

$$\text{Delay: } 51(35.18) + 0.345(0.0396)(2600^2)/51 = 1794 + 1858 = \mathbf{3652 \text{ ps}}.$$

Unbuffered: $0.38rcL^2 = 0.38(0.0396)(6.76 \times 10^6) = 101,700 \text{ ps} = 101.7 \text{ ns}$. **Buffering gives a 28× improvement.**

Area cost: 51 buffers at $\sim 2\mu\text{m}^2 = 102 \mu\text{m}^2$. Trivially worth it.

P12 — Logical effort path sizing

Path: NAND2 $\rightarrow$ NOR2 $\rightarrow$ INV, driving $C_{out} = 48 \times C_{in,1}$, no branching. Find $\hat{f}$, D_{min} , and each gate's input capacitance relative to the first.

Solution

$$G = \frac{4}{3} \times \frac{5}{3} \times 1 = \frac{20}{9} = 2.222. \quad H = 48. \quad F = 106.7. \quad N = 3.$$

$$\hat{f} = 106.7^{1/3} = 4.745$$

$$P = 2 + 2 + 1 = 5. \quad D_{min} = 3(4.745) + 5 = \mathbf{19.2\tau}.$$

Sizes (backwards, $C_{in,i} = g_i C_{out,i} / \hat{f}$, with $C_{in,1} \equiv 1$):

$$\text{INV, } C_{out} = 48 : \quad C_{in,3} = 1(48)/4.745 = 10.12$$

$$\text{NOR2, } C_{out} = 10.12 : \quad C_{in,2} = \frac{5}{3}(10.12)/4.745 = 3.55$$

$$\text{NAND2, } C_{out} = 3.55 : \quad C_{in,1} = \frac{4}{3}(3.55)/4.745 = 1.00 \checkmark$$

Is $N = 3$ right? $\hat{f} = 4.75$ is above the ideal ~ 4 , so adding a stage might help. $N = 4$: $\hat{f} = 106.7^{1/4} = 3.21$, $P = 6$, $D = 4(3.21) + 6 = 18.8\tau$. **Marginally better** — add an inverter pair? No: adding *one* inverter changes the logic polarity. You would need two, giving $N = 5$: $\hat{f} = 106.7^{1/5} = 2.55$, $P = 7$, $D = 5(2.55) + 7 = 19.8$ — worse. **Stay at 3 stages.**

P13 — Congestion diagnosis and fix

Region overflow = +18 tracks on a $15\mu\text{m}$ gcell. Vertical layers: M2 (pitch 0.10), M4 (pitch 0.20), M6 (pitch 0.40). 20% blocked by power. Demand = 210. Compute supply and propose a quantified fix.

Solution

Raw: M2 = 150, M4 = 75, M6 = 37. Total = 262. After 20% blockage: $262 \times 0.8 = 209.6 \rightarrow 209$. Demand = 210. **Overflow** = 1, not 18 — so the stated 18 must come from a sub-region. Take the worst case as given: demand in that sub-region is $209 + 18 = 227$.

Fix options, quantified:

1. **Partial blockage at 40% density.** Demand roughly scales with local cell count: $227 \times$

$(0.40/0.60) = 151$. Massive over-fix; try 50%: $227 \times (0.50/0.60) = 189 < 209$. ✓ **Choose 50%**.

2. **Reduce power blockage from 20% to 14%** by widening straps and increasing their pitch (same total metal): supply becomes $262 \times 0.86 = 225$. Still 2 short — helps but insufficient alone.
3. **Combine:** partial blockage at 55% plus strap re-pitch gives demand 208, supply 225. **Comfortable.**

Recommendation: option 3. It costs no die area and no IR margin (same metal), and leaves headroom for detailed-route detours.

P14 — Clock tree power and the skew trade

32,000 flops, clock pin cap 1.8 fF each, tree overhead 1.8×, $V_{DD} = 0.9\text{ V}$, $f = 1\text{ GHz}$. (a) Clock power. (b) Tightening skew from 60 ps to 25 ps requires 22% more clock capacitance. Is it worth it if the 35 ps of recovered skew lets you drop clock uncertainty by 35 ps on a 1000 ps cycle?

Solution

(a) $C_{leaf} = 32,000 \times 1.8\text{ fF} = 57.6\text{ pF}$. $C_{clk} = 57.6(1 + 1.8) = 161.3\text{ pF}$.

$$P = 161.3 \times 10^{-12} \times 0.81 \times 10^9 = \mathbf{130.6\text{ mW}}$$

(b) New $C_{clk} = 161.3 \times 1.22 = 196.8\text{ pF} \Rightarrow P = 159.4\text{ mW}$. **Cost:** 28.8 mW.

Benefit: 35 ps on a 1000 ps cycle is 3.5% more timing margin on *every* path. What is that worth? If you would otherwise have to LVT-swap, say, 3,000 cells to recover 35 ps, and each swap costs 40 nW of leakage, that is $3000 \times 40\text{ nW} = 0.12\text{ mW}$.

Verdict: not worth it. 28.8 mW to avoid 0.12 mW is a 240× bad trade. **Keep the 60 ps skew and fix timing in the data path.**

The general principle: clock power is enormous and superlinear in skew tightness. Only tighten skew when the alternative is a *structural* change (pipeline stage, frequency reduction), not when it is a handful of cell swaps.

P15 — Crosstalk budget

A victim: $C_{gnd} = 15\text{ fF}$, $C_c = 11\text{ fF}$, $R_{drv} = 1.8\text{ k}\Omega$. The path has 9 such nets and 40 ps of setup slack. Does SI break it? Propose a fix with its track cost.

Solution

Nominal ($k = 1$): $C = 26\text{ fF}$, $t = 0.69(1800)(26 \times 10^{-15}) = 32.3\text{ ps}$. Worst ($k = 2$): $C = 37\text{ fF}$, $t = 46.0\text{ ps}$. Delta = +13.7 ps per net.

Nine nets: if all are worst-case, +123 ps. Slack 40 ps $\Rightarrow$ -83 ps: **broken**.

But statistically not all nine hit worst case simultaneously. With timing windows and a root-sum-square treatment: $13.7\sqrt{9} = 41\text{ ps}$ — still marginally over.

Fix: double spacing on these nets (an NDR). C_c falls roughly by half to 5.5 fF. New worst-case $C = 15 + 11 = 26\text{ fF}$, delta = $0.69(1800)(5.5 \times 10^{-15}) = 6.8\text{ ps}$ per net; RSS over nine = 20.5 ps. Slack becomes $40 - 20.5 = +19.5\text{ ps}$. ✓

Track cost: doubling spacing on 9 nets costs 9 extra tracks in their region. If the region has ~200 tracks, that is 4.5% — affordable unless the region is already congested. **If it is,**

shield only the two worst nets and upsize the drivers on the rest.

P16 — Useful skew across three stages

$T = 1000$ ps, overhead ($T_{cq} + T_{setup} + \text{unc}$) = 180 ps per stage. Logic: S1 = 920 ps, S2 = 760 ps, S3 = 650 ps. Find the skews on FF2 and FF3 that maximize the minimum slack.

Solution

With clock delays $\delta_1 = 0$ (fixed), $\delta_2, \delta_3, \delta_4 = 0$ (fixed at the end):

$$\begin{aligned} S_1 &= 1000 + (\delta_2 - \delta_1) - 180 - 920 = -100 + \delta_2 \\ S_2 &= 1000 + (\delta_3 - \delta_2) - 180 - 760 = 60 + \delta_3 - \delta_2 \\ S_3 &= 1000 + (\delta_4 - \delta_3) - 180 - 650 = 170 - \delta_3 \end{aligned}$$

Maximize $\min(S_1, S_2, S_3)$. Set all equal to S :

$$\begin{aligned} \delta_2 &= S + 100 \\ \delta_3 &= S + \delta_2 - 60 = 2S + 40 \\ \delta_3 &= 170 - S \end{aligned}$$

So $2S + 40 = 170 - S \Rightarrow 3S = 130 \Rightarrow S = 43.3$ ps.

$$\delta_2 = 143.3 \text{ ps}, \quad \delta_3 = 126.7 \text{ ps}, \quad S = +43.3 \text{ ps}$$

Check the total: $\sum(\text{logic}) = 920 + 760 + 650 = 2330$; $\sum(\text{overhead}) = 540$; total = 2870 over three cycles = 3000 ps. Available slack = 130 ps, spread over 3 stages = 43.3 ps each. ✓ The arithmetic confirms that useful skew simply *redistributes* the total path slack — it cannot create any.

This is the key insight: useful skew converts a *worst-case* constraint into an *average* constraint across a chain. It fixes local imbalance, never global insufficiency.

P17 — Multi- V_t leakage/timing front

40,000 cells, all SVT, leaking 85 nW each (3.4 mW total). WNS = +150 ps. HVT: −78% leakage, +40% delay. LVT: +520% leakage, −28% delay. Slack distribution: 25% of cells have > 300 ps, 35% have 100–300 ps, 40% have < 100 ps. Design the optimal mix.

Solution

Assume the average stage delay is ~ 70 ps, so HVT adds ~ 28 ps per cell and a typical path has 3–4 swappable cells.

Bucket 1 (> 300 ps, 10,000 cells): can absorb 4+ HVT swaps. Swap all to HVT. Leakage: $10,000 \times 85 \times 0.22 = 187 \mu\text{W}$ (from $850 \mu\text{W}$). **Saved** $663 \mu\text{W}$.

Bucket 2 (100–300 ps, 14,000 cells): can absorb 1–3 HVT swaps. Swap 60% conservatively. 8,400 cells to HVT: saved $8400 \times 85 \times 0.78 = 557 \mu\text{W}$. Remaining 5,600 stay SVT.

Bucket 3 (< 100 ps, 16,000 cells): leave SVT. Do *not* go LVT — WNS is already +150 ps, so there is no timing need, and LVT would cost $16,000 \times 85 \times 5.2 = 7.1 \text{ mW}$. **Never buy speed you do not need.**

	Cells	Leakage each	Total
HVT	18,400	18.7 nW	344 μ W
SVT	21,600	85.0 nW	1,836 μ W
LVT	0	—	0
Total	40,000		2,180 μW
Baseline			3,400 μ W
Saving			1,220 μW (36%)

Timing check. Bucket 1 paths lose ~ 112 ps of their > 300 ps slack — safe. Bucket 2 paths lose ~ 56 ps of their 100–300 ps — the 100 ps paths land at 44 ps, uncomfortable but positive. Add a 50 ps guard-band to be safe: swap only bucket-2 cells with > 150 ps. Recompute: saving drops to $\sim 32\%$. **Report the guard-banded number** — a 36% saving that fails signoff scores zero.

P18 — Power grid re-topology

Original: 100 straps, one-sided feed, $1000\mu\text{m}$ length, $R_{\text{strap}} = 8\Omega$, $I = 400$ mA. (a) Compute the drop. (b) Redesign with two-sided feed and find the strap count for a 12 mV budget.

Solution

(a) I per strap = $400/100 = 4$ mA. With uniform tapping, effective drop = $\frac{1}{2}IR = 0.5(0.004)(8) = 16$ mV. **Over the 12 mV budget by 33%.**

(b) Two-sided feed: each strap behaves as two halves of $500\mu\text{m}$, each carrying half the strap current. Half-strap $R = 4\Omega$, half-current = $I_{\text{strap}}/2$. Drop = $\frac{1}{2} \cdot \frac{I_{\text{strap}}}{2} \cdot 4 = I_{\text{strap}}$ (in volts if I in amps, Ω implicit) = $I_{\text{strap}} \times 1\Omega$.

With n straps: $I_{\text{strap}} = 0.400/n$, so drop = $0.400/n$ volts.

$$\frac{0.400}{n} \leq 0.012 \Rightarrow n \geq 33.3 \Rightarrow \mathbf{n = 34}$$

From 100 straps to 34 — a 66% reduction in power-grid track consumption, *and* the drop improves from 16 mV to 11.8 mV.

Freed tracks: if each strap+gap consumed 6 tracks, you have returned $66 \times 6 = 396$ tracks to signal routing. On a congested design this single topology change can be the difference between routable and not.

Lesson (repeated for emphasis): in power grid design, topology beats quantity.

P19 — Area recovery with a timing guard-band

88,000 cells, average $1.42\mu\text{m}^2$, total $124,960\mu\text{m}^2$. Slack histogram: 18% have < 50 ps, 27% have 50–150 ps, 55% have > 150 ps. Downsizing costs 38 ps and saves 30% area. Use a 40 ps guard-band. How much area is recoverable, and what is the new utilization if the core was $178,500\mu\text{m}^2$?

Solution

Eligible: cells whose slack exceeds $38 + 40 = 78$ ps.

- < 50 ps bucket: ineligible.
- 50–150 ps: assume uniform; the fraction above 78 ps is $(150 - 78)/(150 - 50) = 72\%$.
Eligible: $0.27 \times 0.72 = 19.4\%$.
- > 150 ps: all eligible, 55%.

Total eligible = 74.4% of 88,000 = 65,472 cells.

Assume 75% have a smaller library variant: 49,104 cells. Area of these = $49,104 \times 1.42 = 69,728 \mu\text{m}^2$. Recovery at 30%: **20,918 μm^2** — a 16.7% cell-area reduction.

New utilization (core unchanged): $(124,960 - 20,918)/178,500 = 104,042/178,500 = \mathbf{58.3\%}$, down from 70.0%.

The follow-through that earns points: do not stop here. Shrink the core to restore 70% utilization: new core = $104,042/0.70 = 148,631 \mu\text{m}^2$, a **16.7%** die shrink. *That* is the answer to an area-scored contest — recovering cell area without shrinking the die scores nothing.

Caveat: shrinking the core lengthens nothing but does raise congestion back toward the original level and shortens wires (helping power). Re-run and verify.

P20 — Decap and dynamic IR

A bank of 6,000 flops switches simultaneously, each drawing $32 \mu\text{A}$ peak for 60 ps. Local grid resistance 0.4Ω , $V_{DD} = 0.85 \text{ V}$, dynamic budget 5%. (a) Is it met? (b) How much decap fixes it?

Solution

(a) Peak current = $6000 \times 32 \mu\text{A} = 192 \text{ mA}$. Drop = $0.192 \times 0.4 = 76.8 \text{ mV}$. Budget = $0.05 \times 0.85 = 42.5 \text{ mV}$. **Fails by 81%.**

(b) Charge drawn = $I \cdot t = 0.192 \times 60 \times 10^{-12} = 11.52 \text{ pC}$.

Decap must supply the excess. Allowed drop from the decap alone: we need total drop $\leq 42.5 \text{ mV}$. The grid contributes what it can; treat the decap as supplying enough charge that the grid current falls to $I_{\text{grid}} = 0.0425/0.4 = 106 \text{ mA}$. Decap must supply $192 - 106 = 86 \text{ mA}$ for 60 ps:

$$Q_{\text{decap}} = 0.086 \times 60 \times 10^{-12} = 5.16 \text{ pC}$$

Allowing the decap itself to droop by 42.5 mV:

$$C_{\text{decap}} = \frac{5.16 \times 10^{-12}}{0.0425} = 121 \text{ pF}$$

At 400 fF per dedicated decap cell: **303 cells**. At $9 \mu\text{m}^2$ each: $2,727 \mu\text{m}^2$. **Very affordable.**

Better fix first: 6,000 flops switching *simultaneously* is the actual problem. Deliberately skewing the clock across four sub-banks by 50 ps each cuts peak dI/dt by $\sim 4\times$, bringing the drop to $\sim 19 \text{ mV}$ *with no decap at all*. Then add a modest decap for margin.

This is the general pattern: reduce the peak before you buffer it.

P21 — Scan reorder, full accounting

1,200 flops in 4 chains of 300. Pre-reorder average scan net = $210 \mu\text{m}$; post-reorder = $26 \mu\text{m}$. $c = 0.19 \text{ fF}/\mu\text{m}$, $V_{DD} = 0.9 \text{ V}$, shift frequency 50 MHz, activity 0.5. Compute wirelength, capacitance, test-mode power saving, and the congestion relief in tracks-microns.

Solution

Wirelength: $1200 \times (210 - 26) = 1200 \times 184 = 220,800\mu\text{m} = \mathbf{220.8\text{ mm}}$ removed.

Capacitance: $220,800 \times 0.19\text{ fF} = 41,952\text{ fF} = \mathbf{41.95\text{ pF}}$.

Test-mode power:

$$P = \alpha CV^2 f = 0.5 \times 41.95 \times 10^{-12} \times 0.81 \times 5 \times 10^7 = 8.5 \times 10^{-4} = \mathbf{0.85\text{ mW}}$$

Congestion: 220.8 mm of routing demand removed. If the core is $700\mu\text{m} \times 700\mu\text{m}$ with an effective total track length of, say, $8 \times (700/0.15) \times 700\mu\text{m} \approx 26,133\text{ mm}$ across 8 layers, the scan reorder frees $220.8/26,133 = 0.85\%$ of total routing resource.

But that global figure understates it, because pre-reorder scan nets are *long and random*, meaning they cross the whole die and load *every* gcell. Post-reorder they are local. The relief in the *worst* gcells is typically 5–15% — far more than the global average suggests.

Constraints to verify: (i) all four chains still have exactly 300 flops (balance); (ii) no chain crosses a clock domain without a lockup latch; (iii) chain head/tail still connect to the correct scan-in/scan-out pins; (iv) fault coverage unchanged (reordering does not change coverage, but re-verify with ATPG).

P22 — Choosing between four timing fixes

A path is 55 ps short. Options: **(A)** Upsize 2 cells: +60 ps, $+7\mu\text{m}^2$, +140 nW leakage, +0.02 mW dynamic. **(B)** LVT-swap 3 cells: +65 ps, $+0\mu\text{m}^2$, +380 nW leakage. **(C)** Insert 1 buffer: +58 ps, $+2.4\mu\text{m}^2$, +90 nW, +0.015 mW dynamic. **(D)** Useful skew +60 ps: $+0\mu\text{m}^2$, +0 nW, but drops hold slack on that path from +75 ps to +15 ps and on the *next* stage's setup by 60 ps (which has +200 ps). Scoring: area weight 1 per μm^2 , power weight 2000 per mW. Choose.

Solution

Convert everything to score units (lower is better).

(A) Area 7. Power = $140\text{ nW} + 0.02\text{ mW} = 0.00014 + 0.02 = 0.02014\text{ mW} \Rightarrow 40.3$. **Total** 47.3.

(B) Area 0. Power = $380\text{ nW} = 0.00038\text{ mW} \Rightarrow 0.76$. **Total** 0.76.

(C) Area 2.4. Power = $0.00009 + 0.015 = 0.01509\text{ mW} \Rightarrow 30.2$. **Total** 32.6.

(D) Area 0, power 0. **Total** 0 — but with risk.

Ranking: **D** < **B** < **C** < **A**.

Recommendation: try **D** first, fall back to **B**.

Reasoning: D is free but the hold margin drops to +15 ps, which is thin for signoff across corners. Check hold at the fast corner; if it survives with $> 10\text{ ps}$, take D. The next stage's $200\text{ ps} \rightarrow 140\text{ ps}$ is comfortable.

If D fails hold, B costs only 0.76 score units — 60× cheaper than upsizing. **Note how badly A scores:** dynamic power dominates the score function, and upsizing is a dynamic-power move. *Read the scoring function before choosing a technique* — with an area-weighted rulebook the ranking would invert.

21.3 Tier 3 — Multi-Constraint Optimization

P23 — Full floorplan design

Design a floorplan. Netlist: 210,000 cells at $1.30\text{ }\mu\text{m}^2$ average. Macros: 6 SRAMs of $380\mu\text{m} \times 240\mu\text{m}$, 1 PLL of $200\mu\text{m} \times 200\mu\text{m}$. Clock = 800 MHz. The design has three modules: CPU (60% of cells, talks to SRAM0–2), DSP (30%, talks to SRAM3–5), IO (10%). Target utilization 68%. Produce dimensions, macro placement, blockages, and justify.

Solution

Area budget. Cells: $210,000 \times 1.30 = 273,000\text{ }\mu\text{m}^2$. SRAMs: $6 \times 91,200 = 547,200\text{ }\mu\text{m}^2$. PLL: $40,000\text{ }\mu\text{m}^2$. Total = $860,200\text{ }\mu\text{m}^2$. Core = $860,200/0.68 = 1,265,000\text{ }\mu\text{m}^2$. Square core: $\sqrt{1,265,000} = 1,124.7\mu\text{m} \rightarrow 1,125\mu\text{m}$ **square**. With a $50\mu\text{m}$ margin, **die** = $1,225\mu\text{m}$ **square**. **Macro fit check.** Three SRAMs at $380\mu\text{m}$ wide side by side need $1,140\mu\text{m} > 1,125$. **Does not fit.** Options: rotate them ($240\mu\text{m}$ wide, three fit in $720\mu\text{m}$ ✓), or widen the core. Rotate.

Placement.

- SRAM0–2 rotated ($240\mu\text{m}$ w $\times$ $380\mu\text{m}$ h) along the **left edge**, stacked vertically: total height $3 \times 380 = 1,140\mu\text{m} > 1,125$. Marginal. Place two on the left edge ($760\mu\text{m}$) and one on the bottom-left corner.
- Practical arrangement: SRAM0, SRAM1 on the left edge; SRAM2 bottom-left; SRAM3, SRAM4 on the right edge; SRAM5 bottom-right.
- PLL in the **top-left corner**, away from the digital switching noise and close to the clock input pad. Surround with a guard ring and a routing blockage.
- **CPU logic** occupies the left/centre region adjacent to SRAM0–2.
- **DSP logic** occupies the right/centre region adjacent to SRAM3–5.
- **IO logic** along the top edge near the pins.

Blockages and halos.

- $12\mu\text{m}$ halo around every SRAM (pin access).
- $25\mu\text{m}$ halo plus a full routing blockage over the PLL (noise isolation).
- Hard placement blockage in any macro-to-macro channel narrower than $30\mu\text{m}$; check each gap after placement.
- Partial blockage (50%) in the centre if the trial route shows the CPU–DSP interface bus congesting there.

Pin orientation. All SRAM pins face the core (right-facing for left-edge macros, left-facing for right-edge macros). This alone typically saves 10–20% of macro-related wirelength.

Clock strategy. PLL in the corner means a long clock trunk to the far corner. Insertion delay at 800 MHz (1250 ps period) should stay under ~ 300 ps. Distance corner-to-corner is $\sim 1,590\mu\text{m}$; at $\sim 45\mu\text{m}$ buffer pitch that is ~ 35 buffers of delay if routed as a chain — far too many. **Solution: run the clock trunk on M7/M8 with a wide NDR**, giving much lower R and hence far fewer stages, and use a balanced H-tree from a central re-drive point.

Power plan. $1,125\mu\text{m}$ straps with two-sided feed (P18 lesson). Estimate current: at 800 MHz, 210k cells, expect ~ 600 mA. Using the P18 formula with $R_{strap} = (1125/2.4) \times 0.025 = 11.7\Omega$ for $2.4\mu\text{m}$ -wide M8: two-sided halves give effective drop = $0.5 \cdot (I_s/2) \cdot (R/2)$. For a 1.5% budget on 0.9 V (13.5 mV): $0.5(0.600/2n)(5.86) = 0.0135 \Rightarrow n \geq 65$. Pitch

$\approx 17\mu\text{m}$ — dense. Widen straps to $5\mu\text{m}$ and re-derive: $R = 5.6\Omega$, $n \geq 31$, pitch $\approx 36\mu\text{m}$. **Choose $5\mu\text{m}$ straps at $36\mu\text{m}$ pitch**, two-sided feed.

Verification loop. Coarse place $\rightarrow$ trial route $\rightarrow$ read HPWL, congestion, WNS. Iterate the macro positions 4–5 times before committing.

P24 — The frequency/power/area three-way

A block currently closes at 600 MHz, 180 mW, 420,000 μm^2 . The rulebook scores $\mathcal{S} = 0.4 \cdot \frac{f}{600} - 0.35 \cdot \frac{P}{180} - 0.25 \cdot \frac{A}{420,000}$ (higher is better; current $\mathcal{S} = -0.20$). Three options:

(A) Push to 700 MHz: $P \rightarrow 235\text{ mW}$, $A \rightarrow 468,000$.

(B) Drop to 520 MHz with V_{DD} scaling: $P \rightarrow 118\text{ mW}$, $A \rightarrow 390,000$.

(C) Stay at 600 MHz but optimize: $P \rightarrow 152\text{ mW}$, $A \rightarrow 384,000$.

Which wins? Is there a better point?

Solution

Baseline: $0.4(1) - 0.35(1) - 0.25(1) = -0.20$.

(A) $0.4(700/600) - 0.35(235/180) - 0.25(468/420) = 0.4(1.1667) - 0.35(1.3056) - 0.25(1.1143) = 0.4667 - 0.4569 - 0.2786 = -0.269$.

(B) $0.4(520/600) - 0.35(118/180) - 0.25(390/420) = 0.4(0.8667) - 0.35(0.6556) - 0.25(0.9286) = 0.3467 - 0.2294 - 0.2321 = -0.115$.

(C) $0.4(1) - 0.35(152/180) - 0.25(384/420) = 0.4 - 0.35(0.8444) - 0.25(0.9143) = 0.4 - 0.2956 - 0.2286 = -0.124$.

Ranking: B (-0.115) > C (-0.124) > baseline > A.

Analysis. The frequency weight (0.4) is barely larger than the power weight (0.35), and power scales worse than linearly with frequency. Option A loses because +16.7% frequency cost +30.6% power and +11.4% area.

Is there a better point? Combine B and C: scale voltage *and* do the optimization work. Estimate 560 MHz, $P = 128\text{ mW}$, $A = 380,000$:

$$0.4(0.9333) - 0.35(0.7111) - 0.25(0.9048) = 0.3733 - 0.2489 - 0.2262 = -0.102$$

Better than all three.

The generalizable lesson: when the score is a weighted sum, do not choose among the offered options — *differentiate*. Set $\partial\mathcal{S}/\partial f = 0$ using your measured $P(f)$ and $A(f)$ relationships and solve for the optimal f . In practice, fit $P \approx af^\beta$ from three or four data points, then optimize analytically. Teams that do this beat teams that pick from a menu.

P25 — Diagnosing a failing design

Post-route results: WNS = -210 ps (target 1000 ps period), TNS = $-38,400\text{ ps}$, 620 violating endpoints; hold WNS = -95 ps with 3,100 violations; DRC = 4,280; congestion overflow = 2.8%; utilization = 84%; clock insertion delay = 410 ps; global skew = 190 ps; total power = 310 mW against a 250 mW budget. You have 6 hours. What do you do?

Solution

Step 0 — Diagnose, do not patch. Every number is bad simultaneously. That is the signature of an *upstream* problem, not a collection of downstream ones.

The smoking gun: utilization = 84%. This single number explains:

- congestion 2.8% (way over the $< 0.5\%$ threshold);

- DRC 4,280 (detailed routing failing in congested regions);
- WNS –210 ps (detours from congestion lengthen critical nets);
- skew 190 ps and insertion 410 ps (no room to place clock buffers where the algorithm wants them);
- hold –95 ps with 3,100 violations (consequence of the bad skew);
- power 310 mW (detoured wires = extra capacitance; over-buffering).

Plan.

1. **(Hour 0–0.5)** Re-floorplan at **70%** utilization. Core area grows by $84/70 = 20\%$; linear dimensions by 9.5%. Accept the area hit — *a design that does not close scores zero*.
2. **(0.5–1.5)** Re-place. Check congestion immediately. Target overflow $< 0.3\%$. If a hot-spot persists, add a partial blockage there.
3. **(1.5–2.5)** Re-run CTS. With room to breathe, expect insertion delay ~ 250 ps and skew ~ 60 ps. The improved skew alone should recover ~ 130 ps of setup and eliminate most of the 3,100 hold violations.
4. **(2.5–3.5)** Route. Expect DRC to fall from thousands to tens.
5. **(3.5–4.5)** Post-route opt: DRVs, setup, then hold. Expect WNS near zero.
6. **(4.5–5.0)** Power recovery: HVT swaps on slack paths, clock gating check, buffer removal. Target the 250 mW budget.
7. **(5.0–5.5)** Full verification: DRC, LVS, antenna, STA all corners.
8. **(5.5–6.0) Freeze. Submit.** Do not start anything new.

Expected outcome: area +20%, everything else clean. If the rulebook weights area heavily you can then try a 76% run *from a copy* while the clean 70% result sits safely submitted.

What NOT to do: spend six hours fixing 4,280 DRCs individually, or inserting 3,100 hold buffers into a design with no whitespace. Both are guaranteed failure modes and both are what panicking teams do.

P26 — Buffer vs. clone vs. split decision

A driver has fanout 48. Sinks form two clusters: 30 sinks within $40\mu\text{m}$ of the driver, 18 sinks about $520\mu\text{m}$ away. Driver: $R_{drv} = 1.1\text{ k}\Omega$. Sink input cap 2.5 fF each. Wire $c = 0.2\text{ fF}/\mu\text{m}$. Estimate the current delay, then evaluate (a) one big buffer to the far cluster, (b) cloning the driver, (c) a two-level buffer tree.

Solution

Current (unbuffered). Near cluster wire: ~ 30 sinks over a $40\mu\text{m}$ radius, estimate $600\mu\text{m}$ of wire = 120 fF . Sink caps = $30 \times 2.5 = 75\text{ fF}$. Far cluster: trunk $520\mu\text{m}$ (104 fF) plus local distribution, say $400\mu\text{m}$ (80 fF); sink caps = $18 \times 2.5 = 45\text{ fF}$. Total $C \approx 120 + 75 + 104 + 80 + 45 = 424\text{ fF}$.

$$t \approx 0.69 \times 1100 \times 424 \times 10^{-15} = \mathbf{322\text{ ps}}$$

Plus the trunk's own RC : at $0.2\Omega/\mu\text{m}$, $R_{trunk} = 104\Omega$, adding $0.69(104)(125\text{ fF}) \approx 9\text{ ps}$. Say $\approx \mathbf{331\text{ ps}}$. Very slow.

(a) One buffer to the far cluster. Driver now sees: near cluster (195 fF) + trunk (104 fF) + buffer input (5 fF) = 304 fF . $t_1 = 0.69(1100)(304 \times 10^{-15}) = 231\text{ ps}$. Buffer ($R_b = 800\Omega$,

$t_b = 25$ ps) drives $80 + 45 = 125$ fF: $t_2 = 25 + 0.69(800)(125 \times 10^{-15}) = 25 + 69 = 94$ ps. **Far path** = $231 + 94 = 325$ ps. **Near path** = 231 ps. Barely better on the far path; good on the near.

(b) **Cloning.** Duplicate the driver; clone placed near the far cluster. Original drives near cluster only: $C = 195$ fF, $t = 0.69(1100)(195 \times 10^{-15}) = 148$ ps. Clone drives far cluster: it must *itself* be reached, so the clone's *inputs* travel $520\mu\text{m}$ — but inputs are typically fewer and less loaded than 18 outputs. Clone then drives 125 fF locally: $t = 0.69(1100)(125 \times 10^{-15}) = 95$ ps, plus whatever the input routing costs. **Far path** $\approx$ (**input routing**) + 95 ps. If the clone's input net is $520\mu\text{m}$ driven by a decent upstream cell, add ~ 80 ps: ≈ 175 ps.

(c) **Two-level buffer tree.** Driver $\rightarrow$ 2 buffers $\rightarrow$ sinks. Driver sees 2×5 fF + short wires ≈ 30 fF: $t_1 = 23$ ps. Near buffer drives 195 fF: $t = 25 + 0.69(800)(195 \times 10^{-15}) = 133$ ps. Far buffer: trunk to it 104 fF plus 125 fF local = 229 fF: $t = 25 + 0.69(800)(229 \times 10^{-15}) = 151$ ps. **Near** = 156 ps, **far** = 174 ps.

Option	Near path	Far path	Cost
Unbuffered	331	331	0
(a) One buffer	231	325	1 buffer
(b) Clone	148	~ 175	1 clone + input reroute
(c) Two-level tree	156	174	2 buffers

Winner: (c), the two-level tree — nearly as fast as cloning, balanced across both clusters, no netlist change, no input rerouting, and only 2 cells.

When cloning would win instead: if the driver's own *input* arrives from a source physically near the far cluster, cloning eliminates a long round-trip that buffering cannot. **Always check where the driver's input comes from before choosing.**

P27 — Budget allocation under a hard power cap

Power budget 200 mW, currently 268 mW. Breakdown: clock 76 mW, combinational switching 88 mW, sequential internal 44 mW, leakage 44 mW, macros 16 mW. Constraint: WNS must stay ≥ 0 (currently +40 ps). Available moves with their timing costs are given below. Choose a plan.

Move	Saving (mW)	Timing cost (ps)
Clock gating (enable-based)	30	0
Tighten max transition 120 $\rightarrow$ 90 ps	-6 (costs)	+12 (gains)
HVT swap, slack > 120 ps	22	-35 on those paths
HVT swap, slack > 60 ps	34	-35 on those paths
Downsize, slack > 100 ps	14	-30
Reduce V_{DD} 0.90 $\rightarrow$ 0.86 V	23	-45 globally
Clock NDR removal (narrower clock)	9	-20 globally (worse skew)
Wirelength re-placement	7	+5

Solution

Need to save 68 mW while keeping WNS ≥ 0 from +40 ps.

Free / gaining moves first.

- Clock gating: −30 mW, 0 ps. **Take it.** Running total: 238 mW, WNS +40.
- Wirelength re-placement: −7 mW, *gains* 5 ps. **Take it.** 231 mW, WNS +45.

Still need 31 mW.

Now spend timing. Available budget: 45 ps.

- HVT swap slack > 120 ps: −22 mW, and critically **costs nothing on the critical path** because it only touches cells with > 120 ps of slack. **Take it.** 209 mW, WNS still +45.

Still need 9 mW, WNS budget still 45 ps.

- Downsize slack > 100 ps: −14 mW, again does not touch the critical path. **Take it.** **195 mW**, WNS +45. ✓

Final plan: clock gating + re-placement + HVT (>120 ps) + downsizing (>100 ps). **Result:** 195 mW (**under the 200 mW cap by 2.5%**) **with WNS = +45 ps (better than the starting +40 ps).**

Moves deliberately rejected and why:

- **V_{DD} reduction** (−23 mW but −45 ps *globally*) would take WNS to exactly 0 — no margin for signoff variation. Rejected as too risky, and unnecessary since the cap is already met.
- **HVT slack** > 60 ps instead of > 120 ps: saves 12 mW more but the 60–95 ps slack cells would go negative after a 35 ps hit. Rejected.
- **Clock NDR removal:** −9 mW but degrades skew globally and hurts crosstalk immunity on the one net you least want to compromise. Rejected on principle.
- **Tightening max transition** *costs* power. Only worth it if you need the 12 ps. You do not.

The method: sort moves by $\Delta P/\Delta t$ (power saved per picosecond spent), take all moves with infinite ratio (zero timing cost) first, then spend the timing budget greedily. This is the same knapsack structure as the V_t problem in Chapter 14.

P28 — Aspect ratio and clock tree interaction

A block of $900,000 \mu\text{m}^2$ core area. Compare $AR = 1.0$ (949×949), $AR = 2.25$ (632×1423), and $AR = 4.0$ (474×1897). For each estimate: maximum Manhattan distance, expected clock insertion delay (assume the clock root is central and 70 ps per 100 μm of unshielded routing plus buffers), and comment on wirelength.

Solution

	$AR = 1.0$	$AR = 2.25$	$AR = 4.0$
Dimensions (μm)	949×949	632×1423	474×1897
Perimeter (μm)	3,796	4,110	4,742
Max Manhattan (corner to corner)	1,898	2,055	2,371
Centre to farthest corner	949	1,028	1,186
Est. clock insertion (ps)	664	720	830

Computation of centre-to-corner: half-width + half-height, i.e. $(949 + 949)/2 = 949$; $(632 + 1423)/2 = 1027.5$; $(474 + 1897)/2 = 1185.5$.

Clock insertion at 70 ps/100 μ m: 949 $\rightarrow$ 664 ps; 1027.5 $\rightarrow$ 719 ps; 1185.5 $\rightarrow$ 830 ps.

Analysis.

- Going from $AR = 1$ to $AR = 4$ costs **166** ps of insertion delay (+25%) for the *same core area*. On a 1000 ps cycle, deeper trees also mean worse skew and more clock buffers — more power.
- Perimeter grows 25%, which is *good* if the design is IO-limited (more pin room) and *bad* for wirelength.
- Average intra-core net length scales roughly with $\sqrt{A} \cdot g(AR)$ where g grows with elongation. Expect 10–18% more wirelength at $AR = 4$ than at $AR = 1$ — and correspondingly more dynamic power.

Recommendation: $AR = 1.0$ unless externally forced. If a top-level slot forces $AR = 2.25$, mitigate by:

1. Using a **spine** clock topology along the long axis rather than an H-tree.
2. Considering **multi-source CTS** with two drive points at the one-quarter and three-quarter positions of the long axis — this roughly halves the effective distance and recovers most of the insertion penalty.
3. Placing macros at the two *ends* so the standard cell region is closer to square.

These mitigations are the answer a judge wants — not just “AR 1 is best,” which is obvious, but what you do when you cannot have it.

P29 — Deriving the optimal utilization empirically

You run five utilizations and record:

Util	Core area (k μ m ²)	HPWL (mm)	WNS (ps)	Overflow	Power (mW)
60%	480	1,820	+85	0.00%	142
65%	443	1,760	+70	0.02%	138
70%	411	1,735	+45	0.11%	136
75%	384	1,790	−15	0.68%	141
80%	360	1,960	−140	2.40%	152

Identify the optimum for (a) an area-weighted rulebook, (b) a power-weighted rulebook, (c) a balanced one. Explain the shape of the HPWL curve.

Solution

The HPWL curve is U-shaped, minimum at 70% (1,735 mm).

- *Below 70%:* cells are spread further apart, so nets are geometrically longer. HPWL rises.
- *Above 70%:* congestion forces detours; the router cannot take direct paths. HPWL rises again — and faster, because detours are multiplicative.

Power tracks HPWL closely (minimum also at 70%, 136 mW) because switching power is dominated by wire capacitance. **This confirms the physical story.**

(a) Area-weighted: you want the smallest core that still *closes*. 80% (WNS −140, overflow

2.4%) and 75% (WNS -15, overflow 0.68%) both fail. **70% at 411k μm^2 is the answer** — and note that 75% is *nearly* recoverable: -15 ps is one ECO away. Worth an attempt from a copy, but 70% is the safe submission.

(b) Power-weighted: 70% at 136 mW. **Same answer.**

(c) Balanced: 70% again.

70% wins on every axis — which is the useful lesson. The classic assumption that “area and power trade off against each other” is false near the congestion cliff, because congestion degrades *everything* simultaneously. Above the cliff there is no trade-off, only loss.

What to do next: refine between 70% and 75%. Run 71, 72, 73%. The true optimum is likely 72–73%, buying another 2–3% of area for free. **Always bisect after a coarse sweep.**

P30 — Hold-fix strategy under an area cap

Post-CTS: 2,240 hold violations, average -38 ps, worst -115 ps. Area headroom: only 4,000 μm^2 . Delay cell: 32 ps, 2.5 μm^2 . HVT swap available on 62% of launch paths, giving +30 ps each at zero area. Downsizing available on 40%, giving +22 ps at -0.8 μm^2 each. Devise a plan that fits.

Solution

Naive plan: all delay cells. Cells per violation: $\lceil 38/32 \rceil = 2$ average, but the worst needs $\lceil 115/32 \rceil = 4$. Estimate 2.3 average $\Rightarrow$ 5,152 cells $\Rightarrow$ 12,880 μm^2 . $3.2\times$ **over the cap. Fails.**

Layered plan.

Layer 1 — HVT swaps (zero area, negative leakage). 62% of 2,240 = 1,389 paths get +30 ps. Of those, the ones needing ≤ 30 ps are fully fixed. Assuming a roughly normal distribution centred at 38 ps, about 40% of violations are ≤ 30 ps. So $1389 \times 0.40 = 556$ fully fixed. The remaining 833 are reduced by 30 ps (average residual ~ 16 ps).

Layer 2 — Downsizing (recovers area). 40% of the remaining 1,684 ($= 2240 - 556$) = 674 paths get +22 ps. Most of the 16 ps residuals from layer 1 are now fixed. Say 500 fully fixed. Area recovered: $674 \times 0.8 = 539 \mu\text{m}^2$.

Layer 3 — Delay cells for the rest. Remaining: $2240 - 556 - 500 = 1,184$ violations, with reduced magnitudes (most have already absorbed 30 or 22 ps). Average residual ~ 20 ps $\Rightarrow$ 1 cell each. Cells = 1,184. Area = $1184 \times 2.5 = 2,960 \mu\text{m}^2$.

Net area: $2,960 - 539 = 2,421 \mu\text{m}^2$. **Fits within the 4,000 μm^2 cap with 39% to spare.** ✓

Bonus effects:

- HVT swaps *reduce* leakage on 1,389 cells — a power win alongside the hold fix.
- Downsizing reduces both area and power.
- Only the residual 1,184 cost anything.

Verification step you must not skip: every hold fix on a launch path also *slows* that path for setup. Confirm that the 1,389 HVT-swapped cells all had setup slack > 30 ps before swapping. If any did not, exclude them and route those violations to layer 3 instead.

This layered approach — free moves, then area-recovering moves, then area-spending moves — is the general template for working under any hard resource cap.

P31 — Reconstructing a design from a report

You are given only this summary. Reconstruct what happened and recommend fixes.

Setup WNS	−8 ps
Setup TNS	−11,940 ps
Violating endpoints	1,640
Hold WNS	+45 ps
Max transition violations	0
Clock insertion delay	180 ps
Clock global skew	22 ps
Clock uncertainty (SDC)	120 ps
Congestion overflow	0.04%
Utilization	68%
DRC	0

Solution

Reconstruct. Average violation = $11,940/1640 = 7.3$ ps; worst is 8 ps. **The distribution is almost flat** — 1,640 endpoints all failing by 7–8 picoseconds.

That is not a design problem. Real design problems produce a long tail: a few paths failing badly and many failing slightly. A flat distribution at a tiny magnitude means **a uniform constant is subtracting from every path**.

The culprit: clock uncertainty = 120 ps while measured global skew is 22 ps. The SDC still carries the pre-CTS estimate. Post-CTS, uncertainty should be reduced to jitter + margin only — typically 30–50 ps, since skew is now *physically modelled* in the propagated clock and no longer needs to be budgeted.

Recommendation 1 (do this first, costs nothing):

```
1 # post-CTS: skew is now real, stop double-counting it
2 set_clock_uncertainty -setup 0.045 [get_clocks clk]
3 set_clock_uncertainty -hold 0.020 [get_clocks clk]
4 update_timing -full
```

This returns 75 ps to every path. WNS goes from −8 to +**67** ps. TNS goes to **0**. **All 1,640 violations vanish.**

Recommendation 2: verify CRPR/CPPR is enabled. With 180 ps insertion delay there is a substantial common path; CRPR typically returns another 10–30 ps.

Recommendation 3: check the derate model. If flat OCV is in use, switching to AOCV/POCV is worth more free margin.

Everything else in the report is healthy: zero DRC, zero DRVs, 0.04% overflow, 68% utilization, +45 ps hold, 22 ps skew, 180 ps insertion. **This is a well-built design with a stale constraint.**

The meta-lesson, worth internalizing: before you change a single gate, verify that the violation is *real*. In competitions and in industry, a large fraction of reported violations are analysis artefacts — stale uncertainty, missing CRPR, over-pessimistic derate, unconstrained or wrongly constrained paths. **Free margin is the cheapest margin there is.**

P32 — The complete contest scenario

You receive at 09:00: a gate-level netlist (145,000 cells), a 130 nm library with HVT/SVT/LVT and drive strengths X1–X16, four SRAM macros, an SDC at 400 MHz, and a LEF/tech stack of 6 metals. Deadline 21:00 (12 hours). Scoring: (i) must be DRC/LVS clean and timing-closed; (ii) 40% die area, 35% total power, 25% total wirelength. Write your plan hour by hour.

Solution

09:00–09:30 — Recon, not work. Read the SDC completely. Run `check_design` and `check_timing`. Count cells, flops, macros. Compute the theoretical minimum core area: $145,000 \times (\text{average area}) + \text{macro area}$, divided by 0.70. *Write the number down.* It is your area target and your sanity check.

09:30–10:30 — Baseline floorplan and first full run. Utilization 68%, $AR = 1.0$, macros at the periphery with pins inward, halos, taps, endcaps. Simple two-sided power grid. Launch a complete flow to GDS with *default everything*. Do not optimize. **The goal is a legal, submittable artefact by 10:30.** Tag it BASELINE. Back it up.

10:30–11:00 — Read the baseline. Record all twelve metrics from Chapter 20. Identify the binding constraint: is it congestion, timing, or neither? Everything after this is directed by that answer.

11:00–13:00 — Sweep, in parallel if you have cores. Launch utilization $\in \{62, 66, 70, 74, 78\}$ and, independently, synthesis strategy $\in \{\text{area, balanced, delay}\}$. That is 5–8 runs. **While they run, do paper work:** compute the expected optimal buffer pitch, the power grid requirement, the clock tree depth. Do not sit and watch logs.

13:00–13:30 — Build the Pareto table. Plot area vs. power vs. wirelength for every run. Apply the scoring weights explicitly: $\mathcal{S} = 0.40\hat{A} + 0.35\hat{P} + 0.25\hat{W}$ with each term normalized to the baseline. **Pick the winner numerically, not by feel.**

13:30–15:30 — Refine the winner. Bisect the utilization around the winner ($\pm 2\%$). Tune the power grid pitch (more pitch $\Rightarrow$ fewer straps $\Rightarrow$ less congestion $\Rightarrow$ shorter wires, as long as IR holds). Improve macro placement based on the fly-line and congestion maps.

15:30–17:00 — Targeted optimization.

- Scan reorder (wirelength: 25% of the score).
- HFNS on reset and any other high-fanout net (wirelength + congestion).
- Clock gating audit — is the tool inserting ICGs? If not, why not? (power: 35% of the score).
- Multi- V_t pass with a 50 ps guard-band (power).
- Area recovery / downsizing (area: 40%).

Re-time after each. Keep the metrics table updated.

17:00–18:30 — Close everything. Post-route optimization: DRVs, setup, hold, SI. Fillers last. Full DRC/LVS/ antenna. All corners STA.

18:30–19:00 — HARD FREEZE. Whatever state the design is in, run the complete verification suite and **submit it**. Tag it SUBMITTED_V1.

19:00–20:30 — Aggressive attempt, from a copy. Now, and only now, try the risky things: +4% utilization, more aggressive V_t mix, tighter transition limits. If it closes clean, submit it as V2. If it does not, you lose nothing.

20:30–21:00 — Documentation. Write up the metrics table, the Pareto plot, and one paragraph per optimization explaining *what you did, what it cost, and why you chose it*. Judges score reasoning. A team that says “we chose 70% utilization because the HPWL

curve minimizes there and 75% pushed overflow past 0.5%” outscores a team with marginally better numbers and no explanation.

Hour	Principle
First 90 min	Get something legal. Never be without a submittable design.
Middle 6 hrs	Sweep broadly, then refine narrowly. Let the machine work while you think.
Last 3 hrs	Close, verify, freeze, submit. Then experiment from a copy.
Throughout	One metrics table. One change per run. Write down why.

The three ways teams lose this contest:

1. No baseline until hour 8, then something breaks and there is nothing to submit.
2. Optimizing the wrong axis — pushing frequency when the score is 40% area.
3. Failing DRC at 20:55.

All three are process failures, not knowledge failures. The plan above exists to prevent them.

CHAPTER 22

Rapid-Fire Question Bank

Answers follow each block. Cover them and work down the list. If you cannot answer one in under 20 seconds, mark it and return to the relevant chapter.

22.1 Block A — Fundamentals

1. Why is cell height fixed but width variable?
2. What is in a tech LEF that is not in a cell LEF?
3. Why does a low- V_t cell have almost the same area as a high- V_t one?
4. What does a Liberty NLDM table index on?
5. Why do rows alternate orientation?
6. What breaks if you forget tap cells?
7. What breaks if you forget endcaps?
8. Why insert fillers last?
9. Why does a tie-hi cell exist instead of wiring the gate to VDD?
10. What is a spare cell for?

Answers — Block A

1. So cells abut in rows and share power rails; width scales with drive strength. **2.** Layers, pitches, spacing/width rules, via definitions, units — process data, not cell data. **3.** V_t is set by channel implant, not geometry; the transistors are the same size. **4.** Input transition (slew) and output load capacitance. **5.** So adjacent rows share a VDD rail and a VSS rail, halving rail count. **6.** Latch-up — the well and substrate float. Also base-layer DRC. **7.** Well/implant edge DRC rules at row ends and macro boundaries. **8.** Because ECOs need free space; fillers would have to be deleted and re-inserted. **9.** Gate oxide reliability and ESD — a directly connected gate is vulnerable during processing. **10.** Post-mask (metal-only) functional ECOs.

22.2 Block B — Timing

11. Write the setup slack equation.
12. Write the hold slack equation.
13. Why can't a slower clock fix hold?
14. Which corner for setup? Which for hold? Why?

15. What is temperature inversion?
16. What is CRPR and why does it give free margin?
17. Difference between OCV, AOCV, and POCV?
18. Why must a multicycle setup exception be paired with a hold exception?
19. What is the danger of a false path?
20. Why does fixing a max-transition violation improve delay accuracy?
21. What is the difference between global and local skew, and which matters?
22. Give the formula for maximum useful skew benefit across a two-stage chain.
23. What does TNS tell you that WNS does not?

Answers — Block B

11. $S = T + T_{skew} - T_{setup} - T_{unc} - T_{cq} - T_{comb}$. **12.** $S = T_{cq,min} + T_{comb,min} - T_{skew} - T_{hold} - T_{unc,hold}$. **13.** The hold equation contains no T — it compares two edges of the *same* clock cycle. **14.** Setup at the slow corner (max delay: SS, low V , high T); hold at the fast corner (min delay: FF, high V , low T). **15.** At low V_{DD} and advanced nodes, cells can be slower cold than hot, because the V_{th} increase outweighs the mobility gain. **16.** Clock Reconvergence Pessimism Removal: it credits back the opposing derates applied to the *shared* portion of launch and capture clock paths, which a single physical buffer cannot exhibit simultaneously. **17.** Flat derate; depth- and distance-dependent derate; per-arc statistical σ combined in quadrature. Decreasing pessimism. **18.** Because the setup exception moves the capture edge, and without a matching hold exception the hold check lands on the wrong edge, generating impossible violations. **19.** A wrongly declared false path is never checked and becomes silent silicon failure. **20.** Because library delay tables are only characterized over a range of input slews; outside it the tool extrapolates, and downstream delays become unreliable. **21.** Global is max–min over all flops; local is between launch/capture pairs. Only local affects timing. **22.** Set the two slacks equal: $\delta = (S_2 - S_1)/2$, giving both $(S_1 + S_2)/2$. **23.** How *much* of the design is failing — WNS is one path, TNS is the aggregate. Their ratio to NVP tells you whether the problem is systemic or local.

22.3 Block C — Floorplan, Power, Placement

24. Why should macros go to the periphery?
25. What is a partial placement blockage for?
26. Why do you add a halo around a macro?
27. Give two reasons a narrow macro channel is bad.
28. Why is IR drop a timing problem?
29. Write Black's equation and state what it predicts.
30. Why does two-sided power feed help so much?
31. What is HPWL and when is it exact?
32. Why does pure wirelength minimization hurt timing?
33. Name five placement-stage optimization transforms.
34. Why is pin swapping free?

35. When do you clone instead of buffer?
36. What does scan reordering buy, and what must it preserve?
37. What is the difference between global placement, legalization, and detailed placement?

Answers — Block C

24. To leave a contiguous standard-cell region; a central macro partitions the core and wrecks routing and CTS. **25.** Locally capping density over a congestion hot-spot without growing the die. **26.** To keep standard cells away from the macro's pin edge so the router can access those pins. **27.** Routing funnels through it (congestion) and cells placed inside it are hard to reach and hard to legalize. **28.** Cell delay rises roughly 1–2% per 1% of V_{DD} lost, so IR drop silently erodes margin everywhere. **29.** $MTTF = (A/J^n)e^{E_a/kT}$; median time to electromigration failure as a function of current density and temperature. **30.** Each strap becomes two half-length halves each carrying half the current, cutting the effective drop by about $4\times$. **31.** Half-perimeter wirelength; exact for 2- and 3-pin nets. **32.** It will lengthen one critical net to shorten several non-critical ones. **33.** Sizing, V_t swap, buffering, cloning, pin swap, restructuring, remapping, relocation, scan reorder (any five). **34.** It exchanges two logically equivalent inputs; no cell is added, removed, resized, or moved. **35.** When the sinks form two spatially separate clusters — cloning adds no stage delay, whereas buffering adds one to every sink. **36.** Large wirelength and congestion reduction; must preserve chain count, chain balance, clock-domain integrity, lockup latches, and fault coverage. **37.** Global finds approximate positions minimizing a smooth objective with overlap allowed; legalization snaps to rows/sites and removes overlap with minimum displacement; detailed does local swaps and reordering.

22.4 Block D — CTS, Routing, Power

38. Why does CTS create hold violations?
39. Name four clock tree topologies and one property of each.
40. Why use a non-default rule on clock nets?
41. Why is a clock mesh so power-hungry?
42. What does an ICG cell contain and why the latch?
43. Give the clock-power formula and explain each factor.
44. What is the antenna effect and give three fixes.
45. Explain the Miller factor and give its worst-case value.
46. Name the two distinct failures caused by crosstalk.
47. Why are multi-cut vias worth the effort?
48. Why does SI analysis iterate?
49. List the three components of power and one fix for each.
50. Why is leakage exponentially temperature dependent?
51. What is the stacking effect?
52. Header vs. footer switch: which is bigger and why?
53. Why does clock gating save more than the clock net capacitance suggests?

Answers — Block D

38. Because it replaces an ideal zero-skew clock with a real one; any positive skew directly subtracts from hold slack. **39.** Buffer tree (low power); H-tree (symmetric, excellent skew); mesh (very low skew, very high power); spine (good for elongated blocks). **40.** Wider wire lowers R and improves EM; wider spacing lowers coupling and crosstalk-induced jitter. **41.** The entire grid toggles every cycle, and it has far more capacitance than a tree. **42.** A latch plus an AND (or equivalent); the latch holds the enable stable through the active clock phase so the gated clock cannot glitch. **43.** $P = \alpha CV_D^2 f$: activity factor, switched capacitance, supply squared, frequency. For a clock α is effectively 1 in the full-cycle convention because it toggles twice per period. **44.** Charge accumulation on a long partially-built metal segment connected to a gate, which can rupture gate oxide. Fixes: layer jumping, diode insertion, buffer insertion, driver upsizing. **45.** A multiplier on coupling capacitance depending on aggressor behaviour: 0 (same direction), 1 (quiet), 2 (opposite direction); up to 3 in worst-case analytical bounds. **46.** Noise glitch on a static victim (functional failure), and delta delay on a switching victim (timing failure). **47.** Lower via resistance, much better yield (single-cut opens are a leading defect), and better EM. **48.** Because delta delay changes arrival times, which changes the timing windows, which changes which aggressor/victim pairs overlap. **49.** Switching (clock gating), short-circuit (slew control), leakage (multi- V_t /power gating). **50.** Subthreshold current $\propto e^{-qV_{th}/nkT}$, and V_{th} itself falls with temperature. **51.** Two series off-transistors leak far less than one, because the intermediate node floats up and reverse-biases the top device. **52.** The header (PMOS) is bigger, because PMOS has lower mobility and needs more width for the same on-resistance. **53.** Because the gated flops also stop their internal clock-driven switching, and their downstream logic stops toggling too.

22.5 Block E — Algorithms and Strategy

54. What does FM improve over KL?
55. What is Hanan's theorem?
56. Why does quadratic placement need a spreading step?
57. What does PathFinder negotiate?
58. What does DME compute in its bottom-up pass?
59. What does van Ginneken's algorithm optimize, and what is its guarantee?
60. State the logical effort delay equation and the optimal stage effort.
61. Why is "sweep the options" the correct strategy rather than laziness?
62. In a contest, what do you do in the first ninety minutes?
63. What is the single most common way strong teams lose?

Answers — Block E

54. Single-cell moves (handles unequal cell sizes and area balance), bucket-list data structure giving $O(1)$ move selection, linear time per pass, and native hypergraph handling. **55.** An optimal rectilinear Steiner minimal tree exists on the grid formed by horizontal and vertical lines through the pins — reducing an infinite search space to $O(n^2)$ candidates. **56.** Because the unconstrained quadratic minimum piles all movable cells at the centroid; density must be enforced separately. **57.** Routing resources: costs rise with historical and present congestion until nets with alternatives yield and nets without keep them. **58.** The merging segment —

the locus of points from which both subtrees have equal delay. **59.** It maximizes required arrival time at the source by choosing buffer positions; optimal for a *fixed* topology and buffer library under the Elmore model. **60.** $d = gh + p$; optimal stage effort $\hat{f} = F^{1/N} \approx 3.6$ (often rounded to 4). **61.** Because essentially every PD problem is NP-hard and every tool is a heuristic, so results genuinely depend on options, seeds, and ordering — sweeping is the only way to find the good region. **62.** Produce a legal, submittable, unoptimized baseline end to end. **63.** Over-optimizing until the deadline and then failing verification with no time to fix it.

CHAPTER 23

Checklists

23.1 Before you start

- ☐ Read the entire problem statement / SDC. Twice.
- ☐ Identify the scoring function and its weights. Write them down.
- ☐ Identify the hard constraints (DRC, LVS, timing) — these are pass/fail, not scored.
- ☐ Compute the theoretical minimum core area from the cell area.
- ☐ Verify every library file loads and every cell has both LEF and LIB.
- ☐ Run `check.design` and `check.timing`; read all output.
- ☐ Confirm every clock is defined and every endpoint constrained.
- ☐ Set up a directory structure and a metrics spreadsheet.
- ☐ Establish version control or at least dated backups.

23.2 Floorplan

- ☐ Utilization chosen with headroom for CTS and hold buffers.
- ☐ Aspect ratio ≈ 1 unless externally forced.
- ☐ Macros at the periphery, pins facing the core.
- ☐ Macros that communicate are adjacent (checked with fly-lines).
- ☐ No channel narrower than ~ 30 tracks; blockages placed in any that remain.
- ☐ Halos on every macro.
- ☐ Tap cells at the required pitch.
- ☐ Endcaps at all row ends and macro boundaries.
- ☐ IO/pin placement matches the logic that drives it.
- ☐ Trial place + trial route done; HPWL, congestion, WNS recorded.
- ☐ At least three floorplan variants compared numerically.

23.3 Power plan

- ☐ Rings on the thick top layers.
- ☐ Straps form a genuine mesh (two orthogonal layers).

- ☐ Two-sided (or four-sided) feed wherever possible.
- ☐ Follow-pin rails routed to every row.
- ☐ Macro power pins connected.
- ☐ Static IR $\leq 1\text{--}2\%$; dynamic $\leq 4\text{--}5\%$ — and the STA voltage matches.
- ☐ EM checked on straps, rails, and clock nets.
- ☐ Via stacks adequate (this is the usual bottleneck).
- ☐ Strap pitch swept against congestion.

23.4 Placement

- ☐ Zero overlaps; all cells legal and on-site.
- ☐ Density map free of hot-spots.
- ☐ Trial route overflow $< 0.3\%$.
- ☐ All DRVs fixed (max transition, cap, fanout).
- ☐ High-fanout nets synthesized into trees.
- ☐ Scan chains reordered.
- ☐ Pre-CTS timing recorded and plausible.

23.5 CTS

- ☐ Clock transition target met at every leaf.
- ☐ *Local* skew acceptable (do not chase global skew).
- ☐ Insertion delay reasonable ($< 25\text{--}30\%$ of the period).
- ☐ Non-default rules applied to clock nets.
- ☐ Clock gating present and effective; ICGs balanced into the tree.
- ☐ Clock uncertainty updated from the pre-CTS estimate to jitter + margin.
- ☐ Post-CTS hold fixed — HVT and downsizing before delay cells.

23.6 Routing

- ☐ Timing-driven and SI-driven routing enabled.
- ☐ Multi-cut via conversion $> 95\%$.
- ☐ Antenna violations zero.
- ☐ Crosstalk delta delay within budget.
- ☐ Layer utilization balanced (check the per-layer table).
- ☐ Post-route extraction done before believing any timing number.

23.7 Sign-off

- ☐ Setup and hold met on *all* corners and modes.
- ☐ DRVs zero.
- ☐ DRC zero. LVS clean. ERC clean.
- ☐ Antenna clean.
- ☐ IR and EM within budget.
- ☐ LEC passes against RTL.
- ☐ Power reported with a real activity file.
- ☐ Fillers and decaps inserted *after* the final ECO.
- ☐ GDS streamed out and re-imported as a sanity check.
- ☐ Metrics table complete; every number explainable.

23.8 The thirty-minutes-left checklist

- ☐ Is a clean, verified design already submitted? If not, do that now and stop everything else.
- ☐ Are the reports saved and readable?
- ☐ Is the metrics table filled in?
- ☐ Can you explain, in one sentence each, the three biggest decisions you made?
- ☐ Have you backed up?

CHAPTER 24

Formula and Concept Cheat Sheet

24.1 Timing

$$\begin{aligned}
 S_{setup} &= T + T_{skew} - T_{setup} - T_{unc} - T_{cq} - T_{comb} \\
 S_{hold} &= T_{cq,min} + T_{comb,min} - T_{skew} - T_{hold} - T_{unc,hold} \\
 f_{max} &= \frac{1}{T_{cq} + T_{comb} + T_{setup} + T_{unc} - T_{skew}} \\
 TNS &= \sum_{\text{endpoints}} \min(0, S_i) \\
 \delta_{opt} &= \frac{S_2 - S_1}{2} \quad (\text{two-stage useful skew})
 \end{aligned}$$

24.2 Interconnect

$$\begin{aligned}
 R_{wire} &= R_{sheet} \frac{L}{W}, & C_{wire} &= c L \\
 t_{lumped} &= 0.69 RC, & t_{distributed} &= 0.38 RC \\
 T_{Elmore,i} &= \sum_k R_{ik} C_k \\
 k_{opt} &= L \sqrt{\frac{0.345 rc}{t_b + 0.69 R_b C_b}} \\
 C_{eff} &= C_{gnd} + k_{MF} C_c, & k_{MF} &\in \{0, 1, 2\} \\
 HPWL &= (x_{max} - x_{min}) + (y_{max} - y_{min})
 \end{aligned}$$

24.3 Logical effort

$$\begin{aligned}
 d &= gh + p, & h &= C_{out}/C_{in} \\
 F &= GBH = \left(\prod g_i\right) \left(\prod b_i\right) \frac{C_{out}}{C_{in}} \\
 \hat{f} &= F^{1/N}, & D_{min} &= NF^{1/N} + P \\
 C_{in,i} &= \frac{g_i C_{out,i}}{\hat{f}} \quad (\text{size backwards from the load}) \\
 \hat{f}_{best} &\approx 3.6\text{--}4
 \end{aligned}$$

g : INV 1; NAND2 4/3; NOR2 5/3; NAND3 5/3; NOR3 7/3; XOR2 4; MUX2 2.
 p : INV 1; NAND n n ; NOR n n ; XOR2 4.

24.4 Power

$$\begin{aligned}
 P_{dyn} &= \alpha C_L V_{DD}^2 f \\
 P_{leak} &= I_{leak} V_{DD}, \quad I_{sub} \propto e^{-qV_{th}/nkT} \\
 P_{total} &= P_{dyn} + P_{sc} + P_{leak} \\
 \Delta P_V &= 1 - \left(\frac{V_{new}}{V_{old}} \right)^2 \quad (\text{dynamic}) \\
 C_{decap} &\geq \frac{C_{sw}}{\delta} \quad (\delta = \text{allowed droop fraction}) \\
 R_{sw} &\leq \frac{\Delta V_{allowed}}{I_{peak}} \quad (\text{power gate sizing})
 \end{aligned}$$

24.5 Physical / floorplan

$$\begin{aligned}
 U &= \frac{A_{std} + A_{macro}}{A_{core}}, \quad A_{core} = \frac{A_{std} + A_{macro}}{U} \\
 W_{core} &= \sqrt{A_{core}/AR}, \quad H_{core} = AR \cdot W_{core} \\
 \text{Tracks} &= \frac{\text{gcell dimension}}{\text{pitch}} \times (1 - \text{blockage}) \\
 \text{Overflow} &= \max(0, \text{Demand} - \text{Supply}) \\
 V_{drop} &= IR \quad (\text{static}), \quad V_{drop} = IR + L \frac{dI}{dt} \quad (\text{dynamic}) \\
 \text{AR}_{antenna} &= \frac{A_{metal}}{A_{gate}} \\
 J &= \frac{I}{Wt}, \quad \text{MTTF} = \frac{A}{J^n} e^{E_a/kT}
 \end{aligned}$$

24.6 Algorithms

$$\begin{aligned}
 \text{Zero-skew tap: } x &= \frac{(t_2 - t_1) + rL \left(\frac{cL}{2} + C_2 \right)}{r(cL + C_1 + C_2)} \\
 \text{Quadratic: } \Phi &= \frac{1}{2} \sum_{ij} w_{ij} [(x_i - x_j)^2 + (y_i - y_j)^2] \Rightarrow \mathbf{Ax} = \mathbf{b} \\
 \text{Smooth HPWL: } \gamma &\left[\log \sum e^{x_i/\gamma} + \log \sum e^{-x_i/\gamma} \right] \\
 \text{Steiner bound: } \frac{2}{3} \text{RMST} &\leq \text{RSMT} \leq \text{RMST}
 \end{aligned}$$

24.7 Quick reference numbers

Utilization sweet spot	65–75% for general logic
Clock insertion delay	< 25–30% of clock period
Local skew target	5–10% of clock period
Clock transition	< 8–10% of period
Static IR budget	1–2% of V_{DD}
Dynamic IR budget	4–5% of V_{DD}
Global route overflow	0%; < 0.3% tolerable
Multi-cut via conversion	> 95%
Clock power share	20–40% of total
Leakage share	10–30% depending on node and V_t mix
Optimal stage effort	≈ 4
Buffer pitch	tens to a few hundred μm ; compute it
LVT vs HVT delay ratio	$\sim 1.8\text{--}2.5\times$
LVT vs HVT leakage ratio	$\sim 20\text{--}50\times$
Downsizing area saving	$\sim 30\%$ per step

24.8 Decision heuristics

If you see...	Do...
Broad, shallow TNS	Check clock uncertainty, CRPR, derate before touching gates
One huge violation	Find the single net or cell; likely an unbuffered long net
Congestion in one region	Partial placement blockage, 30–50%
Congestion everywhere	Reduce utilization or fix the floorplan
Layer utilization imbalanced	Adjust global-route layer adjustment; promote nets
Hold violations after CTS	Normal; HVT and downsize before delay cells
Hold violations before CTS	Ignore — fix after CTS
DRC in the thousands	Find the one systematic cause; do not fix individually
Power over budget	Clock gating first, then multi- V_t , then downsizing
Area over budget	Downsize with a guard-band, then <i>shrink the core</i>
Everything failing at once	Utilization or floorplan; go upstream
Timing closes pre-route, fails post-route	Congestion causing detours

CHAPTER 25

A Four-Week Preparation Plan

25.1 Week 1 — Foundations and a working flow

Day	Study	Do
1	Ch. 1–2: flow, cells, libraries	Install OpenLane + SkyWater PDK
2	Ch. 3: STA, setup/hold, corners	Run the OpenLane example design end to end
3	Ch. 3 continued: OCV, CRPR, exceptions	Read every report the flow produced; find the twelve metrics
4	Ch. 4: interconnect, Elmore, buffering	Work P1–P10 on paper
5	Ch. 5–6: import, floorplan	Change utilization; observe every metric
6	Ch. 7: power planning, IR, EM	Change the PDN pitch; observe congestion and IR
7	Review	Re-work any Tier 1 problem you got wrong

25.2 Week 2 — The flow in depth

Day	Study	Do
8	Ch. 8: placement, HPWL, transforms	Run with three placement densities
9	Ch. 8: optimization moves	Manually apply a V_t swap ECO and measure
10	Ch. 9: CTS, skew, useful skew	Sweep the CTS skew target
11	Ch. 9: clock gating	Add/remove clock gating; measure power
12	Ch. 10: routing, antenna, crosstalk	Read the DRC report; classify the violations
13	Ch. 11: sign-off, ECO	Run a full sign-off; record everything
14	Review	Work P11–P16

25.3 Week 3 — Optimization

Day	Study	Do
15	Ch. 12: area	Run an area-recovery pass; shrink the core
16	Ch. 13: timing, logical effort	Hand-size a path; compare to the tool
17	Ch. 13: fix ordering, TNS diagnosis	Deliberately break timing; diagnose from the report alone
18	Ch. 14: power, multi- V_t	Build your own V_t mix; compare to the tool's
19	Ch. 15: congestion	Deliberately over-utilize; fix with blockages
20	Ch. 16–17: SI, trade-offs, Pareto	Build a Pareto plot from your sweeps
21	Review	Work P17–P22

25.4 Week 4 — Algorithms, speed, and rehearsal

Day	Study	Do
22	Ch. 17: partitioning, floorplan reps	Work Examples 17.1–17.3
23	Ch. 17: placement and routing algorithms	Work Examples 17.4–17.6
24	Ch. 17: DME, van Ginneken	Work Example 17.7
25	Ch. 18–19: tools, reports	Write your own sweep script
26	Ch. 20: Tier 3 problems	Work P23–P28
27	Ch. 20 continued	Work P29–P32
28	Full rehearsal	Give yourself an unseen design and 8 hours. Follow the P32 plan exactly. Produce a submission and a write-up.

The single most valuable day

Day 28. A full-length timed rehearsal will expose every gap in your process — scripts that do not exist, tools you have not installed, reports you cannot read, decisions you cannot make quickly. Every one of those is cheap to fix in rehearsal and catastrophic to discover on the day.

25.5 If you are on a team

Split by *stage ownership*, not by task:

- **Person A — Front end and constraints.** Netlist sanity, SDC, clock definitions, exceptions, corner setup. Owns the answer to “is this violation real?”
- **Person B — Floorplan and power.** Owns die size, macro placement, PDN, IR/EM. Owns the biggest single lever in the flow.
- **Person C — Place, CTS, route.** Owns the run-to-run loop and the sweep harness.

- **Person D — Sign-off and scoring.** Owns the metrics table, the Pareto analysis, verification, and the submission. **Has veto power on the freeze deadline.**

Caution

Give one person — and only one — the authority to call the freeze. Teams without this rule optimize until the deadline and submit nothing.

Glossary

Term	Definition
AOCV	Advanced On-Chip Variation; depth- and distance-dependent derating.
Antenna ratio	Ratio of connected metal area to connected gate oxide area.
Aspect ratio	Core height divided by core width.
B*-tree	Ordered binary tree floorplan representation.
Blockage	Region where placement or routing is forbidden.
CCD / CCOpt	Concurrent Clock and Data optimization.
Clock gating	Stopping the clock to idle registers to save dynamic power.
Cloning	Duplicating a driver to split its fanout.
CRPR / CPPR	Clock Reconvergence (Common Path) Pessimism Removal.
CTS	Clock Tree Synthesis.
Decap	Decoupling capacitor cell; local charge reservoir.
DEF	Design Exchange Format; the design's physical state.
Derate	Multiplicative factor applied to delays to model variation.
DME	Deferred Merge Embedding; exact zero-skew tree construction.
DRC	Design Rule Check (geometric).
DRV	Design Rule Violation (electrical: transition, cap, fanout).
ECO	Engineering Change Order; a late surgical modification.
Electromigration	Metal atom transport under high current density.
Elmore delay	First-moment approximation of RC tree delay.
Endcap	Boundary cell satisfying well/implant edge rules.
FLUTE	Fast Lookup Table Estimation of rectilinear Steiner trees.
FM	Fiduccia–Mattheyses partitioning heuristic.
Gcell	Global routing cell; the unit of congestion measurement.
Halo	Keep-out margin around a macro.
HFNS	High-Fanout Net Synthesis.
HPWL	Half-Perimeter Wire Length.
ICG	Integrated Clock Gating cell.
Insertion delay	Clock source to clock pin latency.
IR drop	Supply voltage lost across grid resistance.

Term	Definition
LEF	Library Exchange Format; physical abstract of cells and technology.
Legalization	Snapping cells to legal row/site positions without overlap.
Liberty (.lib)	Timing, power, and constraint characterization file.
Logical effort	Framework expressing delay as $gh + p$.
LVS	Layout Versus Schematic.
Miller factor	Multiplier on coupling capacitance based on aggressor behaviour.
MMMC	Multi-Mode Multi-Corner analysis.
Multi- V_t	Using cells of several threshold voltages in one design.
NDR	Non-Default Rule (wider/spaced routing).
NLDM	Non-Linear Delay Model; 2-D lookup tables.
OCV	On-Chip Variation.
Overflow	Routing demand exceeding supply on a gcell edge.
Pareto front	Set of non-dominated design points.
PathFinder	Negotiated-congestion routing algorithm.
POCV / SOCV	Parametric/Statistical OCV; per-arc σ .
Power gating	Cutting supply to an idle block via sleep transistors.
Π -model	$C/2 - R - C/2$ lumped wire model.
Retention flop	Flop that preserves state through power-down.
RSMT	Rectilinear Steiner Minimal Tree.
SDC	Synopsys Design Constraints.
Sequence pair	Two-permutation floorplan representation.
Skew	Difference in clock arrival between two flops.
Slack	Required time minus arrival time.
SPEF	Standard Parasitic Exchange Format.
STA	Static Timing Analysis.
Stacking effect	Reduced leakage in series off-transistors.
Tap cell	Well/substrate tie preventing latch-up.
TNS	Total Negative Slack.
Useful skew	Deliberate skew used to borrow slack between stages.
UPF	Unified Power Format.
Utilization	Placed area divided by core area.
Van Ginneken	Dynamic-programming optimal buffer insertion.
V_t swap	Replacing a cell with a different threshold flavour.
WNS	Worst Negative Slack.

Closing Note

Physical design optimization looks, from outside, like a very large collection of tricks. It is not. Nearly everything in this book reduces to four ideas:

1. **Delay is load over drive.** Every timing transform reduces load, increases drive, or splits the load.
2. **Everything is a trade.** There is no free improvement except the removal of pessimism and the correction of mistakes — which is precisely why you check those first.
3. **Decisions made early dominate.** A floorplan choice at hour one outweighs a thousand ECOs at hour ten.
4. **Every algorithm in the flow is a heuristic.** So sweep, measure, and let the data choose — do not argue with the tool from intuition.

If you internalize those four and can do the arithmetic in Chapter 24 without looking it up, you can reason your way through any problem a competition puts in front of you, including ones this book did not anticipate.

Build a baseline early. Sweep broadly. Measure everything. Freeze on time.

